The Linux Kernel Module Programming Guide

Peter Jay Salzman, Michael Burian, Ori Pomerantz, Bob Mottram, Jim Huang

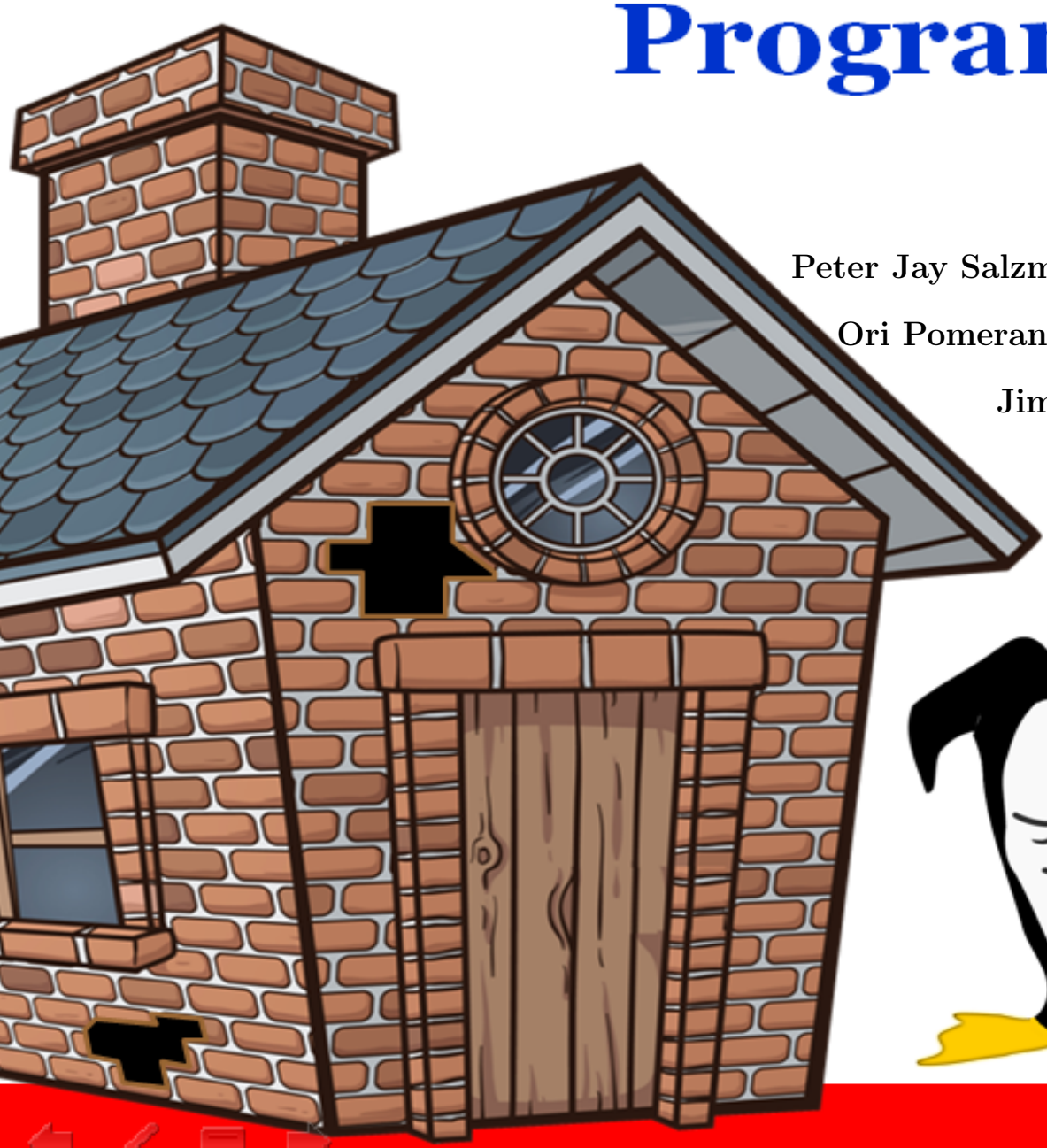
September 7, 2026

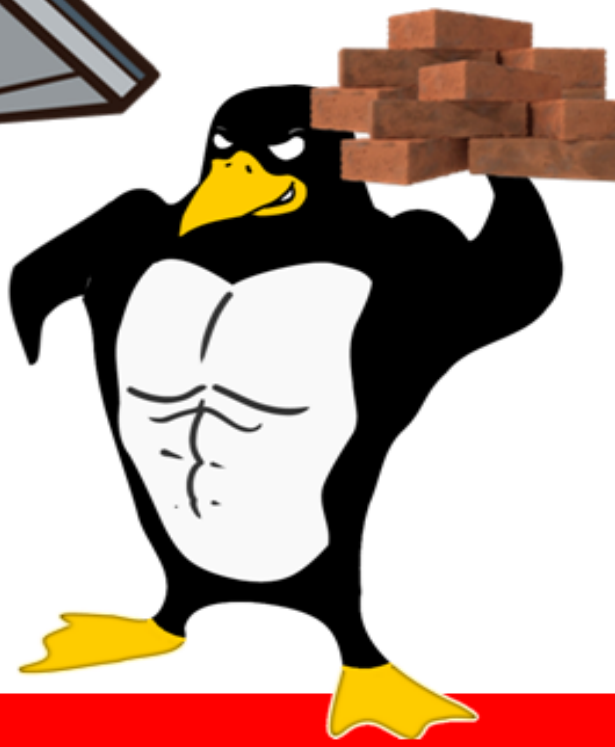
The Linux Kernel Module Programming Guide

Peter Jay Salzman, Michael Burian,

Ori Pomerantz, Bob Mottram,

Jim Huang





Contents

1 Getting Started **6**

1 Introduction . . . 6
- 1.1 Authorship . . . 7
- 1.2 Acknowledgements . . . 7
- 1.3 What Is A Kernel Module? . . . 7
- 1.4 Kernel module package . . . 8
- 1.5 What Modules are in my Kernel? . . . 8
- 1.6 Is there a need to download and compile the kernel? . . . 9
- 1.7 Before We Begin . . . 9
- 1.8 QEMU-Based Development Environment . . . 11

2 Kernel Headers . . . 12

3 Examples . . . 13

4 Hello World . . . 13
- 4.1 The Simplest Module . . . 13
- 4.2 Hello and Goodbye . . . 19
- 4.3 The __init and __exit Macros . . . 20
- 4.4 Licensing and Module Documentation . . . 21
- 4.5 Passing Command Line Arguments to a Module . . . 22
- 4.6 Modules Spanning Multiple Files . . . 27
- 4.7 Building modules for a precompiled kernel . . . 28

2 Core Kernel Interfaces **31**

5 Preliminaries . . . 31
- 5.1 How modules begin and end . . . 31
- 5.2 Functions available to modules . . . 33
- 5.3 User Space vs Kernel Space . . . 34
- 5.4 Name Space . . . 35

5.5 Code space . . . 35
5.6 Device Drivers . . . 36
6 Character Device drivers . . . 38
6.1 The file_operations Structure . . . 38
6.2 The file structure . . . 40
6.3 Registering A Device . . . 40
6.4 Unregistering A Device . . . 42
6.5 chardev.c . . . 43
6.6 Writing Modules for Multiple Kernel Versions . . . 46
7 The /proc Filesystem . . . 46
7.1 The proc_ops Structure . . . 49
7.2 Read and Write a /proc File . . . 49
7.3 Manage /proc file metadata and access hooks . . . 51
7.4 Manage /proc file with seq_file . . . 54
8 sysfs: Interacting with your module . . . 56
9 debugfs: Flexible debugging interfaces . . . 60
10 Talking To Device Files . . . 63
10.1 poll, select, and epoll . . . 75
10.2 Asynchronous notification . . . 75

3 Execution Context and Control Flow 77
11 System Calls . . . 77
12 Blocking Processes and threads . . . 87
12.1 Sleep . . . 87
12.2 Completions . . . 94
13 Synchronization . . . 96
13.1 Mutex . . . 96
13.2 Spinlocks . . . 97
13.3 Read and write locks . . . 100
13.4 Atomic operations . . . 102
13.5 Read-Copy-Update (RCU) . . . 104
14 Replacing Print Macros . . . 106
14.1 Replacement . . . 106
14.2 Flashing keyboard LEDs . . . 108

4 Memory, Time, and Data Movement 112
15 Timekeeping and Deferred Execution . . . 112
16 Memory Allocation and Ownership . . . 113
17 Copying Data Across the User-Kernel Boundary . . . 115
18 Memory-Mapped I/O and Register Access . . . 116
19 DMA and mmap . . . 116
19.1 DMA Addressing and Ownership Rules . . . 117
19.2 Scatter-Gather and DMA API Patterns . . . 117
19.3 DMA Debugging and Common Mistakes . . . 118
20 Modern Interface Choices for Linux 5.10 and Later . . . 121

5 Hardware Integration and Advanced Topics **123**
21 GPIO . . . 123
21.1 GPIO . . . 123
21.2 Control the LED's on/off state . . . 124
21.3 DHT11 sensor . . . 128
22 Scheduling Tasks . . . 134
22.1 Tasklets . . . 134
22.2 Work queues . . . 135
23 Interrupt Handlers . . . 136
23.1 Interrupt Handlers . . . 136
23.2 Detecting button presses . . . 138
23.3 Bottom Half . . . 142
23.4 Threaded IRQ . . . 147
24 Virtual Input Device Driver . . . 153
25 PCI Drivers . . . 165
26 USB Drivers . . . 166
26.1 USB Device Model Basics . . . 166
26.2 Endpoints, Pipes, and URBs . . . 167
26.3 USB and Userspace Interfaces . . . 167
26.4 Hotplug, Suspend, and Failure Modes . . . 168
27 Block Drivers . . . 169
27.1 The Modern Block Stack . . . 169
27.2 BIOs, Requests, and Completion . . . 170
27.3 Queue Limits, Flushes, and Reality . . . 170
28 Network Drivers . . . 175
28.1 net_device and netdev_ops . . . 176
28.2 Socket Buffers and Receive Paths . . . 176
28.3 NAPI, Interrupt Moderation, and Throughput . . . 176
28.4 Offloads, Link State, and Userspace . . . 177
29 Standardizing the interfaces: The Device Model . . . 180
30 Device Tree . . . 182
30.1 Introduction to Device Tree . . . 182
30.2 Device Tree and Kernel Modules . . . 182
30.3 Example: Device Tree Module . . . 183
30.4 Device Tree Source Example . . . 185
30.5 Testing Device Tree Modules . . . 186
30.6 Common Device Tree Functions . . . 186
31 Optimizations . . . 187
31.1 Likely and Unlikely conditions . . . 187
31.2 Static keys . . . 187
32 Common Pitfalls . . . 192
32.1 Using standard libraries . . . 192
32.2 Disabling interrupts . . . 192
32.3 Kernel stack is small . . . 192
32.4 No floating point in the kernel . . . 193
32.5 Initialize buffers before copying to user space . . . 193

32.6 Double-underscore functions are internal 193
33 Secure Boot Signing Guide 193
33.1 Install Dependencies 193
33.2 Generate a MOK Key Pair 194
33.3 Enroll the Key with MOKutil 194
33.4 Complete Enrollment in UEFI 194
33.5 Sign Your Kernel Module 195
34 Where To Go From Here? 195

1 Getting Started

This chapter introduces the guide, outlines the basic development environment, and builds up from the first loadable module. The emphasis is on getting a working workflow in place before moving on to the broader kernel interfaces used by more capable modules.

1 Introduction

The Linux Kernel Module Programming Guide is a free book; you may reproduce or modify it under the terms of the Open Software License, version 3.0.

This book is distributed in the hope that it would be useful, but without any warranty, without even the implied warranty of merchantability or fitness for a particular purpose.

The author encourages wide distribution of this book for personal or commercial use, provided the above copyright notice remains intact and the method adheres to the provisions of the Open Software License. In summary, you may copy and distribute this book free of charge or for a profit. No explicit permission is required from the author for reproduction of this book in any medium, physical or electronic.

Derivative works and translations of this document must be placed under the Open Software License, and the original copyright notice must remain intact. If you have contributed new material to this book, you must make the material and source code available for your revisions. Please make revisions and updates available directly to the document maintainer, Ching-Chun (Jim) Huang <jserv@ccns.ncku.edu.tw>. This will allow for the merging of updates and provide consistent revisions to the Linux community.

If you publish or distribute this book commercially, donations, royalties, or printed copies are greatly appreciated by the author and the Linux Documentation Project (LDP). Contributing in this way shows your support for free

software and the LDP. If you have questions or comments, please contact the address above.

1.1 Authorship

The Linux Kernel Module Programming Guide was initially authored by Ori Pomerantz for Linux v2.2. As the Linux kernel evolved, Ori's availability to maintain the document diminished. Consequently, Peter Jay Salzman assumed the role of maintainer and updated the guide for Linux v2.4. Similar constraints arose for Peter when tracking developments in Linux v2.6, leading to Michael Burian joining as a co-maintainer to bring the guide up to speed with Linux v2.6. Bob Mottram contributed to the guide by updating examples for Linux v3.8 and later. Jim Huang then undertook the task of updating the guide for recent Linux versions (v5.0 and beyond), along with revising the LaTeX document. The guide now uses Linux v5.10 as its minimum supported baseline and aims to keep examples and guidance compatible across current long-term support kernels.

1.2 Acknowledgements

The following people have contributed corrections or good suggestions:

Amit Dhingra, Andrew Kreimer, Andrew Lin, Andy Shevchenko, Arush Sharma, Aykhan Hagverdili, Benno Bielmeier, Bob Lee, Brad Baker, Che-Chia Chang, Cheng-Shian Yeh, Cheng-Yang Chou, Chih-En Lin, Chih-Hsuan Yang, Chih-Yu Chen, Ching-Hua (Vivian) Lin, Chin Yik Ming, Chung-Han Tsai, cvvletter, Cyril Brulebois, Daniele Paolo Scarpazza, David Porter, demonsome, Dimo Velev, Ekang Monyet, Ethan Chan, Francois Audeon, Gilad Reti, Hao.Dong, heartofrain, Horst Schirmeier, Hsin-Hsiang Peng, Hung-Jen Pao, Ignacio Martin, I-Hsin Cheng, Integral, Iûnn Kiàn-îng, Jian-Xing Wu, Jimmy Ma, Johan Calle, keytouch, Kohei Otsuka, Kuan-Wei Chiu, manbing, Marconi Jiang, mengxinayan, Meng-Zong Tsai, Peter Lin, Roman Lakeev, Sam Erickson, Shao-Tse Hung, Shih-Sheng Yang, Stacy Prowell, Steven Lung, Tristan Lelong, Tse-Wei Lin, Tucker Polomik, Tyler Fanelli, VxTeemo, Wei-Hsin Yeh, Wei-Lun Tsai, Xatierlike Lee, Yan-Jie Chan, Yen-Yu Chen, Yin-Chiuan Chen, Yi-Wei Lin, Yo-Jung Lin, Yu-Chun Lin, Yu-Hsiang Tseng, YYGO.

1.3 What Is A Kernel Module?

Involvement in the development of Linux kernel modules requires a foundation in the C programming language and a track record of creating conventional programs intended for process execution. This pursuit delves into a domain where an unregulated pointer, if disregarded, may potentially trigger the total elimination of an entire filesystem, resulting in a scenario that necessitates a complete system reboot.

A Linux kernel module is precisely defined as a code segment capable of dynamic loading and unloading within the kernel as needed. These modules enhance kernel capabilities without necessitating a system reboot. A notable example is seen in the device driver module, which facilitates kernel interaction with hardware components linked to the system. In the absence of modules, the prevailing approach leans toward monolithic kernels, requiring direct integration of new functionalities into the kernel image. This approach leads to larger kernels and necessitates kernel rebuilding and subsequent system rebooting when new functionalities are desired.

1.4 Kernel module package

Linux distributions provide the commands `modprobe`, `insmod` and `depmod` within a package.

On Ubuntu/Debian GNU/Linux:

```
sudo apt-get install build-essential kmod
```

On Arch Linux:

```
sudo pacman -S gcc kmod
```

1.5 What Modules are in my Kernel?

To discover what modules are already loaded within your current kernel, use the command `lsmod`.

```
lsmod
```

Modules are stored within the file `/proc/modules`, so you can also see them with:

```
cat /proc/modules
```

This can be a long list, and you might prefer to search for something particular. To search for the `fat` module:

```
lsmod | grep fat
```

1.6 Is there a need to download and compile the kernel?

To effectively follow this guide, there is no obligatory requirement for performing such actions. Nonetheless, a prudent approach involves executing the examples within an isolated environment, thus mitigating any potential risk of disrupting the system.

This guide includes a `devtools/` directory that automates kernel module testing inside a QEMU virtual machine. It downloads a kernel source tree and a minimal BusyBox root filesystem, builds both, and boots the result in QEMU with the host `examples/` directory shared into the guest via the 9p virtfs protocol. You edit source files on your host and `insmod` the compiled modules in the guest—no disk images to manage, no root filesystem to rebuild. See Section 1.8 for the complete setup procedure.

1.7 Before We Begin

Before delving into code, certain matters require attention. Variances exist among individuals' systems, and distinct personal approaches are evident. The achievement of successful compilation and loading of the inaugural "hello world" program may, at times, present challenges. It is reassuring to note that overcoming the initial obstacle on the first attempt paves the way for subsequent endeavors to proceed seamlessly.

1. Modversioning. A module compiled for one kernel will not load if a different kernel is booted, unless `CONFIG_MODVERSIONS` is enabled in the kernel. Module versioning will be discussed later in this guide. Until module versioning is covered, the examples in this guide may not work correctly if running a kernel with modversioning turned on. However, most stock Linux distribution kernels come with modversioning enabled. If difficulties arise when loading the modules due to versioning errors, consider compiling a kernel with modversioning turned off.

2. Using the X Window System. It is highly recommended to extract, compile, and load all the examples discussed in this guide from a console. Working on these tasks within the X Window System is discouraged.

   Modules cannot directly print to the screen like `printf()` can, but they can log information and warnings to the kernel's log ring buffer. This output is *not* automatically displayed on any console or terminal. To view kernel module messages, you must use `dmesg` to read the kernel log ring buffer, or check the systemd journal with `journalctl -k` for kernel messages. Refer to Section 4 for more information. The terminal or environment from which you load the module does not affect where the output goes—it always goes to the kernel log.

3. SecureBoot. Numerous modern computers arrive pre-configured with UEFI SecureBoot enabled—an essential security standard ensuring booting exclusively through trusted software endorsed by the original equipment

manufacturer. Certain Linux distributions even ship with the default Linux kernel configured to support SecureBoot. In these cases, the kernel module necessitates a signed security key.

Failing that, an attempt to insert your first "hello world" module would result in the message: "*ERROR: could not insert module*". If this message "*Lockdown: insmod: unsigned module loading is restricted; see man kernel lockdown.7*" appears in the `dmesg` output, the simplest approach involves disabling UEFI SecureBoot from the boot menu of your PC or laptop, allowing the successful insertion of the "hello world" module. Naturally, an alternative involves undergoing intricate procedures such as generating keys, system key installation, and module signing to achieve functionality. However, this intricate process is less appropriate for beginners. If interested, more detailed steps for SecureBoot can be explored and followed. In the real world—especailly in enterprise environment, production servers, or laptops where your IT staffs mandate strict security—it is not eligible to disable SecureBoot. If you ever encounter such situations, visit Section 33 for a SecureBoot signing thorough guide.

If you are building your own kernel for module development, these `.config` options are useful for debugging and development:

- `CONFIG_DEBUG_INFO`: include debug symbols so `gdb`, `addr2line`, and `objdump` can map crash addresses to source lines.
- `CONFIG_DYNAMIC_DEBUG`: enable runtime-switchable `pr_debug()` statements (see Item 2).
- `CONFIG_KASAN` (Kernel Address Sanitizer): instruments memory accesses to detect out-of-bounds, use-after-free, and other memory errors at the cost of roughly 2× memory usage and some CPU overhead.
- `CONFIG_LOCKDEP` (lock dependency checker): detects potential deadlocks (lock order inversions, sleeping under spinlocks, wrong lock type for context) at runtime, before they actually hang the system.
- `CONFIG_DEBUG_ATOMIC_SLEEP`: flags attempts to sleep in atomic context, catching the most common spinlock misuse.
- `CONFIG_MODULE_FORCE_UNLOAD`: allows `rmmod -f` as a last resort during development, but use it with care because force-unloading can hide lifetime bugs and leave the kernel in an inconsistent state.

The QEMU-based environment described below already enables `CONFIG_DEBUG_INFO` and `CONFIG_MODULE_FORCE_UNLOAD`; enable the others explicitly if you want those extra diagnostics.

1.8 QEMU-Based Development Environment

An alternative to installing kernel headers on your host and loading modules into your running kernel is to use the QEMU-based development environment provided under `devtools/`. This approach is especially useful on machines where you cannot (or prefer not to) load arbitrary kernel modules (for example, hosts with SecureBoot enabled, corporate laptops, or macOS workstations).

The setup compiles a minimal Linux kernel with module, debugfs, procfs, and 9p filesystem support, along with a statically linked BusyBox initramfs. QEMU boots this kernel and shares the project directory into the guest via the 9p virtfs protocol, so every file you edit on the host is immediately visible inside the virtual machine.

Prerequisites. The kernel and BusyBox builds require a Linux host (or a Linux environment such as Docker or Lima on macOS). QEMU itself runs natively on Linux and macOS.

```
# Debian/Ubuntu: install the build toolchain and QEMU
sudo apt-get install build-essential flex bison bc libelf-dev
↪  libssl-dev cpio qemu-system-x86

# macOS: install QEMU (kernel build runs inside Lima or Docker)
brew install qemu
```

One-time setup (downloads a kernel tarball and BusyBox, builds both, and packs an initramfs):

```
devtools/setup.sh
```

Building modules against the QEMU kernel (must be run on Linux):

```
devtools/build-modules.sh
```

Interactive testing. Boot QEMU and get a shell inside the guest. The project tree is mounted at `/mnt/lkmpg/`, with compiled modules at `/mnt/lkmpg/examples/*.ko`:

```
devtools/boot.sh
# Inside the guest:
insmod /mnt/lkmpg/examples/hello-1.ko
dmesg | tail
rmmod hello_1
```

Automated testing. Build all modules and run `insmod`/`rmmod` for every module in a single command. Modules listed in `.ci/non-working` are automatically skipped:

```
devtools/test-modules.sh
```

Kernel debugging with GDB. GDB requires the `vmlinux` image with debug symbols. Prebuilt setups omit it for size; rebuild from source first:

```
LKMPG_NO_PREBUILT=1 devtools/setup.sh
```

Then start QEMU paused and attach GDB from a second terminal:

```
# Terminal 1: QEMU waits for a debugger connection
devtools/boot.sh --gdb

# Terminal 2: connect GDB
gdb devtools/.cache/kernel-build/vmlinux \
    -ex "target remote :1234" \
    -ex "break do_init_module" \
    -ex continue
```

When you `insmod` a module in the guest, GDB will break at the module initialization entry point.

Overriding defaults. Create `devtools/config.local` (gitignored) to change the kernel version, QEMU memory size, or other settings:

```
KERNEL_VERSION="6.6.70"
QEMU_MEM="1G"
```

Rerun `devtools/setup.sh` after changing `KERNEL_VERSION`.

2 Kernel Headers

Before building anything, it is necessary to install the header files for the kernel. If you are using the QEMU-based environment described in Section 1.8, the kernel headers are built automatically by `devtools/setup.sh` and you can skip this step.

On Ubuntu/Debian GNU/Linux:

```
sudo apt-get update
apt-cache search linux-headers-`uname -r`
```

The following command provides information about the available kernel header files. Then, for example:

```
sudo apt-get install linux-headers-`uname -r`
```

On Arch Linux:

```
sudo pacman -S linux-headers
```

On Fedora:

```
sudo dnf install kernel-devel kernel-headers
```

3 Examples

All the examples from this document are available within the `examples` subdirectory.

To build all examples at once, run `make` from within the `examples/` directory (requires kernel headers installed, see Section 2), or use the QEMU-based environment:

```
devtools/build-modules.sh
```

To run automated `insmod`/`rmmod` tests for every module inside QEMU:

```
devtools/test-modules.sh
```

Should compile errors occur, it may be due to a more recent kernel version being in use, or there might be a need to install the corresponding kernel header files.

4 Hello World

4.1 The Simplest Module

Most individuals beginning their programming journey typically start with some variant of a *hello world* example. It is unclear what the outcomes are for those who deviate from this tradition, but it seems prudent to adhere to it. The learning process will begin with a series of hello world programs that illustrate various fundamental aspects of writing a kernel module.

Presented next is the simplest possible module.

Make a test directory:

```
mkdir -p ~/develop/kernel/hello-1
cd ~/develop/kernel/hello-1
```

Paste this into your favorite editor and save it as `hello-1.c`:

```c
/*
 * hello-1.c - The simplest kernel module.
 */
#include <linux/module.h> /* Needed by all modules */
#include <linux/printk.h> /* Needed for pr_info() */

int init_module(void)
{
    pr_info("Hello world 1.\n");

    /* A nonzero return means init_module failed; module can't be loaded. */
    return 0;
}

void cleanup_module(void)
{
    pr_info("Goodbye world 1.\n");
}

MODULE_LICENSE("GPL");
```

Now you will need a `Makefile`. If you copy and paste this, change the indentation to use *tabs*, not spaces.

```
obj-m += hello-1.o

PWD := $(CURDIR)

all:
	$(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
	$(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

In `Makefile`, `$(CURDIR)` can be set to the absolute pathname of the current working directory (after all `-C` options are processed, if any). See more about `CURDIR` in GNU make manual.

And finally, just run `make` directly.

```
make
```

If there is no `PWD := $(CURDIR)` statement in the Makefile, then it may not compile correctly with `sudo make`. This is because some environment variables

are specified by the security policy and cannot be inherited. The default security policy is `sudoers`. In the `sudoers` security policy, `env_reset` is enabled by default, which restricts environment variables. Specifically, path variables are not retained from the user environment; they are set to default values (for more information, see: sudoers manual). You can see the environment variable settings by:

```
$ sudo -s
# sudo -V
```

Here is a simple Makefile as an example to demonstrate the problem mentioned above.

```
all:
	echo $(PWD)
```

Then, we can use the `-p` flag to print out the environment variable values from the Makefile.

```
$ make -p | grep PWD
PWD = /home/ubuntu/temp
OLDPWD = /home/ubuntu
echo $(PWD)
```

The `PWD` variable will not be inherited with `sudo`.

```
$ sudo make -p | grep PWD
echo $(PWD)
```

However, there are three ways to solve this problem.

1. You can use the `-E` flag to temporarily preserve them.

```
$ sudo -E make -p | grep PWD
PWD = /home/ubuntu/temp
OLDPWD = /home/ubuntu
echo $(PWD)
```

2. You can disable `env_reset` by editing `/etc/sudoers` as root using `visudo`.

```
## sudoers file.
##
...
Defaults env_reset
## Change env_reset to !env_reset in previous line to keep all
↪  environment variables
```

Then execute `env` and `sudo env` individually.

```
# disable the env_reset
echo "user:" > non-env_reset.log; env >> non-env_reset.log
echo "root:" >> non-env_reset.log; sudo env >> non-env_reset.log
# enable the env_reset
echo "user:" > env_reset.log; env >> env_reset.log
echo "root:" >> env_reset.log; sudo env >> env_reset.log
```

You can view and compare these logs to find differences between `env_reset` and `!env_reset`.

3. You can preserve environment variables by appending them to `env_keep` in `/etc/sudoers`.

```
Defaults env_keep += "PWD"
```

After applying the above change, you can check the environment variable settings by:

```
$ sudo -s
# sudo -V
```

If all goes smoothly you should then find that you have a compiled `hello-1.ko` module. You can find info on it with the command:

```
modinfo hello-1.ko
```

At this point the command:

```
lsmod | grep hello
```

should return nothing. You can try loading your new module with:

```
sudo insmod hello-1.ko
```

The dash character will get converted to an underscore, so when you again try:

```
lsmod | grep hello
```

You should now see your loaded module. It can be removed again with:

```
sudo rmmod hello_1
```

Notice that the dash was replaced by an underscore. To see the module's output messages, use `dmesg` to view the kernel log ring buffer:

```
sudo dmesg | tail -10
```

You should see messages like "Hello world 1." and "Goodbye world 1." from your module. Alternatively, you can check the systemd journal for kernel messages:

```
journalctl --since "1 hour ago" | grep kernel
```

If you are using the QEMU environment from Section 1.8 instead, the same exercise looks like this:

```
devtools/build-modules.sh        # compile all examples
devtools/boot.sh                 # boot into the QEMU guest
# inside the guest:
insmod /mnt/lkmpg/examples/hello-1.ko
dmesg | tail
rmmod hello_1
```

No `sudo` is needed because the guest runs as root.

You now know the basics of creating, compiling, installing and removing modules. Now for more of a description of how this module works.

Kernel modules must have at least two functions: a "start" (initialization) function called `init_module()` which is called when the module is `insmod`ed into the kernel, and an "end" (cleanup) function called `cleanup_module()` which is called just before it is removed from the kernel. Actually, things have changed starting with kernel 2.3.13. You can now use whatever name you like for the start and end functions of a module, and you will learn how to do this in Section 4.2. In fact, the new method is the preferred method. The old `init_module()` and `cleanup_module()` have been deprecated for x86 systems with indirect branch tracking (IBT) enabled starting from commit 4fab2d76 in kernel 6.15, causing build failures. However, many existing examples still use these names for their start and end functions.

Typically, `init_module()` either registers a handler for something with the kernel, or it replaces one of the kernel functions with its own code (usually code to do something and then call the original function). The `cleanup_module()` function is supposed to undo whatever `init_module()` did, so the module can be unloaded safely.

Lastly, every kernel module needs to include include/linux/module.h. We needed to include include/linux/printk.h only for the macro expansion for the `pr_alert()` log level, which you'll learn about in Item 2.

1. A point about coding style. Another thing that may not be immediately obvious to anyone getting started with kernel programming is that indentation within your code should use **tabs** and **not spaces**. It is one of the coding conventions of the kernel. You may not like it, but you will need to get used to it if you ever submit a patch upstream.

2. Introducing print macros. In the beginning there was `printk`, usually followed by a priority such as `KERN_INFO` or `KERN_DEBUG`. More recently, this can also be expressed in abbreviated form using a set of print macros, such as `pr_info` and `pr_debug`. This just saves some mindless keyboard bashing and looks a bit neater. They can be found within include/linux/printk.h. Take time to read through the available priority macros.

   Important: These functions write to the kernel log ring buffer, *not* directly to any terminal or console. To view the output from your kernel modules, you must use `dmesg` or `journalctl -k`.

   The kernel log buffer is a fixed-size circular buffer (sized via `CONFIG_LOG_BUF_SHIFT` in the kernel configuration). Excessive logging can overflow the buffer and push out earlier messages before they are read. In hot paths (interrupt handlers, per-packet processing, or tight loops), use the rate-limited variants `pr_info_ratelimited()`, `pr_warn_ratelimited()`, etc., or the explicit `printk_ratelimit()` guard, to avoid flooding the log and slowing down the system due to console output overhead. See include/linux/ratelimit.h for details.

   For development, `pr_debug()` is particularly useful. When the kernel is compiled with `CONFIG_DYNAMIC_DEBUG`, `pr_debug()` calls are compiled in but disabled by default. You can selectively enable them at runtime without recompiling or reloading the module by writing to `/sys/kernel/debug/dynamic_debug/control`:

   ```
   1 echo "module mymodule +p" >
     ↪ /sys/kernel/debug/dynamic_debug/control
   ```

   The `+p` flag activates printing; `-p` disables it. You can filter by file, function, or line number. See Documentation/admin-guide/dynamic-debug-howto.rst for the full syntax.

3. About Compiling. Kernel modules need to be compiled a bit differently from regular userspace apps. Former kernel versions required us to care much about these settings, which are usually stored in Makefiles. Although hierarchically organized, many redundant settings accumulated in sublevel Makefiles and made them large and rather difficult to maintain. Fortunately, there is a new way of doing these things, called kbuild, and the build process for external loadable modules is now fully integrated into the standard kernel build mechanism. To learn more about how to compile modules which are not part of the official kernel (such as all the examples you will find in this guide), see file Documentation/kbuild/modules.rst.

   Additional details about Makefiles for kernel modules are available in Documentation/kbuild/makefiles.rst. Be sure to read this and the related files before starting to hack Makefiles. It will probably save you lots of work.

   > Here is another exercise for the reader. See that comment above the return statement in `init_module()`? Change the return value to something negative, recompile and load the module again. What happens?

4.2 Hello and Goodbye

In early kernel versions you had to use the `init_module` and `cleanup_module` functions, as in the first hello world example, but these days you can name those anything you want by using the `module_init` and `module_exit` macros. These macros are defined in include/linux/module.h. The only requirement is that your init and cleanup functions must be defined before calling those macros, otherwise you will get compilation errors. Here is an example of this technique:

```
1  /*
2   * hello-2.c - Demonstrating the module_init() and module_exit() macros.
3   * This is preferred over using init_module() and cleanup_module().
4   */
5  #include <linux/init.h> /* Needed for the macros */
6  #include <linux/module.h> /* Needed by all modules */
7  #include <linux/printk.h> /* Needed for pr_info() */
8
9  static int __init hello_2_init(void)
10 {
11     pr_info("Hello, world 2\n");
12     return 0;
13 }
14
15 static void __exit hello_2_exit(void)
16 {
17     pr_info("Goodbye, world 2\n");
18 }
19
20 module_init(hello_2_init);
21 module_exit(hello_2_exit);
```

```
22
23  MODULE_LICENSE("GPL");
```

So now we have two real kernel modules under our belt. Adding another module is as simple as this:

```
1   obj-m += hello-1.o
2   obj-m += hello-2.o
3
4   PWD := $(CURDIR)
5
6   all:
7           $(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
8
9   clean:
10          $(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

Now have a look at drivers/char/Makefile for a real world example. As you can see, some things got hardwired into the kernel (`obj-y`) but where have all those `obj-m` gone? Those familiar with shell scripts will easily be able to spot them. For those who are not, the `obj-$(CONFIG_FOO)` entries you see everywhere expand into `obj-y` or `obj-m`, depending on whether the `CONFIG_FOO` variable has been set to `y` or `m`. While we are at it, those were exactly the kind of variables that you have set in the `.config` file in the top-level directory of the Linux kernel source tree, the last time you ran `make menuconfig` or something similar.

4.3 The __init and __exit Macros

The `__init` macro causes the init function to be discarded and its memory freed once the init function finishes for built-in drivers. For loadable modules, `__init` still places the function into an init-only section, `do_free_init` may still be observable during module loading even without `__init`. If you think about when the init function is invoked, this makes perfect sense.

There is also an `__initdata` which works similarly to `__init` but for init variables rather than functions.

The `__exit` macro causes the omission of the function when the module is built into the kernel, and like `__init`, has no effect for loadable modules. Again, if you consider when the cleanup function runs, this makes complete sense; built-in drivers do not need a cleanup function, while loadable modules do.

These macros are defined in include/linux/init.h and serve to free up kernel memory. When you boot your kernel and see something like Freeing unused kernel memory: 236k freed, this is precisely what the kernel is freeing.

```
1   /*
2    * hello-3.c - Illustrating the __init, __initdata and __exit macros.
```

```
 */
#include <linux/init.h> /* Needed for the macros */
#include <linux/module.h> /* Needed by all modules */
#include <linux/printk.h> /* Needed for pr_info() */

static int hello3_data __initdata = 3;

static int __init hello_3_init(void)
{
    pr_info("Hello, world %d\n", hello3_data);
    return 0;
}

static void __exit hello_3_exit(void)
{
    pr_info("Goodbye, world 3\n");
}

module_init(hello_3_init);
module_exit(hello_3_exit);

MODULE_LICENSE("GPL");
```

4.4 Licensing and Module Documentation

Honestly, who loads or even cares about proprietary modules? If you do then you might have seen something like this:

```
$ sudo insmod xxxxxx.ko
loading out-of-tree module taints kernel.
module license 'unspecified' taints kernel.
```

You can use a few macros to indicate the license for your module. Some examples are "GPL", "GPL v2", "GPL and additional rights", "Dual BSD/GPL", "Dual MIT/GPL", "Dual MPL/GPL" and "Proprietary". They are defined within include/linux/module.h.

To reference what license you are using, a macro is available called `MODULE_LICENSE`. This and a few other macros describing the module are illustrated in the example below.

```
/*
 * hello-4.c - Demonstrates module documentation.
 */
#include <linux/init.h> /* Needed for the macros */
#include <linux/module.h> /* Needed by all modules */
#include <linux/printk.h> /* Needed for pr_info() */

MODULE_LICENSE("GPL");
MODULE_AUTHOR("LKMPG");
MODULE_DESCRIPTION("A sample driver");

static int __init init_hello_4(void)
{
```

```
    pr_info("Hello, world 4\n");
    return 0;
}

static void __exit cleanup_hello_4(void)
{
    pr_info("Goodbye, world 4\n");
}

module_init(init_hello_4);
module_exit(cleanup_hello_4);
```

4.5 Passing Command Line Arguments to a Module

Modules can take command line arguments, but not with the argc/argv you might be used to.

To allow arguments to be passed to your module, declare the variables that will take the values of the command line arguments as global and then use the `module_param()` macro (defined in include/linux/moduleparam.h) to set the mechanism up. At runtime, `insmod` will fill the variables with any command line arguments that are given, like `insmod mymodule.ko myvariable=5`. The variable declarations and macros should be placed at the beginning of the module for clarity. The example code should clear up my admittedly lousy explanation.

The `module_param()` macro takes 3 arguments: the name of the variable, its type and permissions for the corresponding file in sysfs. Integer types can be signed as usual or unsigned. If you would like to use arrays of integers or strings, see `module_param_array()` and `module_param_string()`.

```
int myint = 3;
module_param(myint, int, 0);
```

Arrays are supported too, but things are a bit different now than they were in the olden days. To keep track of the number of parameters, you need to pass a pointer to a count variable as the third parameter. At your option, you could also ignore the count and pass `NULL` instead. We show both possibilities here:

```
int myintarray[2];
module_param_array(myintarray, int, NULL, 0); /* not interested in count */

short myshortarray[4];
int count;
module_param_array(myshortarray, short, &count, 0); /* put count into "count"
↪  variable */
```

A good use for this is to have the module variable's default values set, like a port or IO address. If the variables contain the default values, then perform

autodetection (explained elsewhere). Otherwise, keep the current value. This will be made clear later on.

Lastly, there is a macro function, `MODULE_PARM_DESC()`, that is used to document arguments that the module can take. It takes two parameters: a variable name and a free form string describing that variable.

```c
/*
 * hello-5.c - Demonstrates command line argument passing to a module.
 */
#include <linux/init.h>
#include <linux/kernel.h> /* for ARRAY_SIZE() */
#include <linux/module.h>
#include <linux/moduleparam.h>
#include <linux/printk.h>
#include <linux/stat.h>

MODULE_LICENSE("GPL");

static short int myshort = 1;
static int myint = 420;
static long int mylong = 9999;
static char *mystring = "blah";
static int myintarray[2] = { 420, 420 };
static int arr_argc = 0;

/* module_param(foo, int, 0000)
 * The first param is the parameter's name.
 * The second param is its data type.
 * The final argument is the permissions bits,
 * for exposing parameters in sysfs (if non-zero) at a later stage.
 */
module_param(myshort, short, S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP);
MODULE_PARM_DESC(myshort, "A short integer");
module_param(myint, int, S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);
MODULE_PARM_DESC(myint, "An integer");
module_param(mylong, long, S_IRUSR);
MODULE_PARM_DESC(mylong, "A long integer");
module_param(mystring, charp, 0000);
MODULE_PARM_DESC(mystring, "A character string");

/* module_param_array(name, type, num, perm);
 * The first param is the parameter's (in this case the array's) name.
 * The second param is the data type of the elements of the array.
 * The third argument is a pointer to the variable that will store the number
 * of elements of the array initialized by the user at module loading time.
 * The fourth argument is the permission bits.
 */
module_param_array(myintarray, int, &arr_argc, 0000);
MODULE_PARM_DESC(myintarray, "An array of integers");

static int __init hello_5_init(void)
{
    int i;

    pr_info("Hello, world 5\n=============\n");
    pr_info("myshort is a short integer: %hd\n", myshort);
```

```
    pr_info("myint is an integer: %d\n", myint);
    pr_info("mylong is a long integer: %ld\n", mylong);
    pr_info("mystring is a string: %s\n", mystring);

    for (i = 0; i < ARRAY_SIZE(myintarray); i++)
        pr_info("myintarray[%d] = %d\n", i, myintarray[i]);

    pr_info("got %d arguments for myintarray.\n", arr_argc);
    return 0;
}

static void __exit hello_5_exit(void)
{
    pr_info("Goodbye, world 5\n");
}

module_init(hello_5_init);
module_exit(hello_5_exit);
```

If you need to validate a parameter or react whenever it changes, use `module_param_cb()`. This variant takes four arguments: the parameter name, a pointer to a `struct kernel_param_ops`, an argument pointer (typically the address of the backing variable), and the sysfs permissions. The callbacks stored in `struct kernel_param_ops` let you override the `set` and `get` operations instead of relying on the default helpers such as `param_set_int()` and `param_get_int()`. With non-zero permissions, the parameter is also exposed through sysfs so the same callbacks can run after the module has already been loaded.

```
static int watched = 42;

static int watched_set(const char *val, const struct kernel_param *kp)
{
    int ret = param_set_int(val, kp);

    if (ret)
        return ret;

    pr_info("watched updated to %d\n", watched);
    return 0;
}

static const struct kernel_param_ops watched_ops = {
    .set = watched_set,
    .get = param_get_int,
};

module_param_cb(watched, &watched_ops, &watched, 0644);
```

The full example below also overrides `.get` so that every sysfs read is logged:

```
/*
 * hello-6.c - Demonstrates module_param_cb() callbacks.
 */
```

```c
#include <linux/init.h>
#include <linux/module.h>
#include <linux/moduleparam.h>
#include <linux/printk.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("LKMPG");
MODULE_DESCRIPTION("A module_param_cb() example");

static int watched = 42;

static int watched_set(const char *val, const struct kernel_param *kp)
{
    int ret;

    ret = param_set_int(val, kp);
    if (ret)
        return ret;

    pr_info("watched updated to %d\n", watched);
    return 0;
}

static int watched_get(char *buffer, const struct kernel_param *kp)
{
    int ret;

    ret = param_get_int(buffer, kp);
    if (ret >= 0)
        pr_info("watched was read\n");

    return ret;
}

static const struct kernel_param_ops watched_ops = {
    .set = watched_set,
    .get = watched_get,
};

module_param_cb(watched, &watched_ops, &watched, 0644);
MODULE_PARM_DESC(watched, "An integer that logs every update");

static int __init hello_6_init(void)
{
    pr_info("Hello, world 6\n");
    pr_info("watched starts at %d\n", watched);
    return 0;
}

static void __exit hello_6_exit(void)
{
    pr_info("Goodbye, world 6\n");
}

module_init(hello_6_init);
module_exit(hello_6_exit);
```

It is recommended to experiment with the following code:

```
$ sudo insmod hello-5.ko mystring="bebop" myintarray=-1
$ sudo dmesg -t | tail -7
myshort is a short integer: 1
myint is an integer: 420
mylong is a long integer: 9999
mystring is a string: bebop
myintarray[0] = -1
myintarray[1] = 420
got 1 arguments for myintarray.

$ sudo rmmod hello-5
$ sudo dmesg -t | tail -1
Goodbye, world 5

$ sudo insmod hello-5.ko mystring="supercalifragilisticexpialidocious" myintarray=-1,-1
$ sudo dmesg -t | tail -7
myshort is a short integer: 1
myint is an integer: 420
mylong is a long integer: 9999
mystring is a string: supercalifragilisticexpialidocious
myintarray[0] = -1
myintarray[1] = -1
got 2 arguments for myintarray.

$ sudo rmmod hello-5
$ sudo dmesg -t | tail -1
Goodbye, world 5

$ sudo insmod hello-5.ko mylong=hello
insmod: ERROR: could not insert module hello-5.ko: Invalid parameters

$ sudo insmod hello-6.ko watched=7
$ sudo dmesg -t | tail -3
watched updated to 7
Hello, world 6
watched starts at 7

$ cat /sys/module/hello_6/parameters/watched
7
$ sudo dmesg -t | tail -1
watched was read

$ echo 21 | sudo tee /sys/module/hello_6/parameters/watched
21
$ sudo dmesg -t | tail -1
```

```
watched updated to 21

$ sudo rmmod hello-6
```

4.6 Modules Spanning Multiple Files

Sometimes it makes sense to divide a kernel module between several source files.

Here is an example of such a kernel module.

```c
/*
 * start.c - Illustration of multi filed modules
 */

#include <linux/kernel.h> /* We are doing kernel work */
#include <linux/module.h> /* Specifically, a module */

int init_module(void)
{
    pr_info("Hello, world - this is the kernel speaking\n");
    return 0;
}

MODULE_LICENSE("GPL");
```

The next file:

```c
/*
 * stop.c - Illustration of multi filed modules
 */

#include <linux/kernel.h> /* We are doing kernel work */
#include <linux/module.h> /* Specifically, a module  */

void cleanup_module(void)
{
    pr_info("Short is the life of a kernel module\n");
}

MODULE_LICENSE("GPL");
```

And finally, the makefile:

```
obj-m += hello-1.o
obj-m += hello-2.o
obj-m += hello-3.o
obj-m += hello-4.o
obj-m += hello-5.o
obj-m += hello-6.o
obj-m += startstop.o
startstop-objs := start.o stop.o

PWD := $(CURDIR)

```

```
12 all:
13         $(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
14
15 clean:
16         $(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

This is the complete makefile for all the examples we have seen so far. The first five lines are nothing special, but for the last example we will need two lines. First we invent an object name for our combined module, second we tell `make` what object files are part of that module.

4.7 Building modules for a precompiled kernel

Obviously, we strongly suggest you to recompile your kernel, so that you can enable a number of useful debugging features, such as forced module unloading (`MODULE_FORCE_UNLOAD`): when this option is enabled, you can force the kernel to unload a module even when it believes it is unsafe, via a `sudo rmmod -f module` command. This option can save you a lot of time and a number of reboots during the development of a module. If you do not want to recompile your kernel then you should consider using the QEMU-based environment described in Section 1.8, which already enables `MODULE_FORCE_UNLOAD` and other debugging options in its kernel configuration. If you mess anything up inside the guest you can simply close and reboot the virtual machine.

There are a number of cases in which you may want to load your module into a precompiled running kernel, such as the ones shipped with common Linux distributions, or a kernel you have compiled in the past. In certain circumstances you could require to compile and insert a module into a running kernel which you are not allowed to recompile, or on a machine that you prefer not to reboot. If you can't think of a case that will force you to use modules for a precompiled kernel you might want to skip this and treat the rest of this chapter as a big footnote.

Now, if you just install a kernel source tree, use it to compile your kernel module and you try to insert your module into the kernel, in most cases you would obtain an error as follows:

```
insmod: ERROR: could not insert module poet.ko: Invalid module format
```

Less cryptic information is logged to the systemd journal:

```
kernel: poet: disagrees about version of symbol module_layout
```

In other words, your kernel refuses to accept your module because version strings (more precisely, *version magic*, see include/linux/vermagic.h) do not match. The version magic string encodes the kernel version, the SMP configuration, preemption model, and other build flags that affect the kernel ABI. When you `insmod` a module, the kernel compares the module's version magic against its own; any mismatch means the module was compiled for a different

kernel configuration and loading it could corrupt kernel data structures. Additionally, if `CONFIG_MODVERSIONS` is enabled, each exported symbol carries a CRC checksum of its prototype. A module will be rejected if any symbol signature changed between the kernel it was compiled against and the one it is being loaded into.

Version magic strings are stored in the module object in the form of a static string, starting with `vermagic:`. Version data are inserted in your module when it is linked against the `kernel/module.o` file. To inspect version magics and other strings stored in a given module, issue the command `modinfo module.ko`:

```
$ modinfo hello-4.ko
description:    A sample driver
author:         LKMPG
license:        GPL
srcversion:     B2AA7FBFCC2C39AED665382
depends:
retpoline:      Y
name:           hello_4
vermagic:       5.4.0-70-generic SMP mod_unload modversions
```

To overcome this problem we could resort to the `--force-vermagic` option, but this solution is potentially unsafe, and unquestionably unacceptable in production modules. Consequently, we want to compile our module in an environment which was identical to the one in which our precompiled kernel was built. How to do this, is the subject of the remainder of this chapter.

First of all, make sure that a kernel source tree is available, having exactly the same version as your current kernel. Then, find the configuration file which was used to compile your precompiled kernel. Usually, this is available in your current `boot` directory, under a name like `config-5.14.x`. You may just want to copy it to your kernel source tree: `cp /boot/config-`uname -r` .config`.

Let's focus again on the previous error message: a closer look at the version magic strings suggests that, even with two configuration files which are exactly the same, a slight difference in the version magic could be possible, and it is sufficient to prevent insertion of the module into the kernel. That slight difference, namely the custom string which appears in the module's version magic and not in the kernel's one, is due to a modification with respect to the original, in the makefile that some distributions include. Then, examine your `Makefile`, and make sure that the specified version information matches exactly the one used for your current kernel. For example, your makefile could start as follows:

```
VERSION = 5
PATCHLEVEL = 14
SUBLEVEL = 0
EXTRAVERSION = -rc2
```

In this case, you need to restore the value of symbol **EXTRAVERSION** to **-rc2**. We suggest keeping a backup copy of the makefile used to compile your

kernel available in `/lib/modules/5.14.0-rc2/build`. A simple command as follows should suffice.

```
cp /lib/modules/`uname -r`/build/Makefile linux-`uname -r`
```

Here `linux-`uname -r`` is the Linux kernel source you are attempting to build.

Now, please run `make` to update configuration and version headers and objects:

```
$ make
  SYNC    include/config/auto.conf.cmd
  HOSTCC  scripts/basic/fixdep
  HOSTCC  scripts/kconfig/conf.o
  HOSTCC  scripts/kconfig/confdata.o
  HOSTCC  scripts/kconfig/expr.o
  LEX     scripts/kconfig/lexer.lex.c
  YACC    scripts/kconfig/parser.tab.[ch]
  HOSTCC  scripts/kconfig/preprocess.o
  HOSTCC  scripts/kconfig/symbol.o
  HOSTCC  scripts/kconfig/util.o
  HOSTCC  scripts/kconfig/lexer.lex.o
  HOSTCC  scripts/kconfig/parser.tab.o
  HOSTLD  scripts/kconfig/conf
```

If you do not desire to actually compile the kernel, you can interrupt the build process (CTRL-C) just after the SPLIT line, because at that time, the files you need are ready. Now you can turn back to the directory of your module and compile it: It will be built exactly according to your current kernel settings, and it will load into it without any errors.

2

Core Kernel Interfaces

With the toolchain and first examples out of the way, the focus shifts to the interfaces that let modules cooperate with the rest of the kernel. The following sections introduce execution basics and then move through the common entry points used to expose functionality to user space.

5 Preliminaries

5.1 How modules begin and end

A typical program starts with a `main()` function, executes a series of instructions, and terminates after completing these instructions. Kernel modules, however, follow a different pattern. A module always begins with either the `init_module` function or a function designated by the `module_init` call. This function acts as the module's entry point, informing the kernel of the module's functionalities and preparing the kernel to utilize the module's functions when necessary. After performing these tasks, the entry function returns, and the module remains inactive until the kernel requires its code.

All modules conclude by invoking either `cleanup_module` or a function specified through the `module_exit` call. This serves as the module's exit function, reversing the actions of the entry function by unregistering the previously registered functionalities.

It is mandatory for every module to have both an entry and an exit function. While there are multiple methods to define these functions, the terms "entry function" and "exit function" are generally used. However, they may occasionally be referred to as `init_module` and `cleanup_module`, which are understood to mean the same.

Error handling during initialization

Registration and allocation calls in the init function can fail. Every resource acquired must be released on error, otherwise the system leaks memory or, worse, becomes unstable. The canonical kernel pattern uses `goto`-based cleanup in reverse allocation order:

```
static int __init my_init(void)
{
    int err;

    err = register_A();
    if (err)
        return err;

    err = register_B();
    if (err)
        goto undo_A;

    err = register_C();
    if (err)
        goto undo_B;

    return 0;

undo_B:
    unregister_B();
undo_A:
    unregister_A();
    return err;
}
```

This pattern avoids deeply nested `if`/`else` blocks and guarantees that resources are freed in the exact reverse order of acquisition. Always return a proper negative error code defined in include/linux/errno.h (e.g. `-ENOMEM`, `-ENODEV`) so that `insmod` can report a meaningful error.

Some kernel functions encode errors directly in the returned pointer value rather than returning `NULL`. Functions documented to use this convention (e.g. `class_create()`) return values produced by `ERR_PTR()`. The macros `IS_ERR()`, `PTR_ERR()`, and `ERR_PTR()` (from include/linux/err.h) handle this: `ERR_PTR(-ENOMEM)` converts an error code into a pointer, `IS_ERR(ptr)` tests whether a returned pointer is actually an encoded error, and `PTR_ERR(ptr)` extracts the negative error code. Not all pointer-returning APIs use this convention; for example, `proc_create()`, `kmalloc()`, and `kzalloc()` return plain `NULL` on failure. Always check the documentation of the specific function to know which error convention it follows.

Module-loading races

Once you register a callback structure with the kernel (e.g. a `file_operations` via `cdev_add()` or a `/proc` entry via `proc_create()`), user space can invoke

those callbacks immediately, even before the init function returns. Therefore, complete all internal initialization before registering any callback-bearing structure. If registration is the first thing init does and some later step fails, the cleanup path may race with an in-progress callback. The rule of thumb: register last, unregister first.

5.2 Functions available to modules

Programmers use functions they do not define all the time. A prime example of this is `printf()`. You use these library functions which are provided by the standard C library, libc. The definitions for these functions do not actually enter your program until the linking stage, which ensures that the code (for `printf()` for example) is available, and fixes the call instruction to point to that code.

Kernel modules are different here, too. In the hello world example, you might have noticed that we used a function, `pr_info()` but did not include a standard I/O library. That is because modules are object files whose symbols get resolved upon running `insmod` or `modprobe`. The definition for the symbols comes from the kernel itself; the only external functions you can use are the ones provided by the kernel. If you're curious about what symbols have been exported by your kernel, take a look at `/proc/kallsyms`.

One point to keep in mind is the difference between library functions and system calls. Library functions are higher level, run completely in user space and provide a more convenient interface for the programmer to the functions that do the real work — system calls. System calls run in kernel mode on the user's behalf and are provided by the kernel itself. The library function `printf()` may look like a very general printing function, but all it really does is format the data into strings and write the string data using the low-level system call `write()`, which then sends the data to standard output.

Would you like to see what system calls are made by `printf()`? It is easy! Compile the following program:

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      printf("hello");
6      return 0;
7  }
```

with `gcc -Wall -o hello hello.c`. Run the executable with `strace ./hello`. Are you impressed? Every line you see corresponds to a system call. strace is a handy program that gives you details about what system calls a program is making, including which call is made, what its arguments are and what it returns. It is an invaluable tool for figuring out things like what files a program is trying to access. Towards the end, you will see a line which looks like `write(1, "hello", 5hello)`. There it is. The face behind the `printf()`

mask. You may not be familiar with write, since most people use library functions for file I/O (like `fopen`, `fputs`, `fclose`). If that is the case, try looking at man 2 write. The 2nd man section is devoted to system calls (like `kill()` and `read()`). The 3rd man section is devoted to library calls, which you would probably be more familiar with (like `cosh()` and `random()`).

You can even write modules to replace the kernel's system calls, which we will do shortly. Crackers often make use of this sort of thing for backdoors or trojans, but you can write your own modules to do more benign things, like have the kernel log a message whenever someone attempts to delete a file on your system.

5.3 User Space vs Kernel Space

The kernel primarily manages access to resources, be it a video card, hard drive, or memory. Programs frequently vie for the same resources. For instance, as a document is saved, updatedb might commence updating the locate database. Sessions in editors like vim and processes like updatedb can simultaneously utilize the hard drive. The kernel's role is to maintain order, ensuring that users do not access resources indiscriminately.

To manage this, CPUs operate in different modes, each offering varying levels of system control. The Intel 80386 architecture, for example, featured four such modes, known as rings. Unix, however, utilizes only two of these rings: the highest ring (ring 0, also known as "supervisor mode", where all actions are permissible) and the lowest ring, referred to as "user mode".

Recall the discussion about library functions vs system calls. Typically, you use a library function in user mode. The library function calls one or more system calls, and these system calls execute on the library function's behalf, but do so in supervisor mode since they are part of the kernel itself. Once the system call completes its task, it returns and execution gets transferred back to user mode.

Kernel code begins executing in response to one of three events: system calls, hardware interrupts, or processor exceptions (traps). System calls are the usual user-to-kernel transition, initiated by a running process. Interrupts and exceptions may arrive while the CPU is already in kernel mode, so they are not strictly user-to-kernel crossings; they are entries into new kernel execution contexts. Understanding these distinctions matters because each context places different constraints on what a module may do.

Concurrency in the kernel

Unlike a single-threaded user-space program, kernel code is inherently concurrent. Multiple CPUs may execute kernel paths simultaneously, hardware interrupts can preempt process context at any time, and the scheduler may preempt kernel code on `PREEMPT`-enabled kernels. This means that any shared data structure must be protected by appropriate synchronization primitives (see Section 13).

Each CPU has exactly one "current" process at any given moment, accessible via the `current` macro. When your module code runs in process context (e.g. a system call or a file operation callback), `current` points to the `struct task_struct` of the process that triggered the call. In interrupt context, `current` is technically still valid but refers to whichever process happened to be running when the interrupt fired; it is meaningless for your handler's logic and should not be used.

The table below summarizes what is allowed in each execution context. Getting this wrong is one of the most common sources of kernel bugs.

Operation	Process ctx	Softirq/tasklet ctx	Hardirq ctx
Sleep / schedule	Yes	No	No
Access user memory (`copy_to_user`)	Yes	No	No
`kmalloc(GFP_KERNEL)`	Yes	No	No
`kmalloc(GFP_ATOMIC)`	Yes	Yes	Yes
Acquire mutex	Yes	No	No
Acquire spinlock	Yes	Yes	Yes
`current` pointer meaningful	Yes	No	No

5.4 Name Space

When you write a small C program, you use variables which are convenient and make sense to the reader. If, on the other hand, you are writing routines which will be part of a bigger problem, any global variables you have are part of a community of other peoples' global variables; some of the variable names can clash. When a program has lots of global variables which aren't meaningful enough to be distinguished, you get namespace pollution. In large projects, effort must be made to remember reserved names, and to find ways to develop a scheme for naming unique variable names and symbols.

When writing kernel code, even the smallest module will be linked against the entire kernel, so this is definitely an issue. The best way to deal with this is to declare all your variables as static and to use a well-defined prefix for your symbols. By convention, all kernel prefixes are lowercase. If you do not want to declare everything as static, another option is to declare a symbol table and register it with the kernel. We will get to this later.

The file `/proc/kallsyms` holds all the symbols that the kernel knows about and which are therefore accessible to your modules since they share the kernel's codespace.

5.5 Code space

Memory management is a very complicated subject and the majority of O'Reilly's Understanding The Linux Kernel exclusively covers memory management! We are not setting out to be experts on memory management, but we do need to know a couple of facts to even begin worrying about writing real modules.

If you have not thought about what a segfault really means, you may be surprised to hear that pointers do not actually point to memory locations. Not real ones, anyway. When a process is created, the kernel sets aside a portion of real physical memory and hands it to the process to use for its executing code, variables, stack, heap and other things which a computer scientist would know about. This memory begins with 0x00000000 and extends up to whatever it needs to be. Since the memory space for any two processes does not overlap, every process that can access a memory address, say 0xbffff978, would be accessing a different location in real physical memory! The processes would be accessing an index named 0xbffff978 which points to some kind of offset into the region of memory set aside for that particular process. For the most part, a process like our Hello, World program cannot access the space of another process, although there are ways which we will talk about later.

The kernel has its own space of memory as well. Since a module is code which can be dynamically inserted and removed in the kernel (as opposed to a semi-autonomous object), it shares the kernel's codespace rather than having its own. Therefore, if your module segfaults, the kernel segfaults. And if you start writing over data because of an off-by-one error, then you're trampling on kernel data (or code). This is even worse than it sounds, so try your best to be careful.

It should be noted that the aforementioned discussion applies to any operating system utilizing a monolithic kernel. This concept differs slightly from *"building all your modules into the kernel"*, although the underlying principle is similar. In contrast, there are microkernels, where modules are allocated their own code space. Two notable examples of microkernels include the GNU Hurd and the Zircon kernel of Google's Fuchsia.

5.6 Device Drivers

One class of module is the device driver, which provides functionality for hardware like a serial port. On Unix, each piece of hardware is represented by a file located in `/dev` named a device file which provides the means to communicate with the hardware. The device driver provides the communication on behalf of a user program. So the es1370.ko sound card device driver might connect the `/dev/sound` device file to the Ensoniq ES1370 sound card. A userspace program like mp3blaster can use `/dev/sound` without ever knowing what kind of sound card is installed.

Let's look at some device files. Here are device files which represent the first three partitions on the primary SCSI storage devices:

```
$ ls -l /dev/sda[1-3]
brw-rw----  1 root  disk  8, 1 Apr  9  2025 /dev/sda1
brw-rw----  1 root  disk  8, 2 Apr  9  2025 /dev/sda2
brw-rw----  1 root  disk  8, 3 Apr  9  2025 /dev/sda3
```

Notice the column of numbers separated by a comma. The first number is called the device's major number. The second number is the minor number.

The major number tells you which driver is used to access the hardware. Each driver is assigned a unique major number; all device files with the same major number are controlled by the same driver. All the above major numbers are 8, because they're all controlled by the same driver.

The minor number is used by the driver to distinguish between the various hardware it controls. Returning to the example above, although all three devices are handled by the same driver they have unique minor numbers because the driver sees them as being different pieces of hardware.

Devices are divided into two types: character devices and block devices. The difference is that block devices have a buffer for requests, so they can choose the best order in which to respond to the requests. This is important in the case of storage devices, where it is faster to read or write sectors which are close to each other, rather than those which are further apart. Another difference is that block devices can only accept input and return output in blocks (whose size can vary according to the device), whereas character devices are allowed to use as many or as few bytes as they like. Most devices in the world are character, because they don't need this type of buffering, and they don't operate with a fixed block size. You can tell whether a device file is for a block device or a character device by looking at the first character in the output of `ls -l`. If it is 'b' then it is a block device, and if it is 'c' then it is a character device. The devices you see above are block devices. Here are some character devices (the serial ports):

```
crw-rw----  1 root  dial 4, 64 Feb 18 23:34 /dev/ttyS0
crw-r-----  1 root  dial 4, 65 Nov 17 10:26 /dev/ttyS1
crw-rw----  1 root  dial 4, 66 Jul  5  2000 /dev/ttyS2
crw-rw----  1 root  dial 4, 67 Jul  5  2000 /dev/ttyS3
```

If you want to see which major numbers have been assigned, you can look at Documentation/admin-guide/devices.txt.

When the system was installed, all of those device files were created by the `mknod` command. To create a new char device named `coffee` with major/minor number 12 and 2, simply do `mknod /dev/coffee c 12 2`. You do not have to put your device files into `/dev`, but it is done by convention. Linus put his device files in `/dev`, and so should you. However, when creating a device file for testing purposes, it is probably OK to place it in your working directory where you compile the kernel module. Just be sure to put it in the right place when you're done writing the device driver.

A few final points, although implicit in the previous discussion, are worth stating explicitly for clarity. When a device file is accessed, the kernel utilizes the file's major number to identify the appropriate driver for handling the access. This indicates that the kernel does not necessarily rely on or need to be aware of the minor number. It is the driver that concerns itself with the minor number, using it to differentiate between various pieces of hardware.

It is important to note that when referring to *"hardware"*, the term is used in a slightly more abstract sense than just a physical PCI card that can be held in hand. Consider the following two device files:

```
$ ls -l /dev/sda /dev/sdb
brw-rw---- 1 root disk 8,  0 Jan  3 09:02 /dev/sda
brw-rw---- 1 root disk 8, 16 Jan  3 09:02 /dev/sdb
```

By now you can look at these two device files and know instantly that they are block devices and are handled by same driver (block major 8). Sometimes two device files with the same major but different minor number can actually represent the same piece of physical hardware. So just be aware that the word "hardware" in our discussion can mean something very abstract.

6 Character Device drivers

6.1 The file_operations Structure

The `file_operations` structure is defined in include/linux/fs.h, and holds pointers to functions defined by the driver that perform various operations on the device. Each field of the structure corresponds to the address of some function defined by the driver to handle a requested operation.

For example, every character driver needs to define a function that reads from the device. The `file_operations` structure holds the address of the module's function that performs that operation. Here is what the definition looks like for kernel 5.4 and later versions:

```
1  struct file_operations {
2      struct module *owner;
3      loff_t (*llseek) (struct file *, loff_t, int);
4      ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
5      ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
6      ssize_t (*read_iter) (struct kiocb *, struct iov_iter *);
7      ssize_t (*write_iter) (struct kiocb *, struct iov_iter *);
8      int (*iopoll)(struct kiocb *kiocb, bool spin);
9      int (*iterate) (struct file *, struct dir_context *);
10     int (*iterate_shared) (struct file *, struct dir_context *);
11     __poll_t (*poll) (struct file *, struct poll_table_struct *);
12     long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long);
13     long (*compat_ioctl) (struct file *, unsigned int, unsigned long);
14     int (*mmap) (struct file *, struct vm_area_struct *);
15     unsigned long mmap_supported_flags;
16     int (*open) (struct inode *, struct file *);
17     int (*flush) (struct file *, fl_owner_t id);
18     int (*release) (struct inode *, struct file *);
19     int (*fsync) (struct file *, loff_t, loff_t, int datasync);
20     int (*fasync) (int, struct file *, int);
21     int (*lock) (struct file *, int, struct file_lock *);
22     ssize_t (*sendpage) (struct file *, struct page *, int, size_t, loff_t *,
       ↪  int);
23     unsigned long (*get_unmapped_area)(struct file *, unsigned long, unsigned
       ↪  long, unsigned long, unsigned long);
24     int (*check_flags)(int);
25     int (*flock) (struct file *, int, struct file_lock *);
26     ssize_t (*splice_write)(struct pipe_inode_info *, struct file *, loff_t *,
       ↪  size_t, unsigned int);
```

```
    ssize_t (*splice_read)(struct file *, loff_t *, struct pipe_inode_info *,
    ↪  size_t, unsigned int);
    int (*setlease)(struct file *, long, struct file_lock **, void **);
    long (*fallocate)(struct file *file, int mode, loff_t offset,
        loff_t len);
    void (*show_fdinfo)(struct seq_file *m, struct file *f);
    ssize_t (*copy_file_range)(struct file *, loff_t, struct file *,
        loff_t, size_t, unsigned int);
    loff_t (*remap_file_range)(struct file *file_in, loff_t pos_in,
              struct file *file_out, loff_t pos_out,
              loff_t len, unsigned int remap_flags);
    int (*fadvise)(struct file *, loff_t, loff_t, int);
} __randomize_layout;
```

Some operations are not implemented by a driver. For example, a driver that handles a video card will not need to read from a directory structure. The corresponding entries in the `file_operations` structure should be set to `NULL`. [1]

There is a gcc extension that makes assigning to this structure more convenient. You will see it in modern drivers, and may catch you by surprise. This is what the new way of assigning to the structure looks like:

```
struct file_operations fops = {
        read: device_read,
        write: device_write,
        open: device_open,
        release: device_release
};
```

However, there is also a C99 way of assigning to elements of a structure, designated initializers, and this is definitely preferred over using the GNU extension. You should use this syntax in case someone wants to port your driver. It will help with compatibility:

```
struct file_operations fops = {
        .read = device_read,
        .write = device_write,
        .open = device_open,
        .release = device_release
};
```

The meaning is clear, and you should be aware that any member of the structure which you do not explicitly assign will be initialized to `NULL` by gcc.

An instance of `struct file_operations` containing pointers to functions that are used to implement `read`, `write`, `open`, ... system calls is commonly named `fops`.

[1] As of Linux kernel 6.12, several member fields have been added, removed, or had their prototypes changed. For example, additions include `fop_flags`, `splice_eof`, and `uring_cmd`; removals include `iterate` and `sendpage`; and the prototype for `iopoll` was modified.

Since Linux v3.14, the read, write and seek operations are guaranteed to be thread-safe by using the `f_pos` specific lock, which makes the file position update to become the mutual exclusion. So, we can safely implement those operations without unnecessary locking.

Additionally, since Linux v5.6, the `proc_ops` structure was introduced to replace the use of the `file_operations` structure when registering proc handlers. See more information in the Section 7.1.

6.2 The file structure

Each device is represented in the kernel by a file structure, which is defined in include/linux/fs.h. Be aware that a file is a kernel level structure and never appears in a user space program. It is not the same thing as a `FILE`, which is defined by glibc and would never appear in a kernel space function. Also, its name is a bit misleading; it represents an abstract open 'file', not a file on a disk, which is represented by a structure named `inode`.

An instance of struct file is commonly named `filp`. You'll also see it referred to as a struct file object. Resist the temptation.

Go ahead and look at the definition of file. Most of the entries you see, like struct dentry, are not used by device drivers, and you can ignore them. This is because drivers do not fill file directly; they only use structures contained in file which are created elsewhere.

One subtlety worth noting: the `open` callback is called once for every `open()` system call, but `release` is called only when the last file descriptor referring to that `struct file` is closed. If a process calls `dup()` or `fork()`, multiple file descriptors share the same `struct file`; `release` is deferred until all of them are gone. The `flush` callback, by contrast, is invoked on every `close()`. The `private_data` field of `struct file` is the standard place to attach per-open state allocated in `open` and freed in `release`.

6.3 Registering A Device

As discussed earlier, char devices are accessed through device files, usually located in `/dev`. This is by convention. When writing a driver, it is OK to put the device file in your current directory. Just make sure you place it in `/dev` for a production driver. The major number tells you which driver handles which device file. The minor number is used only by the driver itself to differentiate which device it is operating on, just in case the driver handles more than one device.

Adding a driver to your system means registering it with the kernel. This is synonymous with assigning it a major number during the module's initialization. You do this by using the `register_chrdev` function, defined by include/linux/fs.h.

```
1  int register_chrdev(unsigned int major, const char *name, struct
   ↪  file_operations *fops);
```

Where `unsigned int major` is the major number you want to request, `const char *name` is the name of the device as it will appear in `/proc/devices` and `struct file_operations *fops` is a pointer to the `file_operations` table for your driver. A negative return value means the registration failed. Note that we didn't pass the minor number to `register_chrdev`. That is because the kernel doesn't care about the minor number; only our driver uses it.

Now the question is, how do you get a major number without hijacking one that's already in use? The easiest way would be to look through Documentation/admin-guide/devices.txt and pick an unused one. That is a bad way of doing things because you will never be sure if the number you picked will be assigned later. The answer is that you can ask the kernel to assign you a dynamic major number.

If you pass a major number of 0 to `register_chrdev`, the return value will be the dynamically allocated major number. The downside is that you can not make a device file in advance, since you do not know what the major number will be. There are a couple of ways to do this. First, the driver itself can print the newly assigned number and we can make the device file by hand. Second, the newly registered device will have an entry in `/proc/devices`, and we can either make the device file by hand or write a shell script to read the file in and make the device file. The third method is that we can have our driver make the device file using the `device_create` function after a successful registration and `device_destroy` during the call to `cleanup_module`.

However, `register_chrdev()` would occupy a range of minor numbers associated with the given major. The recommended way to reduce waste for char device registration is using cdev interface.

The newer interface completes the char device registration in two distinct steps. First, we should register a range of device numbers, which can be completed with `register_chrdev_region` or `alloc_chrdev_region`.

```
1 int register_chrdev_region(dev_t from, unsigned count, const char *name);
2 int alloc_chrdev_region(dev_t *dev, unsigned baseminor, unsigned count, const
  ↪  char *name);
```

The choice between two different functions depends on whether you know the major numbers for your device. Using `register_chrdev_region` if you know the device major number and `alloc_chrdev_region` if you would like to allocate a dynamically-allocated major number.

Second, we should initialize the data structure `struct cdev` for our char device and associate it with the device numbers. To initialize the `struct cdev`, we can achieve by the similar sequence of the following codes.

```
1 struct cdev *my_cdev = cdev_alloc();
2 my_cdev->ops = &my_fops;
```

However, the common usage pattern will embed the `struct cdev` within a

device-specific structure of your own. In this case, we'll need `cdev_init` for the initialization.

```
1 void cdev_init(struct cdev *cdev, const struct file_operations *fops);
```

Once we finish the initialization, we can add the char device to the system by using the `cdev_add`.

```
1 int cdev_add(struct cdev *p, dev_t dev, unsigned count);
```

To find an example using the interface, you can see `ioctl.c` described in Section 10.

6.4 Unregistering A Device

We can not allow the kernel module to be `rmmod`'ed whenever root feels like it. If the device file is opened by a process and then we remove the kernel module, using the file would cause a call to the memory location where the appropriate function (read/write) used to be. If we are lucky, no other code was loaded there, and we'll get an ugly error message. If we are unlucky, another kernel module was loaded into the same location, which means a jump into the middle of another function within the kernel. The results of this would be impossible to predict, but they can not be very positive.

Normally, when you do not want to allow something, you return an error code (a negative number) from the function which is supposed to do it. With `cleanup_module` that's impossible because it is a void function. However, there is a counter which keeps track of how many processes are using your module. You can see what its value is by looking at the 3rd field with the command `cat /proc/modules` or `lsmod`. If this number isn't zero, `rmmod` will fail. Note that you do not have to check the counter within `cleanup_module` because the check will be performed for you by the system call `sys_delete_module`, defined in include/linux/syscalls.h. You should not use this counter directly, but there are functions defined in include/linux/module.h which let you display this counter:

- `module_refcount(THIS_MODULE)`: Return the value of reference count of current module.

Note: The use of `try_module_get(THIS_MODULE)` and `module_put(THIS_MODULE)` within a module's own code is considered unsafe and should be avoided. The kernel automatically manages the reference count when file operations are in progress, so manual reference counting is unnecessary and can lead to race conditions. For a deeper understanding of when and how to properly use module reference counting, see https://stackoverflow.com/questions/1741415/linux-kernel-modules-when-to-use-try-module-get-module-put.

6.5 chardev.c

The next code sample creates a char driver named `chardev`. You can verify it has been registered by checking:

```
cat /proc/devices
```

This will show the device's major number. To actually use the device, you need to read from `/dev/chardev` (or open the file with a program) and the driver will put the number of times the device file has been read from into the file. We do not support writing to the file (like `echo "hi" > /dev/chardev`), but catch these attempts and tell the user that the operation is not supported. Do not worry if you do not see what we do with the data we read into the buffer; we do not do much with it. We simply read in the data and print a message acknowledging that we received it.

In a multi-threaded environment, without any protection, concurrent access to the same memory may lead to race conditions and will not preserve performance. In the kernel module, this problem may happen due to multiple instances accessing the shared resources. Therefore, a solution is to enforce exclusive access. We use atomic Compare-And-Swap (CAS) to maintain the states, `CDEV_NOT_USED` and `CDEV_EXCLUSIVE_OPEN`, to determine whether the file is currently opened by someone or not. CAS compares the contents of a memory location with the expected value and, only if they are the same, modifies the contents of that memory location to the desired value. See more concurrency details in the Section 13.

```c
/*
 * chardev.c: Creates a read-only char device that says how many times
 * you have read from the dev file
 */

#include <linux/atomic.h>
#include <linux/cdev.h>
#include <linux/delay.h>
#include <linux/device.h>
#include <linux/fs.h>
#include <linux/init.h>
#include <linux/kernel.h> /* for sprintf() */
#include <linux/module.h>
#include <linux/printk.h>
#include <linux/types.h>
#include <linux/uaccess.h> /* for get_user and put_user */
#include <linux/version.h>

#include <asm/errno.h>

/*  Prototypes - this would normally go in a .h file */
static int device_open(struct inode *, struct file *);
static int device_release(struct inode *, struct file *);
static ssize_t device_read(struct file *, char __user *, size_t, loff_t *);
```

```
25  static ssize_t device_write(struct file *, const char __user *, size_t,
26                              loff_t *);
27
28  #define DEVICE_NAME "chardev" /* Dev name as it appears in /proc/devices   */
29  #define BUF_LEN 80 /* Max length of the message from the device */
30
31  /* Global variables are declared as static, so are global within the file. */
32
33  static int major; /* major number assigned to our device driver */
34
35  enum {
36      CDEV_NOT_USED,
37      CDEV_EXCLUSIVE_OPEN,
38  };
39
40  /* Is device open? Used to prevent multiple access to device */
41  static atomic_t already_open = ATOMIC_INIT(CDEV_NOT_USED);
42
43  static char msg[BUF_LEN + 1]; /* The msg the device will give when asked */
44
45  static struct class *cls;
46
47  static struct file_operations chardev_fops = {
48      .read = device_read,
49      .write = device_write,
50      .open = device_open,
51      .release = device_release,
52  };
53
54  static int __init chardev_init(void)
55  {
56      major = register_chrdev(0, DEVICE_NAME, &chardev_fops);
57
58      if (major < 0) {
59          pr_alert("Registering char device failed with %d\n", major);
60          return major;
61      }
62
63      pr_info("I was assigned major number %d.\n", major);
64
65  #if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 4, 0)
66      cls = class_create(DEVICE_NAME);
67  #else
68      cls = class_create(THIS_MODULE, DEVICE_NAME);
69  #endif
70      if (IS_ERR(cls)) {
71          pr_err("Failed to create class for device\n");
72          unregister_chrdev(major, DEVICE_NAME);
73          return PTR_ERR(cls);
74      }
75      device_create(cls, NULL, MKDEV(major, 0), NULL, DEVICE_NAME);
76
77      pr_info("Device created on /dev/%s\n", DEVICE_NAME);
78
79      return 0;
80  }
81
```

```
82  static void __exit chardev_exit(void)
83  {
84      device_destroy(cls, MKDEV(major, 0));
85      class_destroy(cls);
86
87      /* Unregister the device */
88      unregister_chrdev(major, DEVICE_NAME);
89  }
90
91  /* Methods */
92
93  /* Called when a process tries to open the device file, like
94   * "sudo cat /dev/chardev"
95   */
96  static int device_open(struct inode *inode, struct file *file)
97  {
98      static int counter = 0;
99
100     if (atomic_cmpxchg(&already_open, CDEV_NOT_USED, CDEV_EXCLUSIVE_OPEN))
101         return -EBUSY;
102
103     sprintf(msg, "I already told you %d times Hello world!\n", counter++);
104
105     return 0;
106 }
107
108 /* Called when a process closes the device file. */
109 static int device_release(struct inode *inode, struct file *file)
110 {
111     /* We're now ready for our next caller */
112     atomic_set(&already_open, CDEV_NOT_USED);
113
114     return 0;
115 }
116
117 /* Called when a process, which already opened the dev file, attempts to
118  * read from it.
119  */
120 static ssize_t device_read(struct file *filp, /* see include/linux/fs.h   */
121                            char __user *buffer, /* buffer to fill with data */
122                            size_t length, /* length of the buffer     */
123                            loff_t *offset)
124 {
125     /* Number of bytes actually written to the buffer */
126     int bytes_read = 0;
127     const char *msg_ptr = msg;
128
129     if (!*(msg_ptr + *offset)) { /* we are at the end of message */
130         *offset = 0; /* reset the offset */
131         return 0; /* signify end of file */
132     }
133
134     msg_ptr += *offset;
135
136     /* Actually put the data into the buffer */
137     while (length && *msg_ptr) {
138         /* The buffer is in the user data segment, not the kernel
```

```
139             * segment so "*" assignment won't work.  We have to use
140             * put_user which copies data from the kernel data segment to
141             * the user data segment.
142             */
143            put_user(*(msg_ptr++), buffer++);
144            length--;
145            bytes_read++;
146        }
147
148        *offset += bytes_read;
149
150        /* Most read functions return the number of bytes put into the buffer. */
151        return bytes_read;
152    }
153
154    /* Called when a process writes to dev file: echo "hi" | sudo tee /dev/chardev
    ↪  */
155    static ssize_t device_write(struct file *filp, const char __user *buff,
156                                size_t len, loff_t *off)
157    {
158        pr_alert("Sorry, this operation is not supported.\n");
159        return -EINVAL;
160    }
161
162    module_init(chardev_init);
163    module_exit(chardev_exit);
164
165    MODULE_LICENSE("GPL");
```

6.6 Writing Modules for Multiple Kernel Versions

The system calls, which are the major interface the kernel shows to the processes, generally stay the same across versions. A new system call may be added, but usually the old ones will behave exactly like they used to. This is necessary for backward compatibility – a new kernel version is not supposed to break regular processes. In most cases, the device files will also remain the same. On the other hand, the internal interfaces within the kernel can and do change between versions.

There are differences between different kernel versions, and if you want to support multiple kernel versions, you will find yourself having to code conditional compilation directives. The way to do this is to compare the macro `LINUX_VERSION_CODE` to the macro `KERNEL_VERSION`. In version `a.b.c` of the kernel, the value of this macro would be $2^{16}a + 2^8b + c$.

7 The /proc Filesystem

In Linux, there is an additional mechanism for the kernel and kernel modules to send information to processes — the `/proc` filesystem. Originally designed to allow easy access to information about processes (hence the name), it is now used by every bit of the kernel which has something interesting to report, such

as `/proc/modules` which provides the list of modules and `/proc/meminfo` which gathers memory usage statistics.

The method to use the proc filesystem is very similar to the one used with device drivers — a structure is created with all the information needed for the `/proc` file, including pointers to any handler functions (in our case there is only one, the one called when somebody attempts to read from the `/proc` file). Then, `init_module` registers the structure with the kernel and `cleanup_module` unregisters it.

Normal filesystems are located on a disk, rather than just in memory (which is where `/proc` is), and in that case the index-node (inode for short) number is a pointer to a disk location where the file's inode is located. The inode contains information about the file, for example the file's permissions, together with a pointer to the disk location or locations where the file's data can be found.

Because we do not get called when the file is opened or closed, there is nowhere for us to put `try_module_get` and `module_put` in this module, and if the file is opened and then the module is removed, there is no way to avoid the consequences. The kernel's automatic reference counting for file operations helps prevent module removal while files are in use, but `/proc` files require careful handling due to their different lifecycle.

Here is a simple example showing how to use a `/proc` file. This is the HelloWorld for the `/proc` filesystem. There are three parts: create the file `/proc/helloworld` in the function `init_module`, return a value (and a buffer) when the file `/proc/helloworld` is read in the callback function `procfile_read`, and delete the file `/proc/helloworld` in the function `cleanup_module`.

The `/proc/helloworld` is created when the module is loaded with the function `proc_create`. The return value is a pointer to `struct proc_dir_entry`, and it will be used to configure the file `/proc/helloworld` (for example, the owner of this file). A null return value means that the creation has failed.

Every time the file `/proc/helloworld` is read, the function `procfile_read` is called. Two parameters of this function are very important: the buffer (the second parameter) and the offset (the fourth one). The content of the buffer will be returned to the application which read it (for example the `cat` command). The offset is the current position in the file. If the return value of the function is not null, then this function is called again. So be careful with this function, if it never returns zero, the read function is called endlessly.

```
$ cat /proc/helloworld
HelloWorld!
```

```c
/*
 * procfs1.c
 */

#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/proc_fs.h>
#include <linux/uaccess.h>
#include <linux/version.h>
```

```c

#if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 6, 0)
#define HAVE_PROC_OPS
#endif

#define procfs_name "helloworld"

static struct proc_dir_entry *our_proc_file;

static ssize_t procfile_read(struct file *file_pointer, char __user *buffer,
                             size_t buffer_length, loff_t *offset)
{
    char s[13] = "HelloWorld!\n";
    int len = sizeof(s);
    ssize_t ret = len;

    if (*offset >= len || copy_to_user(buffer, s, len)) {
        pr_info("copy_to_user failed\n");
        ret = 0;
    } else {
        pr_info("procfile read %s\n",
        ↪  file_pointer->f_path.dentry->d_name.name);
        *offset += len;
    }

    return ret;
}

#ifdef HAVE_PROC_OPS
static const struct proc_ops proc_file_fops = {
    .proc_read = procfile_read,
};
#else
static const struct file_operations proc_file_fops = {
    .read = procfile_read,
};
#endif

static int __init procfs1_init(void)
{
    our_proc_file = proc_create(procfs_name, 0644, NULL, &proc_file_fops);
    if (NULL == our_proc_file) {
        pr_alert("Error:Could not initialize /proc/%s\n", procfs_name);
        return -ENOMEM;
    }

    pr_info("/proc/%s created\n", procfs_name);
    return 0;
}

static void __exit procfs1_exit(void)
{
    proc_remove(our_proc_file);
    pr_info("/proc/%s removed\n", procfs_name);
}

module_init(procfs1_init);
```

```
66 module_exit(procfs1_exit);
67
68 MODULE_LICENSE("GPL");
```

7.1 The proc_ops Structure

The `proc_ops` structure is defined in include/linux/proc_fs.h in Linux v5.6+. In older kernels, it used `file_operations` for custom hooks in `/proc` filesystem, but it contains some members that are unnecessary in VFS, and every time VFS expands `file_operations` set, `/proc` code comes bloated. On the other hand, not only the space, but also some operations were saved by this structure to improve its performance. For example, the file which never disappears in `/proc` can set the `proc_flag` as `PROC_ENTRY_PERMANENT` to save 2 atomic ops, 1 allocation, 1 free in per open/read/close sequence.

7.2 Read and Write a /proc File

We have seen a very simple example for a `/proc` file where we only read the file `/proc/helloworld`. It is also possible to write in a `/proc` file. It works the same way as read, a function is called when the `/proc` file is written. But there is a little difference with read, data comes from user, so you have to import data from user space to kernel space (with `copy_from_user` or `get_user`)

The reason for `copy_from_user` or `get_user` is that Linux memory (on Intel architecture, it may be different under some other processors) is segmented. This means that a pointer, by itself, does not reference a unique location in memory, only a location in a memory segment, and you need to know which memory segment it is to be able to use it. There is one memory segment for the kernel, and one for each of the processes.

The only memory segment accessible to a process is its own, so when writing regular programs to run as processes, there is no need to worry about segments. When you write a kernel module, normally you want to access the kernel memory segment, which is handled automatically by the system. However, when the content of a memory buffer needs to be passed between the currently running process and the kernel, the kernel function receives a pointer to the memory buffer which is in the process segment. The `put_user` and `get_user` macros allow you to access that memory. These functions handle only one character, you can handle several characters with `copy_to_user` and `copy_from_user`. As the buffer (in read or write function) is in kernel space, for write function you need to import data because it comes from user space, but not for the read function because data is already in kernel space.

```
1 /*
2  * procfs2.c -  create a "file" in /proc
3  */
4
5 #include <linux/kernel.h> /* We're doing kernel work */
6 #include <linux/module.h> /* Specifically, a module */
```

```c
#include <linux/proc_fs.h> /* Necessary because we use the proc fs */
#include <linux/uaccess.h> /* for copy_from_user */
#include <linux/version.h>

#if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 6, 0)
#define HAVE_PROC_OPS
#endif

#define PROCFS_MAX_SIZE 1024
#define PROCFS_NAME "buffer1k"

/* This structure hold information about the /proc file */
static struct proc_dir_entry *our_proc_file;

/* The buffer used to store character for this module */
static char procfs_buffer[PROCFS_MAX_SIZE];

/* The size of the buffer */
static unsigned long procfs_buffer_size = 0;

/* This function is called then the /proc file is read */
static ssize_t procfile_read(struct file *file_pointer, char __user *buffer,
                             size_t buffer_length, loff_t *offset)
{
    char s[13] = "HelloWorld!\n";
    int len = sizeof(s);
    ssize_t ret = len;

    if (*offset >= len || copy_to_user(buffer, s, len)) {
        pr_info("copy_to_user failed\n");
        ret = 0;
    } else {
        pr_info("procfile read %s\n",
        ↪  file_pointer->f_path.dentry->d_name.name);
        *offset += len;
    }

    return ret;
}

/* This function is called when the /proc file is written. */
static ssize_t procfile_write(struct file *file, const char __user *buff,
                              size_t len, loff_t *off)
{
    procfs_buffer_size = len;
    if (procfs_buffer_size >= PROCFS_MAX_SIZE)
        procfs_buffer_size = PROCFS_MAX_SIZE - 1;

    if (copy_from_user(procfs_buffer, buff, procfs_buffer_size))
        return -EFAULT;

    procfs_buffer[procfs_buffer_size] = '\0';
    *off += procfs_buffer_size;
    pr_info("procfile write %s\n", procfs_buffer);

    return procfs_buffer_size;
}
```

```
63
64  #ifdef HAVE_PROC_OPS
65  static const struct proc_ops proc_file_fops = {
66      .proc_read = procfile_read,
67      .proc_write = procfile_write,
68  };
69  #else
70  static const struct file_operations proc_file_fops = {
71      .read = procfile_read,
72      .write = procfile_write,
73  };
74  #endif
75
76  static int __init procfs2_init(void)
77  {
78      our_proc_file = proc_create(PROCFS_NAME, 0644, NULL, &proc_file_fops);
79      if (NULL == our_proc_file) {
80          pr_alert("Error:Could not initialize /proc/%s\n", PROCFS_NAME);
81          return -ENOMEM;
82      }
83
84      pr_info("/proc/%s created\n", PROCFS_NAME);
85      return 0;
86  }
87
88  static void __exit procfs2_exit(void)
89  {
90      proc_remove(our_proc_file);
91      pr_info("/proc/%s removed\n", PROCFS_NAME);
92  }
93
94  module_init(procfs2_init);
95  module_exit(procfs2_exit);
96
97  MODULE_LICENSE("GPL");
```

7.3 Manage /proc file metadata and access hooks

We have seen how to read and write a `/proc` file with the `/proc` interface. The `proc_create` helper also lets us control metadata and add extra hooks, such as explicit `open` and `release` handlers. That is useful when we want more control than a minimal read/write example provides.

The sample in this section keeps the same read and write callbacks as before, but adds `procfs_open` and `procfs_close` to show where setup and teardown logic can live. This is also where you would normally attach per-open state if the file needed it.

Permissions are still important, but in this example they are controlled through the proc entry itself. The mode argument passed to `proc_create` sets the access bits, and `proc_set_user` adjusts the owner and group of the `/proc` entry.

Another helper used here is `proc_set_size`, which records the expected size of the virtual file. That information is optional, but it can improve the way user

space tools report the entry.

It is important to note that the standard roles of read and write are reversed in the kernel. Read functions are used for output, whereas write functions are used for input. The reason for that is that read and write refer to the user's point of view — if a process reads something from the kernel, then the kernel needs to output it, and if a process writes something to the kernel, then the kernel receives it as input.

```
/*
 * procfs3.c
 */

#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/proc_fs.h>
#include <linux/sched.h>
#include <linux/uaccess.h>
#include <linux/version.h>
#if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 10, 0)
#include <linux/minmax.h>
#endif

#if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 6, 0)
#define HAVE_PROC_OPS
#endif

#define PROCFS_MAX_SIZE 2048UL
#define PROCFS_ENTRY_FILENAME "buffer2k"

static struct proc_dir_entry *our_proc_file;
static char procfs_buffer[PROCFS_MAX_SIZE];
static unsigned long procfs_buffer_size = 0;

static ssize_t procfs_read(struct file *filp, char __user *buffer,
                           size_t length, loff_t *offset)
{
    if (*offset || procfs_buffer_size == 0) {
        pr_debug("procfs_read: END\n");
        *offset = 0;
        return 0;
    }
    procfs_buffer_size = min(procfs_buffer_size, length);
    if (copy_to_user(buffer, procfs_buffer, procfs_buffer_size))
        return -EFAULT;
    *offset += procfs_buffer_size;

    pr_debug("procfs_read: read %lu bytes\n", procfs_buffer_size);
    return procfs_buffer_size;
}
static ssize_t procfs_write(struct file *file, const char __user *buffer,
                            size_t len, loff_t *off)
{
    procfs_buffer_size = min(PROCFS_MAX_SIZE, len);
    if (copy_from_user(procfs_buffer, buffer, procfs_buffer_size))
        return -EFAULT;
    *off += procfs_buffer_size;
```

```
49
50      pr_debug("procfs_write: write %lu bytes\n", procfs_buffer_size);
51      return procfs_buffer_size;
52  }
53  static int procfs_open(struct inode *inode, struct file *file)
54  {
55      return 0;
56  }
57  static int procfs_close(struct inode *inode, struct file *file)
58  {
59      return 0;
60  }
61
62  #ifdef HAVE_PROC_OPS
63  static struct proc_ops file_ops_4_our_proc_file = {
64      .proc_read = procfs_read,
65      .proc_write = procfs_write,
66      .proc_open = procfs_open,
67      .proc_release = procfs_close,
68  };
69  #else
70  static const struct file_operations file_ops_4_our_proc_file = {
71      .read = procfs_read,
72      .write = procfs_write,
73      .open = procfs_open,
74      .release = procfs_close,
75  };
76  #endif
77
78  static int __init procfs3_init(void)
79  {
80      our_proc_file = proc_create(PROCFS_ENTRY_FILENAME, 0644, NULL,
81                                  &file_ops_4_our_proc_file);
82      if (our_proc_file == NULL) {
83          pr_debug("Error: Could not initialize /proc/%s\n",
84                   PROCFS_ENTRY_FILENAME);
85          return -ENOMEM;
86      }
87      proc_set_size(our_proc_file, 80);
88      proc_set_user(our_proc_file, GLOBAL_ROOT_UID, GLOBAL_ROOT_GID);
89
90      pr_debug("/proc/%s created\n", PROCFS_ENTRY_FILENAME);
91      return 0;
92  }
93
94  static void __exit procfs3_exit(void)
95  {
96      remove_proc_entry(PROCFS_ENTRY_FILENAME, NULL);
97      pr_debug("/proc/%s removed\n", PROCFS_ENTRY_FILENAME);
98  }
99
100 module_init(procfs3_init);
101 module_exit(procfs3_exit);
102
103 MODULE_LICENSE("GPL");
```

Still hungry for procfs examples? Well, first of all keep in mind, there are

rumors around, claiming that procfs is on its way out, consider using `sysfs` instead. Consider using this mechanism, in case you want to document something kernel related yourself.

7.4 Manage /proc file with seq_file

As we have seen, writing a `/proc` file may be quite "complex". So to help people writing `/proc` file, there is an API named `seq_file` that helps formatting a `/proc` file for output. It is based on sequence, which is composed of 3 functions: `start()`, `next()`, and `stop()`. The `seq_file` API starts a sequence when a user reads the `/proc` file.

A sequence begins with the call of the function `start()`. If the return is a non `NULL` value, the function `next()` is called; otherwise, the `stop()` function is called directly. This function is an iterator, the goal is to go through all the data. Each time `next()` is called, the function `show()` is also called. It writes data values in the buffer read by the user. The function `next()` is called until it returns `NULL`. The sequence ends when `next()` returns `NULL`, then the function `stop()` is called.

BE CAREFUL: when a sequence is finished, another one starts. That means that at the end of function `stop()`, the function `start()` is called again. This loop finishes when the function `start()` returns `NULL`. You can see a scheme of this in the Figure 2.1.

The `seq_file` provides basic functions for `proc_ops`, such as `seq_read`, `seq_lseek`, and some others. But nothing to write in the `/proc` file. Of course, you can still use the same way as in the previous example.

```
1  /*
2   * procfs4.c -  create a "file" in /proc
3   * This program uses the seq_file library to manage the /proc file.
4   */
5
6  #include <linux/kernel.h> /* We are doing kernel work */
7  #include <linux/module.h> /* Specifically, a module */
8  #include <linux/proc_fs.h> /* Necessary because we use proc fs */
9  #include <linux/seq_file.h> /* for seq_file */
10 #include <linux/version.h>
11
12 #if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 6, 0)
13 #define HAVE_PROC_OPS
14 #endif
15
16 #define PROC_NAME "iter"
17
18 /* This function is called at the beginning of a sequence.
19  * ie, when:
20  *   - the /proc file is read (first time)
21  *   - after the function stop (end of sequence)
22  */
23 static void *my_seq_start(struct seq_file *s, loff_t *pos)
24 {
25     static unsigned long counter = 0;
```

```
26
27      /* beginning a new sequence? */
28      if (*pos == 0) {
29          /* yes => return a non null value to begin the sequence */
30          return &counter;
31      }
32
33      /* no => it is the end of the sequence, return end to stop reading */
34      *pos = 0;
35      return NULL;
36  }
37
38  /* This function is called after the beginning of a sequence.
39   * It is called until the return is NULL (this ends the sequence).
40   */
41  static void *my_seq_next(struct seq_file *s, void *v, loff_t *pos)
42  {
43      unsigned long *tmp_v = (unsigned long *)v;
44      (*tmp_v)++;
45      (*pos)++;
46      return NULL;
47  }
48
49  /* This function is called at the end of a sequence. */
50  static void my_seq_stop(struct seq_file *s, void *v)
51  {
52      /* nothing to do, we use a static value in start() */
53  }
54
55  /* This function is called for each "step" of a sequence. */
56  static int my_seq_show(struct seq_file *s, void *v)
57  {
58      loff_t *spos = (loff_t *)v;
59
60      seq_printf(s, "%Ld\n", *spos);
61      return 0;
62  }
63
64  /* This structure gathers functions to manage the sequence */
65  static struct seq_operations my_seq_ops = {
66      .start = my_seq_start,
67      .next = my_seq_next,
68      .stop = my_seq_stop,
69      .show = my_seq_show,
70  };
71
72  /* This function is called when the /proc file is open. */
73  static int my_open(struct inode *inode, struct file *file)
74  {
75      return seq_open(file, &my_seq_ops);
76  };
77
78  /* This structure gathers functions that manage the /proc file */
79  #ifdef HAVE_PROC_OPS
80  static const struct proc_ops my_file_ops = {
81      .proc_open = my_open,
82      .proc_read = seq_read,
```

```
    .proc_lseek = seq_lseek,
    .proc_release = seq_release,
};
#else
static const struct file_operations my_file_ops = {
    .open = my_open,
    .read = seq_read,
    .llseek = seq_lseek,
    .release = seq_release,
};
#endif

static int __init procfs4_init(void)
{
    struct proc_dir_entry *entry;

    entry = proc_create(PROC_NAME, 0, NULL, &my_file_ops);
    if (entry == NULL) {
        pr_debug("Error: Could not initialize /proc/%s\n", PROC_NAME);
        return -ENOMEM;
    }

    return 0;
}

static void __exit procfs4_exit(void)
{
    remove_proc_entry(PROC_NAME, NULL);
    pr_debug("/proc/%s removed\n", PROC_NAME);
}

module_init(procfs4_init);
module_exit(procfs4_exit);

MODULE_LICENSE("GPL");
```

If you want more information, you can read this web page:

- https://lwn.net/Articles/22355/
- https://kernelnewbies.org/Documents/SeqFileHowTo

You can also read the code of fs/seq_file.c in the Linux kernel.

8 sysfs: Interacting with your module

sysfs allows you to interact with the running kernel from userspace by reading or setting variables inside of modules. This can be useful for debugging purposes, or just as an interface for applications or scripts. You can find sysfs directories and files under the `/sys` directory on your system.

```
ls -l /sys
```

Unlike `/proc`, which grew organically and mixes process information with driver knobs, sysfs is tightly tied to the kernel's device model and provides multiple coherent views of the same hardware:

- `/sys/devices/`: the actual device tree, rooted at the platform/CPU level.
- `/sys/bus/`: one subdirectory per bus type (PCI, USB, I2C, platform, ...), each containing `devices/` and `drivers/` subdirectories. Entries under `devices/` are symlinks back into `/sys/devices/`.
- `/sys/class/`: devices grouped by function (net, block, input, tty, ...) regardless of what bus they sit on.
- `/sys/module/`: one directory per module (including built-in ones), exposing parameters and, for loadable modules, reference count.
- `/sys/kernel/`: miscellaneous kernel-wide tunables.
- `/sys/firmware/`: firmware and ACPI/device-tree data.
- `/sys/power/`: power-management state.

Every directory in sysfs corresponds to a `struct kobject` in the kernel. Files inside those directories are kobject attributes; read or write operations on them invoke `show()` or `store()` callbacks inside the driver. This one-value-per-file convention keeps the interface simple and scriptable compared to the free-form output of many `/proc` entries.

Attributes can be exported for kobjects in the form of regular files in the filesystem. Sysfs forwards file I/O operations to methods defined for the attributes, providing a means to read and write kernel attributes.

A simple attribute definition:

```
struct attribute {
    char *name;
    struct module *owner;
    umode_t mode;
};

int sysfs_create_file(struct kobject * kobj, const struct attribute * attr);
void sysfs_remove_file(struct kobject * kobj, const struct attribute * attr);
```

For example, the driver model defines `struct device_attribute` like:

```
struct device_attribute {
    struct attribute attr;
    ssize_t (*show)(struct device *dev, struct device_attribute *attr,
                    char *buf);
    ssize_t (*store)(struct device *dev, struct device_attribute *attr,
                     const char *buf, size_t count);
};

```

```
int device_create_file(struct device *, const struct device_attribute *);
void device_remove_file(struct device *, const struct device_attribute *);
```

To read or write attributes, the `show()` or `store()` method must be specified when declaring the attribute. For the common cases include/linux/sysfs.h provides convenience macros (`__ATTR`, `__ATTR_RO`, `__ATTR_WO`, etc.) to make defining attributes easier as well as making code more concise and readable.

An example of a hello world module which includes the creation of a variable accessible via sysfs is given below.

```
/*
 * hello-sysfs.c sysfs example
 */
#include <linux/fs.h>
#include <linux/init.h>
#include <linux/kobject.h>
#include <linux/module.h>
#include <linux/string.h>
#include <linux/sysfs.h>

static struct kobject *mymodule;

/* the variable you want to be able to change */
static int myvariable = 0;

static ssize_t myvariable_show(struct kobject *kobj,
                               struct kobj_attribute *attr, char *buf)
{
    return sprintf(buf, "%d\n", myvariable);
}

static ssize_t myvariable_store(struct kobject *kobj,
                                struct kobj_attribute *attr, const char *buf,
                                size_t count)
{
    sscanf(buf, "%d", &myvariable);
    return count;
}

static struct kobj_attribute myvariable_attribute =
    __ATTR(myvariable, 0660, myvariable_show, myvariable_store);

static int __init mymodule_init(void)
{
    int error = 0;

    pr_info("mymodule: initialized\n");

    mymodule = kobject_create_and_add("mymodule", kernel_kobj);
    if (!mymodule)
        return -ENOMEM;

    error = sysfs_create_file(mymodule, &myvariable_attribute.attr);
    if (error) {
        kobject_put(mymodule);
```

```
46          pr_info("failed to create the myvariable file "
47                  "in /sys/kernel/mymodule\n");
48      }
49
50      return error;
51  }
52
53  static void __exit mymodule_exit(void)
54  {
55      pr_info("mymodule: Exit success\n");
56      kobject_put(mymodule);
57  }
58
59  module_init(mymodule_init);
60  module_exit(mymodule_exit);
61
62  MODULE_LICENSE("GPL");
```

Make and install the module:

```
1  make
2  sudo insmod hello-sysfs.ko
```

Check that it exists:

```
1  lsmod | grep hello_sysfs
```

What is the current value of `myvariable` ?

```
1  cat /sys/kernel/mymodule/myvariable
```

Set the value of `myvariable` and check that it changed.

```
1  echo "32" | sudo tee /sys/kernel/mymodule/myvariable
2  cat /sys/kernel/mymodule/myvariable
```

Finally, remove the test module:

```
1  sudo rmmod hello_sysfs
```

In the above case, we use a simple kobject to create a directory under sysfs, and communicate with its attributes. Since Linux v2.6.0, the `kobject` structure made its appearance. It was initially meant as a simple way of unifying kernel code which manages reference counted objects. After a bit of

mission creep, it is now the glue that holds much of the device model and its sysfs interface together. For more information about kobject and sysfs, see Documentation/driver-api/driver-model/driver.rst and https://lwn.net/Articles/51437/.

9 debugfs: Flexible debugging interfaces

debugfs is a lightweight filesystem for exporting debugging information from the kernel to userspace. Unlike `sysfs`, it is not intended to provide a stable userspace ABI, and unlike `/proc` it does not try to model processes or broader kernel state. That makes it a good fit for ad hoc diagnostics, counters, and developer-oriented control points while a module is under active development.

The canonical introduction is Documentation/filesystems/debugfs.rst, which describes both the design goals and the helper APIs. On most systems, debugfs is mounted at `/sys/kernel/debug`:

```
mount | grep debugfs
ls -l /sys/kernel/debug
```

To use the interface, include include/linux/debugfs.h and create a top-level directory for your module:

```
struct dentry *debugfs_create_dir(const char *name, struct dentry *parent);
```

Passing `NULL` as the parent places the new directory at the root of debugfs. When you unload the module, remove the whole subtree with `debugfs_remove_recursive()`. Note that `debugfs_create_dir()` and the other `debugfs_create_*()` helpers are designed to be used in a fire-and-forget fashion: their return values should not be checked. If debugfs is unavailable or an entry cannot be created, the helpers return an error pointer that is silently accepted by subsequent calls and by `debugfs_remove_recursive()`, so the module continues to work without its debug interface.

For simple scalar values, debugfs provides typed helpers that map a kernel variable directly to a file. Common examples include `debugfs_create_u32()`, `debugfs_create_bool()`, and the hexadecimal variants such as `debugfs_create_x32()`. These helpers are convenient when the data already lives in a fixed-size variable and does not need custom formatting.

The following example creates a debugfs directory and exports a writable integer plus a boolean flag:

```
/*
 * hello-debugfs.c - Export simple values through debugfs.
 */
#include <linux/debugfs.h>
```

```c
#include <linux/init.h>
#include <linux/module.h>
#include <linux/moduleparam.h>
#include <linux/printk.h>

MODULE_DESCRIPTION("Export simple values through debugfs");
MODULE_LICENSE("GPL");

#define DEBUGFS_DIR "lkmpg_debugfs"

static struct dentry *debugfs_dir;

static u32 debug_value = 42;
static bool debug_enabled = true;

module_param(debug_value, uint, 0600);
MODULE_PARM_DESC(debug_value, "Value exported through both sysfs and
↪  debugfs");

module_param(debug_enabled, bool, 0600);
MODULE_PARM_DESC(debug_enabled, "Enable or disable the example flag");

static int __init hello_debugfs_init(void)
{
    debugfs_dir = debugfs_create_dir(DEBUGFS_DIR, NULL);

    debugfs_create_u32("debug_value", 0600, debugfs_dir, &debug_value);
    debugfs_create_bool("debug_enabled", 0600, debugfs_dir, &debug_enabled);

    pr_info("debugfs interface registered under /sys/kernel/debug/%s\n",
            DEBUGFS_DIR);
    return 0;
}

static void __exit hello_debugfs_exit(void)
{
    debugfs_remove_recursive(debugfs_dir);
    pr_info("debugfs example removed\n");
}

module_init(hello_debugfs_init);
module_exit(hello_debugfs_exit);
```

If you need a custom text format or more control over reads and writes, create a file with your own `struct file_operations`:

```c
struct dentry *debugfs_create_file(const char *name, umode_t mode,
                                   struct dentry *parent, void *data,
                                   const struct file_operations *fops);
```

debugfs also includes helpers for a few common read-only payloads. One of them is `debugfs_create_blob()`, which exposes an existing memory region as a file.

The next example combines both approaches by creating a writable debugfs file and a read-only blob in the same directory:

```c
/*
 * hello-debugfs-file.c - Custom debugfs file and blob examples.
 */
#include <linux/debugfs.h>
#include <linux/fs.h>
#include <linux/init.h>
#include <linux/module.h>
#include <linux/mutex.h>
#include <linux/printk.h>
#include <linux/uaccess.h>

MODULE_DESCRIPTION("Custom debugfs file and blob examples");
MODULE_LICENSE("GPL");

#define DEBUGFS_DIR "lkmpg_debugfs_file"
#define MESSAGE_MAX_LEN 128

static struct dentry *debugfs_dir;
static DEFINE_MUTEX(message_lock);
static char message[MESSAGE_MAX_LEN] = "debugfs says hello\n";
static size_t message_len = sizeof("debugfs says hello\n") - 1;

static const char blob_data[] =
    "This blob is read-only and exported with debugfs_create_blob().\n";
static struct debugfs_blob_wrapper debug_blob = {
    .data = (void *)blob_data,
    .size = sizeof(blob_data) - 1,
};

static ssize_t message_read(struct file *file, char __user *buffer, size_t
↪  len,
                            loff_t *ppos)
{
    ssize_t ret;

    mutex_lock(&message_lock);
    ret = simple_read_from_buffer(buffer, len, ppos, message, message_len);
    mutex_unlock(&message_lock);

    return ret;
}

static ssize_t message_write(struct file *file, const char __user *buffer,
                             size_t len, loff_t *ppos)
{
    ssize_t copied;

    if (*ppos != 0)
        return -EINVAL;

    mutex_lock(&message_lock);
    copied =
        simple_write_to_buffer(message, sizeof(message) - 1, ppos, buffer,
        ↪  len);
```

```
53      if (copied > 0) {
54          message_len = *ppos;
55          message[message_len] = '\0';
56      }
57      mutex_unlock(&message_lock);
58
59      return copied;
60  }
61
62  static const struct file_operations message_fops = {
63      .owner = THIS_MODULE,
64      .read = message_read,
65      .write = message_write,
66      .llseek = default_llseek,
67  };
68
69  static int __init hello_debugfs_file_init(void)
70  {
71      debugfs_dir = debugfs_create_dir(DEBUGFS_DIR, NULL);
72
73      debugfs_create_file("message", 0600, debugfs_dir, NULL, &message_fops);
74      debugfs_create_blob("blob", 0400, debugfs_dir, &debug_blob);
75
76      pr_info("debugfs file example registered under /sys/kernel/debug/%s\n",
77              DEBUGFS_DIR);
78      return 0;
79  }
80
81  static void __exit hello_debugfs_file_exit(void)
82  {
83      debugfs_remove_recursive(debugfs_dir);
84      pr_info("debugfs file example removed\n");
85  }
86
87  module_init(hello_debugfs_file_init);
88  module_exit(hello_debugfs_file_exit);
```

Because debugfs is explicitly for debugging, its contents should be treated as implementation details rather than a supported external interface. If a setting must remain stable for tooling, scripts, or end users, prefer `sysfs` or another documented ABI instead.

10 Talking To Device Files

Device files are supposed to represent physical devices. Most physical devices are used for output as well as input, so there has to be some mechanism for device drivers in the kernel to get the output to send to the device from processes. This is done by opening the device file for output and writing to it, just like writing to a file. In the following example, this is implemented by `device_write`.

This is not always enough. Imagine you had a serial port connected to a modem (even if you have an internal modem, it is still implemented from the CPU's perspective as a serial port connected to a modem, so you don't have

to tax your imagination too hard). The natural thing to do would be to use the device file to write things to the modem (either modem commands or data to be sent through the phone line) and read things from the modem (either responses for commands or the data received through the phone line). However, this leaves open the question of what to do when you need to talk to the serial port itself, for example to configure the rate at which data is sent and received.

The answer in Unix is to use a special function called `ioctl` (short for Input Output ConTroL). Every device can have its own `ioctl` commands, which can be read ioctl operations (to send information from a process to the kernel), write ioctl operations (to return information to a process), both or neither. Notice here the roles of read and write are reversed again, so in ioctl operations, read is to send information to the kernel and write is to receive information from the kernel.

The ioctl function is called with three parameters: the file descriptor of the appropriate device file, the ioctl number, and a parameter, which is of type long so you can use a cast to use it to pass anything. You will not be able to pass a structure this way, but you will be able to pass a pointer to the structure. Here is an example:

```c
/*
 * ioctl.c
 */
#include <linux/cdev.h>
#include <linux/fs.h>
#include <linux/init.h>
#include <linux/ioctl.h>
#include <linux/module.h>
#include <linux/slab.h>
#include <linux/uaccess.h>
#include <linux/version.h>

struct ioctl_arg {
    unsigned int val;
};

/* Documentation/userspace-api/ioctl/ioctl-number.rst */
#define IOC_MAGIC '\x66'

#define IOCTL_VALSET _IOW(IOC_MAGIC, 0, struct ioctl_arg)
#define IOCTL_VALGET _IOR(IOC_MAGIC, 1, struct ioctl_arg)
#define IOCTL_VALGET_NUM _IOR(IOC_MAGIC, 2, int)
#define IOCTL_VALSET_NUM _IO(IOC_MAGIC, 3)

#define IOCTL_VAL_MAXNR 3
#define DRIVER_NAME "ioctltest"

static unsigned int test_ioctl_major = 0;
static unsigned int num_of_dev = 1;
static struct cdev test_ioctl_cdev;
static int ioctl_num = 0;

struct test_ioctl_data {
    unsigned char val;
```

```
35      rwlock_t lock;
36  };
37
38  static long test_ioctl_ioctl(struct file *filp, unsigned int cmd,
39                               unsigned long arg)
40  {
41      struct test_ioctl_data *ioctl_data = filp->private_data;
42      int retval = 0;
43      unsigned char val;
44      struct ioctl_arg data;
45      memset(&data, 0, sizeof(data));
46
47      switch (cmd) {
48      case IOCTL_VALSET:
49          if (copy_from_user(&data, (int __user *)arg, sizeof(data))) {
50              retval = -EFAULT;
51              goto done;
52          }
53
54          pr_alert("IOCTL set val:%x .\n", data.val);
55          write_lock(&ioctl_data->lock);
56          ioctl_data->val = data.val;
57          write_unlock(&ioctl_data->lock);
58          break;
59
60      case IOCTL_VALGET:
61          read_lock(&ioctl_data->lock);
62          val = ioctl_data->val;
63          read_unlock(&ioctl_data->lock);
64          data.val = val;
65
66          if (copy_to_user((int __user *)arg, &data, sizeof(data))) {
67              retval = -EFAULT;
68              goto done;
69          }
70
71          break;
72
73      case IOCTL_VALGET_NUM:
74          retval = __put_user(ioctl_num, (int __user *)arg);
75          break;
76
77      case IOCTL_VALSET_NUM:
78          ioctl_num = arg;
79          break;
80
81      default:
82          retval = -ENOTTY;
83      }
84
85  done:
86      return retval;
87  }
88
89  static ssize_t test_ioctl_read(struct file *filp, char __user *buf,
90                                 size_t count, loff_t *f_pos)
91  {
```

```
92      struct test_ioctl_data *ioctl_data = filp->private_data;
93      unsigned char val;
94      int retval;
95      int i = 0;
96
97      read_lock(&ioctl_data->lock);
98      val = ioctl_data->val;
99      read_unlock(&ioctl_data->lock);
100
101     for (; i < count; i++) {
102         if (copy_to_user(&buf[i], &val, 1)) {
103             retval = -EFAULT;
104             goto out;
105         }
106     }
107
108     retval = count;
109 out:
110     return retval;
111 }
112
113 static int test_ioctl_close(struct inode *inode, struct file *filp)
114 {
115     pr_alert("%s call.\n", __func__);
116
117     if (filp->private_data) {
118         kfree(filp->private_data);
119         filp->private_data = NULL;
120     }
121
122     return 0;
123 }
124
125 static int test_ioctl_open(struct inode *inode, struct file *filp)
126 {
127     struct test_ioctl_data *ioctl_data;
128
129     pr_alert("%s call.\n", __func__);
130     ioctl_data = kmalloc(sizeof(struct test_ioctl_data), GFP_KERNEL);
131
132     if (ioctl_data == NULL)
133         return -ENOMEM;
134
135     rwlock_init(&ioctl_data->lock);
136     ioctl_data->val = 0xFF;
137     filp->private_data = ioctl_data;
138
139     return 0;
140 }
141
142 static struct file_operations fops = {
143     .owner = THIS_MODULE,
144     .open = test_ioctl_open,
145     .release = test_ioctl_close,
146     .read = test_ioctl_read,
147     .unlocked_ioctl = test_ioctl_ioctl,
148 };
```

```
149
150 static int __init ioctl_init(void)
151 {
152     dev_t dev;
153     int ret;
154
155     ret = alloc_chrdev_region(&dev, 0, num_of_dev, DRIVER_NAME);
156
157     if (ret)
158         return ret;
159
160     test_ioctl_major = MAJOR(dev);
161     cdev_init(&test_ioctl_cdev, &fops);
162     ret = cdev_add(&test_ioctl_cdev, dev, num_of_dev);
163
164     if (ret) {
165         unregister_chrdev_region(dev, num_of_dev);
166         return ret;
167     }
168
169     pr_alert("%s driver(major: %d) installed.\n", DRIVER_NAME,
170              test_ioctl_major);
171     return 0;
172 }
173
174 static void __exit ioctl_exit(void)
175 {
176     dev_t dev = MKDEV(test_ioctl_major, 0);
177
178     cdev_del(&test_ioctl_cdev);
179     unregister_chrdev_region(dev, num_of_dev);
180     pr_alert("%s driver removed.\n", DRIVER_NAME);
181 }
182
183 module_init(ioctl_init);
184 module_exit(ioctl_exit);
185
186 MODULE_LICENSE("GPL");
187 MODULE_DESCRIPTION("This is test_ioctl module");
```

You can see there is an argument called `cmd` in `test_ioctl_ioctl()` function. It is the ioctl number. The ioctl number encodes the major device number, the type of the ioctl, the command, and the type of the parameter. This ioctl number is usually created by a macro call (`_IO`, `_IOR`, `_IOW` or `_IOWR` — depending on the type) in a header file. This header file should then be included both by the programs which will use ioctl (so they can generate the appropriate ioctl operations) and by the kernel module (so it can understand it). In the example below, the header file is `chardev.h` and the program which uses it is `userspace_ioctl.c`.

If you want to use ioctls in your own kernel modules, it is best to receive an official ioctl assignment, so if you accidentally get somebody else's ioctls, or if they get yours, you'll know something is wrong. For more information, consult the kernel source tree at Documentation/userspace-api/ioctl/ioctl-number.rst.

Also, we need to be careful that concurrent access to the shared resources will lead to the race condition. The solution is using atomic Compare-And-Swap (CAS), which we mentioned at Section 6.5, to enforce the exclusive access.

```c
/*
 * chardev2.c - Create an input/output character device
 */

#include <linux/atomic.h>
#include <linux/cdev.h>
#include <linux/delay.h>
#include <linux/device.h>
#include <linux/fs.h>
#include <linux/init.h>
#include <linux/module.h> /* Specifically, a module */
#include <linux/printk.h>
#include <linux/types.h>
#include <linux/uaccess.h> /* for get_user and put_user */
#include <linux/version.h>

#include <asm/errno.h>

#include "chardev.h"
#define DEVICE_NAME "char_dev"
#define BUF_LEN 80

enum {
    CDEV_NOT_USED,
    CDEV_EXCLUSIVE_OPEN,
};

/* Is the device open right now? Used to prevent concurrent access into
 * the same device
 */
static atomic_t already_open = ATOMIC_INIT(CDEV_NOT_USED);

/* The message the device will give when asked */
static char message[BUF_LEN + 1];

static struct class *cls;

/* This is called whenever a process attempts to open the device file */
static int device_open(struct inode *inode, struct file *file)
{
    pr_info("device_open(%p)\n", file);

    return 0;
}

static int device_release(struct inode *inode, struct file *file)
{
    pr_info("device_release(%p,%p)\n", inode, file);

    return 0;
}

/* This function is called whenever a process which has already opened the
```

```
 * device file attempts to read from it.
 */
static ssize_t device_read(struct file *file, /* see include/linux/fs.h   */
                           char __user *buffer, /* buffer to be filled  */
                           size_t length, /* length of the buffer     */
                           loff_t *offset)
{
    /* Number of bytes actually written to the buffer */
    int bytes_read = 0;
    /* How far did the process reading the message get? Useful if the message
     * is larger than the size of the buffer we get to fill in device_read.
     */
    const char *message_ptr = message;

    if (!*(message_ptr + *offset)) { /* we are at the end of message */
        *offset = 0; /* reset the offset */
        return 0; /* signify end of file */
    }

    message_ptr += *offset;

    /* Actually put the data into the buffer */
    while (length && *message_ptr) {
        /* Because the buffer is in the user data segment, not the kernel
         * data segment, assignment would not work. Instead, we have to
         * use put_user which copies data from the kernel data segment to
         * the user data segment.
         */
        put_user(*(message_ptr++), buffer++);
        length--;
        bytes_read++;
    }

    pr_info("Read %d bytes, %ld left\n", bytes_read, length);

    *offset += bytes_read;

    /* Read functions are supposed to return the number of bytes actually
     * inserted into the buffer.
     */
    return bytes_read;
}

/* called when somebody tries to write into our device file. */
static ssize_t device_write(struct file *file, const char __user *buffer,
                            size_t length, loff_t *offset)
{
    int i;

    pr_info("device_write(%p,%p,%ld)", file, buffer, length);

    for (i = 0; i < length && i < BUF_LEN; i++)
        get_user(message[i], buffer + i);

    /* Again, return the number of input characters used. */
    return i;
}
```

```
111
112 /* This function is called whenever a process tries to do an ioctl on our
113  * device file. We get two extra parameters (additional to the inode and file
114  * structures, which all device functions get): the number of the ioctl
    called
115  * and the parameter given to the ioctl function.
116  *
117  * If the ioctl is write or read/write (meaning output is returned to the
118  * calling process), the ioctl call returns the output of this function.
119  */
120 static long
121 device_ioctl(struct file *file, /* ditto */
122              unsigned int ioctl_num, /* number and param for ioctl */
123              unsigned long ioctl_param)
124 {
125     int i;
126     long ret = 0;
127
128     /* We don't want to talk to two processes at the same time. */
129     if (atomic_cmpxchg(&already_open, CDEV_NOT_USED, CDEV_EXCLUSIVE_OPEN))
130         return -EBUSY;
131
132     /* Switch according to the ioctl called */
133     switch (ioctl_num) {
134     case IOCTL_SET_MSG: {
135         /* Receive a pointer to a message (in user space) and set that to
136          * be the device's message. Get the parameter given to ioctl by
137          * the process.
138          */
139         char __user *tmp = (char __user *)ioctl_param;
140         char ch;
141
142         /* Find the length of the message */
143         get_user(ch, tmp);
144         for (i = 0; ch && i < BUF_LEN; i++, tmp++)
145             get_user(ch, tmp);
146
147         device_write(file, (char __user *)ioctl_param, i, NULL);
148         break;
149     }
150     case IOCTL_GET_MSG: {
151         loff_t offset = 0;
152
153         /* Give the current message to the calling process - the parameter
154          * we got is a pointer, fill it.
155          */
156         i = device_read(file, (char __user *)ioctl_param, 99, &offset);
157
158         /* Put a zero at the end of the buffer, so it will be properly
159          * terminated.
160          */
161         put_user('\0', (char __user *)ioctl_param + i);
162         break;
163     }
164     case IOCTL_GET_NTH_BYTE:
165         /* This ioctl is both input (ioctl_param) and output (the return
166          * value of this function).
```

```
167             */
168             ret = (long)message[ioctl_param];
169             break;
170         }
171 
172         /* We're now ready for our next caller */
173         atomic_set(&already_open, CDEV_NOT_USED);
174 
175         return ret;
176     }
177 
178     /* Module Declarations */
179 
180     /* This structure will hold the functions to be called when a process does
181      * something to the device we created. Since a pointer to this structure
182      * is kept in the devices table, it can't be local to init_module. NULL is
183      * for unimplemented functions.
184      */
185     static struct file_operations fops = {
186         .read = device_read,
187         .write = device_write,
188         .unlocked_ioctl = device_ioctl,
189         .open = device_open,
190         .release = device_release, /* a.k.a. close */
191     };
192 
193     /* Initialize the module - Register the character device */
194     static int __init chardev2_init(void)
195     {
196         /* Register the character device (at least try) */
197         int ret_val = register_chrdev(MAJOR_NUM, DEVICE_NAME, &fops);
198 
199         /* Negative values signify an error */
200         if (ret_val < 0) {
201             pr_alert("%s failed with %d\n",
202                      "Sorry, registering the character device ", ret_val);
203             return ret_val;
204         }
205 
206     #if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 4, 0)
207         cls = class_create(DEVICE_FILE_NAME);
208     #else
209         cls = class_create(THIS_MODULE, DEVICE_FILE_NAME);
210     #endif
211         if (IS_ERR(cls)) {
212             pr_err("Failed to create class for device\n");
213             unregister_chrdev(MAJOR_NUM, DEVICE_NAME);
214             return PTR_ERR(cls);
215         }
216         device_create(cls, NULL, MKDEV(MAJOR_NUM, 0), NULL, DEVICE_FILE_NAME);
217 
218         pr_info("Device created on /dev/%s\n", DEVICE_FILE_NAME);
219 
220         return 0;
221     }
222 
223     /* Cleanup - unregister the appropriate file from /proc */
```

```
static void __exit chardev2_exit(void)
{
    device_destroy(cls, MKDEV(MAJOR_NUM, 0));
    class_destroy(cls);

    /* Unregister the device */
    unregister_chrdev(MAJOR_NUM, DEVICE_NAME);
}

module_init(chardev2_init);
module_exit(chardev2_exit);

MODULE_LICENSE("GPL");
```

```
/*
 * chardev.h - the header file with the ioctl definitions.
 *
 * The declarations here have to be in a header file, because they need
 * to be known both to the kernel module (in chardev2.c) and the process
 * calling ioctl() (in userspace_ioctl.c).
 */

#ifndef CHARDEV_H
#define CHARDEV_H

#include <linux/ioctl.h>

/* The major device number. We can not rely on dynamic registration
 * any more, because ioctls need to know it.
 */
#define MAJOR_NUM 100

/* Set the message of the device driver */
#define IOCTL_SET_MSG _IOW(MAJOR_NUM, 0, char *)
/* _IOW means that we are creating an ioctl command number for passing
 * information from a user process to the kernel module.
 *
 * The first argument, MAJOR_NUM, is the major device number we are using.
 *
 * The second argument is the number of the command (there could be several
 * with different meanings).
 *
 * The third argument is the type we want to get from the process to the
 * kernel.
 */

/* Get the message of the device driver */
#define IOCTL_GET_MSG _IOR(MAJOR_NUM, 1, char *)
/* This IOCTL is used for output, to get the message of the device driver.
 * However, we still need the buffer to place the message in to be input,
 * as it is allocated by the process.
 */

/* Get the n'th byte of the message */
#define IOCTL_GET_NTH_BYTE _IOWR(MAJOR_NUM, 2, int)
/* The IOCTL is used for both input and output. It receives from the user
 * a number, n, and returns message[n].
```

```
 */

/* The name of the device file */
#define DEVICE_FILE_NAME "char_dev"
#define DEVICE_PATH "/dev/char_dev"

#endif
```

```
/*  userspace_ioctl.c - the process to use ioctl operations to control the
    kernel module
 *
 *  Until now we could have used cat for input and output.  But now
 *  we need to do ioctl operations, which require writing our own process.
 */

/* device specifics, such as ioctl numbers and the
 * major device file. */
#include "../chardev.h"

#include <stdio.h> /* standard I/O */
#include <fcntl.h> /* open */
#include <unistd.h> /* close */
#include <stdlib.h> /* exit */
#include <sys/ioctl.h> /* ioctl */

/* Functions for the ioctl calls */

int ioctl_set_msg(int file_desc, char *message)
{
    int ret_val;

    ret_val = ioctl(file_desc, IOCTL_SET_MSG, message);

    if (ret_val < 0) {
        printf("ioctl_set_msg failed:%d\n", ret_val);
    }

    return ret_val;
}

int ioctl_get_msg(int file_desc)
{
    int ret_val;
    char message[100] = { 0 };

    /* Warning - this is dangerous because we don't tell
   * the kernel how far it's allowed to write, so it
   * might overflow the buffer. In a real production
   * program, we would have used two ioctls - one to tell
   * the kernel the buffer length and another to give
   * it the buffer to fill
   */
    ret_val = ioctl(file_desc, IOCTL_GET_MSG, message);

    if (ret_val < 0) {
        printf("ioctl_get_msg failed:%d\n", ret_val);
    }
```

```
49      printf("get_msg message:%s", message);
50
51      return ret_val;
52  }
53
54  int ioctl_get_nth_byte(int file_desc)
55  {
56      int i, c;
57
58      printf("get_nth_byte message:");
59
60      i = 0;
61      do {
62          c = ioctl(file_desc, IOCTL_GET_NTH_BYTE, i++);
63
64          if (c < 0) {
65              printf("\nioctl_get_nth_byte failed at the %d'th byte:\n", i);
66              return c;
67          }
68
69          putchar(c);
70      } while (c != 0);
71
72      return 0;
73  }
74
75  /* Main - Call the ioctl functions */
76  int main(void)
77  {
78      int file_desc, ret_val;
79      char *msg = "Message passed by ioctl\n";
80
81      file_desc = open(DEVICE_PATH, O_RDWR);
82      if (file_desc < 0) {
83          printf("Can't open device file: %s, error:%d\n", DEVICE_PATH,
84                 file_desc);
85          exit(EXIT_FAILURE);
86      }
87
88      ret_val = ioctl_set_msg(file_desc, msg);
89      if (ret_val)
90          goto error;
91      ret_val = ioctl_get_nth_byte(file_desc);
92      if (ret_val)
93          goto error;
94      ret_val = ioctl_get_msg(file_desc);
95      if (ret_val)
96          goto error;
97
98      close(file_desc);
99      return 0;
100 error:
101     close(file_desc);
102     exit(EXIT_FAILURE);
103 }
```

10.1 poll, select, and epoll

The traditional `read()` and `write()` paths are not enough for every character device. Many programs need to wait on multiple file descriptors at once, and for that they use `poll()`, `select()`, or most commonly `epoll`. If a driver can block in `read()` or `write()`, it usually also needs a `poll` callback so userspace can discover when the operation would make progress without spinning.

The kernel-side implementation is centered on the `poll` file operation. The callback receives a `poll_table_struct`, and the usual pattern is to register the caller with one or more wait queues using `poll_wait()` before computing the readiness mask to return. That mask is built from `EPOLLIN`, `EPOLLOUT`, `EPOLLERR`, `EPOLLHUP`, and related bits. The names still mention epoll, but the same mask is used by `poll()` and `select()`.

Correct readiness reporting matters because modern event loops often rely on edge-triggered epoll. If a driver returns readable status without guaranteeing that a subsequent `read()` can consume data, or forgets to wake waiters after state changes, userspace can wedge in hard-to-debug ways. Likewise, returning `EPOLLHUP` or `EPOLLERR` too eagerly can make a device appear permanently dead. The rule of thumb is simple: the mask should describe what a nonblocking call would be able to do *right now*.

When device state changes, wake the relevant wait queue. If you want fine-grained wakeups, use `wake_up_interruptible_poll()` with the same event bits your `poll` callback reports. That allows userspace to sleep efficiently until the device becomes readable, writable, or otherwise transitions into a new state.

The sleep example in Section 12.1 already demonstrates the wait-queue side of blocking I/O. Adding `poll` support to a driver is mostly a matter of exposing that same state transition to event-driven applications instead of forcing them to block in one system call at a time.

10.2 Asynchronous notification

Another classic mechanism described in LDD3 is asynchronous notification via `SIGIO`. This is implemented with the `fasync` file operation and helpers such as `fasync_helper()` and `kill_fasync()`. When enabled from userspace with `fcntl(F\_SETFL, O\_ASYNC)`, the process receives a signal when the device becomes ready.

This interface still exists in Linux 5.10 and later, but it is much less common than `poll` and epoll-based designs. Signals are process-directed, relatively coarse, and awkward in multithreaded programs. For new user interfaces, a well-behaved `poll` implementation is usually the better default. Still, `fasync` remains useful for compatibility with older applications or for small control-style devices that only need to notify userspace that some event happened.

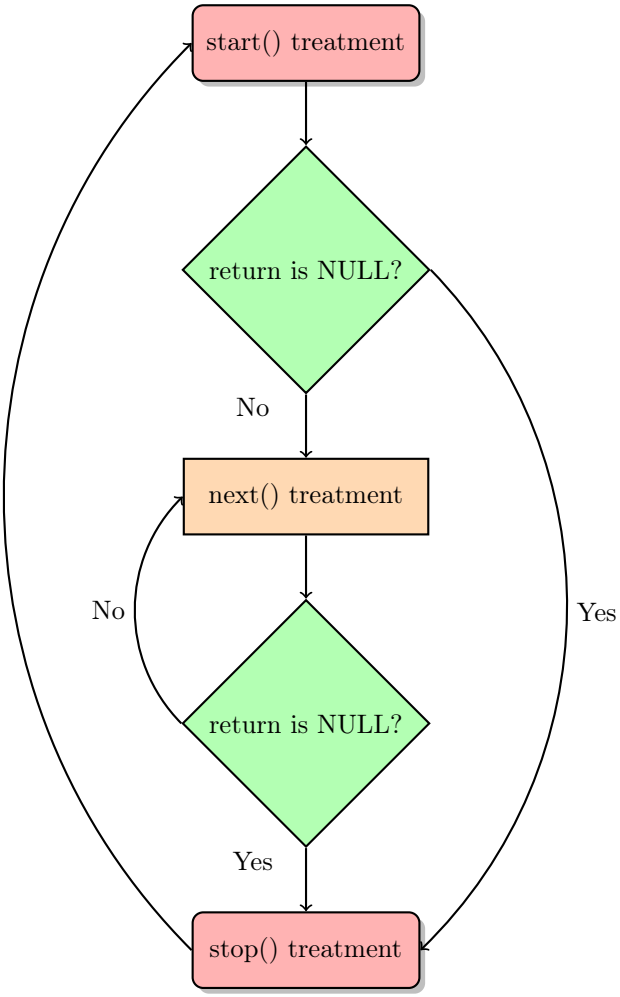


Figure 2.1: How seq_file works

3 Execution Context and Control Flow

Kernel code runs under a variety of constraints depending on where it executes. Some paths may sleep, some must return quickly, and some need careful synchronization to stay correct on modern SMP systems. This chapter collects the mechanisms that shape control flow, blocking behavior, deferred work, and debugging.

11 System Calls

So far, the only thing we've done was to use well defined kernel mechanisms to register `/proc` files and device handlers. This is fine if you want to do something the kernel programmers thought you'd want, such as write a device driver. But what if you want to do something unusual, to change the behavior of the system in some way? Then, you are mostly on your own.

Notice that this example has been unavailable since Linux v6.9. Specifically, after this commit, the system call dispatch mechanism changed from using an indirect function call table to a switch statement for security issues, such as mitigating Branch History Injection (BHI) attacks. See more information here.

Should one choose not to use a virtual machine, kernel programming can become risky. For example, while writing the code below, the `open()` system call was inadvertently disrupted. This resulted in an inability to open any files, run programs, or shut down the system, necessitating a restart of the virtual machine. Fortunately, no critical files were lost in this instance. However, if such modifications were made on a live, mission-critical system, the consequences could be severe. To mitigate the risk of file loss, even in a test environment, it is advised to execute `sync` right before using `insmod` and `rmmod`.

Forget about `/proc` files, forget about device files. They are just minor details. Minutiae in the vast expanse of the universe. The real process to kernel communication mechanism, the one used by all processes, is *system calls*. When a process requests a service from the kernel (such as opening a file, forking to a

new process, or requesting more memory), this is the mechanism used. If you want to change the behaviour of the kernel in interesting ways, this is the place to do it. By the way, if you want to see which system calls a program uses, run `strace <arguments>`.

In general, a process is not supposed to be able to access the kernel. It can not access kernel memory and it can't call kernel functions. The hardware of the CPU enforces this (that is the reason why it is called "protected mode" or "page protection").

System calls are an exception to this general rule. What happens is that the process fills the registers with the appropriate values and then calls a special instruction which jumps to a previously defined location in the kernel (of course, that location is readable by user processes, it is not writable by them). Under Intel CPUs, this is done by means of interrupt 0x80. The hardware knows that once you jump to this location, you are no longer running in restricted user mode, but as the operating system kernel — and therefore you're allowed to do whatever you want.

The location in the kernel a process can jump to is called `system_call`. The procedure at that location checks the system call number, which tells the kernel what service the process requested. Then, it looks at the table of system calls (`sys_call_table`) to see the address of the kernel function to call. Then it calls the function, and after it returns, does a few system checks and then return back to the process (or to a different process, if the process time ran out). If you want to read this code, it is at the source file `arch/$(architecture)/kernel/entry.S`, after the line `ENTRY(system_call)`.

So, if we want to change the way a certain system call works, what we need to do is to write our own function to implement it (usually by adding a bit of our own code, and then calling the original function) and then change the pointer at `sys_call_table` to point to our function. Because we might be removed later and we don't want to leave the system in an unstable state, it's important for `cleanup_module` to restore the table to its original state.

To modify the content of `sys_call_table`, we need to consider the control register. A control register is a processor register that changes or controls the general behavior of the CPU. For x86 architecture, the `cr0` register has various control flags that modify the basic operation of the processor. The `WP` flag in `cr0` stands for write protection. Once the `WP` flag is set, the processor disallows further write attempts to the read-only sections. Therefore, we must disable the `WP` flag before modifying `sys_call_table`. Since Linux v5.3, the `write_cr0` function cannot be used because of the sensitive `cr0` bits pinned by the security issue, the attacker may write into CPU control registers to disable CPU protections like write protection. As a result, we have to provide the custom assembly routine to bypass it.

However, `sys_call_table` symbol is unexported to prevent misuse. But there have few ways to get the symbol, manual symbol lookup and `kallsyms_lookup_name`. Here we use both depend on the kernel version.

Because of the *control-flow integrity*, which is a technique to prevent the redirect execution code from the attacker, for making sure that the indirect calls

go to the expected addresses and the return addresses are not changed. Since Linux v5.7, the kernel patched the series of *control-flow enforcement* (CET) for x86, and some configurations of GCC, like GCC versions 9 and 10 in Ubuntu Linux, will add with CET (the `-fcf-protection` option) in the kernel by default. Using that GCC to compile the kernel with retpoline off may result in CET being enabled in the kernel. You can use the following command to check out the `-fcf-protection` option is enabled or not:

```
$ gcc -v -Q -O2 --help=target | grep protection
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/lib/gcc/x86_64-linux-gnu/9/lto-wrapper
...
gcc version 9.3.0 (Ubuntu 9.3.0-17ubuntu1~20.04)
COLLECT_GCC_OPTIONS='-v' '-Q' '-O2' '--help=target' '-mtune=generic' '-march=x86-64'
 /usr/lib/gcc/x86_64-linux-gnu/9/cc1 -v ... -fcf-protection ...
 GNU C17 (Ubuntu 9.3.0-17ubuntu1~20.04) version 9.3.0 (x86_64-linux-gnu)
...
```

But CET should not be enabled in the kernel, it may break the Kprobes and bpf. Consequently, CET is disabled since v5.11. To guarantee the manual symbol lookup worked, we only use up to v5.4.

Unfortunately, since Linux v5.7 `kallsyms_lookup_name` is also unexported, it needs certain trick to get the address of `kallsyms_lookup_name`. If `CONFIG_KPROBES` is enabled, we can facilitate the retrieval of function addresses by means of Kprobes to dynamically break into the specific kernel routine. Kprobes inserts a breakpoint at the entry of function by replacing the first bytes of the probed instruction. When a CPU hits the breakpoint, registers are stored, and the control will pass to Kprobes. It passes the addresses of the saved registers and the Kprobe struct to the handler you defined, then executes it. Kprobes can be registered by symbol name or address. Within the symbol name, the address will be handled by the kernel.

Otherwise, specify the address of `sys_call_table` from `/proc/kallsyms` and `/boot/System.map` into `sym` parameter. Following is the sample usage for `/proc/kallsyms`:

```
$ sudo grep sys_call_table /proc/kallsyms
ffffffff82000280 R x32_sys_call_table
ffffffff820013a0 R sys_call_table
ffffffff820023e0 R ia32_sys_call_table
$ sudo insmod syscall-steal.ko sym=0xffffffff820013a0
```

Using the address from `/boot/System.map`, be careful about `KASLR` (Kernel Address Space Layout Randomization). `KASLR` may randomize the address of kernel code and data at every boot time, such as the static address listed in `/boot/System.map` will offset by some entropy. The purpose of `KASLR` is to protect the kernel space from the attacker. Without `KASLR`, the attacker

may find the target address in the fixed address easily. Then the attacker can use return-oriented programming to insert some malicious codes to execute or receive the target data by a tampered pointer. `KASLR` mitigates these kinds of attacks because the attacker cannot immediately know the target address, but a brute-force attack can still work. If the address of a symbol in `/proc/kallsyms` is different from the address in `/boot/System.map`, `KASLR` is enabled with the kernel, on which your system is running.

```
$ grep GRUB_CMDLINE_LINUX_DEFAULT /etc/default/grub
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
$ sudo grep sys_call_table /boot/System.map-$(uname -r)
ffffffff82000300 R sys_call_table
$ sudo grep sys_call_table /proc/kallsyms
ffffffff820013a0 R sys_call_table
# Reboot
$ sudo grep sys_call_table /boot/System.map-$(uname -r)
ffffffff82000300 R sys_call_table
$ sudo grep sys_call_table /proc/kallsyms
ffffffff86400300 R sys_call_table
```

If `KASLR` is enabled, we have to take care of the address from `/proc/kallsyms` each time we reboot the machine. In order to use the address from `/boot/System.map`, make sure that `KASLR` is disabled. You can add the `nokaslr` for disabling `KASLR` in next booting time:

```
$ grep GRUB_CMDLINE_LINUX_DEFAULT /etc/default/grub
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
$ sudo perl -i -pe 'm/quiet/ and s//quiet nokaslr/' /etc/default/grub
$ grep quiet /etc/default/grub
GRUB_CMDLINE_LINUX_DEFAULT="quiet nokaslr splash"
$ sudo update-grub
```

For more information, check out the following:

- Cook: Security things in Linux v5.3
- Unexporting the system call table
- Control-flow integrity for the kernel
- Unexporting kallsyms_lookup_name()
- Kernel Probes (Kprobes)
- Kernel address space layout randomization

The source code here is an example of such a kernel module. We want to "spy" on a certain user, and to `pr_info()` a message whenever that user opens a file. Towards this end, we replace the system call to open a file with our own

function, called `our_sys_openat`. This function checks the uid (user's id) of the current process, and if it is equal to the uid we spy on, it calls `pr_info()` to display the name of the file to be opened. Then, either way, it calls the original `openat()` function with the same parameters, to actually open the file.

The `init_module` function replaces the appropriate location in `sys_call_table` and keeps the original pointer in a variable. The `cleanup_module` function uses that variable to restore everything back to normal. This approach is dangerous, because of the possibility of two kernel modules changing the same system call. Imagine we have two kernel modules, A and B. A's openat system call will be `A_openat` and B's will be `B_openat`. Now, when A is inserted into the kernel, the system call is replaced with `A_openat`, which will call the original `sys_openat` when it is done. Next, B is inserted into the kernel, which replaces the system call with `B_openat`, which will call what it thinks is the original system call, `A_openat`, when it's done.

Now, if B is removed first, everything will be well — it will simply restore the system call to `A_openat`, which calls the original. However, if A is removed and then B is removed, the system will crash. A's removal will restore the system call to the original, `sys_openat`, cutting B out of the loop. Then, when B is removed, it will restore the system call to what it thinks is the original, `A_openat`, which is no longer in memory. At first glance, it appears we could solve this particular problem by checking if the system call is equal to our open function and if so not changing it at all (so that B won't change the system call when it is removed), but that will cause an even worse problem. When A is removed, it sees that the system call was changed to `B_openat` so that it is no longer pointing to `A_openat`, so it will not restore it to `sys_openat` before it is removed from memory. Unfortunately, `B_openat` will still try to call `A_openat` which is no longer there, so that even without removing B the system would crash.

For x86 architecture, the system call table cannot be used to invoke a system call after commit 1e3ad78 since v6.9. This commit has been backported to long term stable kernels, like v5.15.154+, v6.1.85+, v6.6.26+ and v6.8.5+, see this answer for more details. In this case, thanks to Kprobes, a hook can be used instead on the system call entry to intercept the system call.

Note that all the related problems make syscall stealing unfeasible for production use. In order to keep people from doing potentially harmful things `sys_call_table` is no longer exported. This means, if you want to do something more than a mere dry run of this example, you will have to patch your current kernel in order to have `sys_call_table` exported.

```
1  /*
2   * syscall-steal.c
3   *
4   * System call "stealing" sample.
5   *
6   * Disables page protection at a processor level by changing the 16th bit
7   * in the cr0 register (could be Intel specific).
8   */
```

```c

#include <linux/delay.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/moduleparam.h> /* which will have params */
#include <linux/unistd.h> /* The list of system calls */
#include <linux/cred.h> /* For current_uid() */
#include <linux/uidgid.h> /* For __kuid_val() */
#include <linux/version.h>

/* For the current (process) structure, we need this to know who the
 * current user is.
 */
#include <linux/sched.h>
#include <linux/uaccess.h>

/* The way we access "sys_call_table" varies as kernel internal changes.
 * - Prior to v5.4 : manual symbol lookup
 * - v5.5 to v5.6  : use kallsyms_lookup_name()
 * - v5.7+         : Kprobes or specific kernel module parameter
 */

/* The in-kernel calls to the ksys_close() syscall were removed in Linux
↪  v5.11+.
 */
#if (LINUX_VERSION_CODE >= KERNEL_VERSION(5, 7, 0))

#if defined(CONFIG_KPROBES)
#define HAVE_KPROBES 1
#if defined(CONFIG_X86_64)
/* If you have tried to use the syscall table to intercept syscalls and it
 * doesn't work, you can try to use Kprobes to intercept syscalls.
 * Set USE_KPROBES_PRE_HANDLER_BEFORE_SYSCALL to 1 to register a pre-handler
 * before the syscall.
 */
#define USE_KPROBES_PRE_HANDLER_BEFORE_SYSCALL 0
#endif
#include <linux/kprobes.h>
#else
#define HAVE_PARAM 1
#include <linux/kallsyms.h> /* For sprint_symbol */
/* The address of the sys_call_table, which can be obtained with looking up
 * "/boot/System.map" or "/proc/kallsyms". When the kernel version is v5.7+,
 * without CONFIG_KPROBES, you can input the parameter or the module will look
 * up all the memory.
 */
static unsigned long sym = 0;
module_param(sym, ulong, 0644);
#endif /* CONFIG_KPROBES */

#else

#if LINUX_VERSION_CODE <= KERNEL_VERSION(5, 4, 0)
#define HAVE_KSYS_CLOSE 1
#include <linux/syscalls.h> /* For ksys_close() */
#else
#include <linux/kallsyms.h> /* For kallsyms_lookup_name */
```

```
65  #endif
66
67  #endif /* Version >= v5.7 */
68
69  /* UID we want to spy on - will be filled from the command line. */
70  static uid_t uid = -1;
71  module_param(uid, int, 0644);
72
73  #if USE_KPROBES_PRE_HANDLER_BEFORE_SYSCALL
74
75  /* syscall_sym is the symbol name of the syscall to spy on. The default is
76   * "__x64_sys_openat", which can be changed by the module parameter. You can
77   * look up the symbol name of a syscall in /proc/kallsyms.
78   */
79  static char *syscall_sym = "__x64_sys_openat";
80  module_param(syscall_sym, charp, 0644);
81
82  static int sys_call_kprobe_pre_handler(struct kprobe *p, struct pt_regs *regs)
83  {
84      if (__kuid_val(current_uid()) != uid) {
85          return 0;
86      }
87
88      pr_info("%s called by %d\n", syscall_sym, uid);
89      return 0;
90  }
91
92  static struct kprobe syscall_kprobe = {
93      .symbol_name = "__x64_sys_openat",
94      .pre_handler = sys_call_kprobe_pre_handler,
95  };
96
97  #else
98
99  static unsigned long **sys_call_table_stolen;
100
101 /* A pointer to the original system call. We retain this instead of
102  * calling the original function (sys_openat), because another
103  * module might have replaced the system call before us. Note that this is not
104  * 100% safe, because if another module replaced sys_openat before us,
105  * then when we are inserted, we will call the function in that module -
106  * and it might be removed before we are.
107  *
108  * Another reason for this is that we can not get sys_openat.
109  * It is a static function, so it is not exported.
110  */
111 #ifdef CONFIG_ARCH_HAS_SYSCALL_WRAPPER
112 static asmlinkage long (*original_call)(const struct pt_regs *);
113 #else
114 static asmlinkage long (*original_call)(int, const char __user *, int,
    ↪  umode_t);
115 #endif
116
117 /* Our replacement function for sys_openat (the function called when you
118  * call the open system call). To find the exact prototype, with
119  * the number and type of arguments, we find the original function first
120  * (it is at fs/open.c).
```

```
 *
 * In theory, this means that we are tied to the current version of the
 * kernel. In practice, the system calls almost never change (it would
 * wreak havoc and require programs to be recompiled, since the system
 * calls are the interface between the kernel and the processes).
 */
#ifdef CONFIG_ARCH_HAS_SYSCALL_WRAPPER
static asmlinkage long our_sys_openat(const struct pt_regs *regs)
#else
static asmlinkage long our_sys_openat(int dfd, const char __user *filename,
                                      int flags, umode_t mode)
#endif
{
    int i = 0;
    char ch;

    if (__kuid_val(current_uid()) != uid)
        goto orig_call;

    /* Report the file, if relevant */
    pr_info("Opened file by %d: ", uid);
    do {
#ifdef CONFIG_ARCH_HAS_SYSCALL_WRAPPER
        get_user(ch, (char __user *)regs->si + i);
#else
        get_user(ch, (char __user *)filename + i);
#endif
        i++;
        pr_info("%c", ch);
    } while (ch != 0);
    pr_info("\n");

orig_call:
    /* Call the original sys_openat - otherwise, we lose the ability to
     * open files.
     */
#ifdef CONFIG_ARCH_HAS_SYSCALL_WRAPPER
    return original_call(regs);
#else
    return original_call(dfd, filename, flags, mode);
#endif
}

static unsigned long **acquire_sys_call_table(void)
{
#ifdef HAVE_KSYS_CLOSE
    unsigned long int offset = PAGE_OFFSET;
    unsigned long **sct;

    while (offset < ULLONG_MAX) {
        sct = (unsigned long **)offset;

        if (sct[__NR_close] == (unsigned long *)ksys_close)
            return sct;

        offset += sizeof(void *);
    }
```

```
178
179     return NULL;
180 #endif
181
182 #ifdef HAVE_PARAM
183     const char sct_name[15] = "sys_call_table";
184     char symbol[40] = { 0 };
185
186     if (sym == 0) {
187         pr_alert("For Linux v5.7+, Kprobes is the preferable way to get "
188                  "symbol.\n");
189         pr_info("If Kprobes is absent, you have to specify the address of "
190                 "sys_call_table symbol\n");
191         pr_info("by /boot/System.map or /proc/kallsyms, which contains all the
            ↪  "
192                 "symbol addresses, into sym parameter.\n");
193         return NULL;
194     }
195     sprint_symbol(symbol, sym);
196     if (!strncmp(sct_name, symbol, sizeof(sct_name) - 1))
197         return (unsigned long **)sym;
198
199     return NULL;
200 #endif
201
202 #ifdef HAVE_KPROBES
203     unsigned long (*kallsyms_lookup_name)(const char *name);
204     struct kprobe kp = {
205         .symbol_name = "kallsyms_lookup_name",
206     };
207
208     if (register_kprobe(&kp) < 0)
209         return NULL;
210     kallsyms_lookup_name = (unsigned long (*)(const char *name))kp.addr;
211     unregister_kprobe(&kp);
212 #endif
213
214     return (unsigned long **)kallsyms_lookup_name("sys_call_table");
215 }
216
217 #if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 3, 0)
218 static inline void __write_cr0(unsigned long cr0)
219 {
220     asm volatile("mov %0,%%cr0" : "+r"(cr0) : : "memory");
221 }
222 #else
223 #define __write_cr0 write_cr0
224 #endif
225
226 static void enable_write_protection(void)
227 {
228     unsigned long cr0 = read_cr0();
229     set_bit(16, &cr0);
230     __write_cr0(cr0);
231 }
232
233 static void disable_write_protection(void)
```

```
{
    unsigned long cr0 = read_cr0();
    clear_bit(16, &cr0);
    __write_cr0(cr0);
}
#endif

static int __init syscall_steal_start(void)
{
#if USE_KPROBES_PRE_HANDLER_BEFORE_SYSCALL
    int err;
    /* use symbol name from the module parameter */
    syscall_kprobe.symbol_name = syscall_sym;
    err = register_kprobe(&syscall_kprobe);
    if (err) {
        pr_err("register_kprobe() on %s failed: %d\n", syscall_sym, err);
        pr_err("Please check the symbol name from 'syscall_sym'
        ↪  parameter.\n");
        return err;
    }
#else
    if (!(sys_call_table_stolen = acquire_sys_call_table()))
        return -1;

    disable_write_protection();

    /* keep track of the original open function */
    original_call = (void *)sys_call_table_stolen[__NR_openat];

    /* use our openat function instead */
    sys_call_table_stolen[__NR_openat] = (unsigned long *)our_sys_openat;

    enable_write_protection();
#endif

    pr_info("Spying on UID:%d\n", uid);
    return 0;
}

static void __exit syscall_steal_end(void)
{
#if USE_KPROBES_PRE_HANDLER_BEFORE_SYSCALL
    unregister_kprobe(&syscall_kprobe);
#else
    if (!sys_call_table_stolen)
        return;

    /* Return the system call back to normal */
    if (sys_call_table_stolen[__NR_openat] != (unsigned long *)our_sys_openat)
    ↪  {
        pr_alert("Somebody else also played with the ");
        pr_alert("open system call\n");
        pr_alert("The system may be left in ");
        pr_alert("an unstable state.\n");
    }

    disable_write_protection();
```

```
289     sys_call_table_stolen[__NR_openat] = (unsigned long *)original_call;
290     enable_write_protection();
291 #endif
292 
293     msleep(2000);
294 }
295 
296 module_init(syscall_steal_start);
297 module_exit(syscall_steal_end);
298 
299 MODULE_LICENSE("GPL");
```

12 Blocking Processes and threads

12.1 Sleep

What do you do when somebody asks you for something you can not do right away? If you are a human being and you are bothered by a human being, the only thing you can say is: "*Not right now, I'm busy. Go away!*". But if you are a kernel module and you are bothered by a process, you have another possibility. You can put the process to sleep until you can service it. After all, processes are being put to sleep by the kernel and woken up all the time (that is the way multiple processes appear to run on the same time on a single CPU).

This kernel module is an example of this. The file (called `/proc/sleep`) can only be opened by a single process at a time. If the file is already open, the kernel module calls `wait_event_interruptible`. The easiest way to keep a file open is to open it with:

```
1 tail -f
```

This function changes the status of the task (a task is the kernel data structure which holds information about a process and the system call it is in, if any) to `TASK_INTERRUPTIBLE`, which means that the task will not run until it is woken up somehow, and adds it to WaitQ, the queue of tasks waiting to access the file. Then, the function calls the scheduler to context switch to a different process, one which has some use for the CPU.

When a process is done with the file, it closes it, and `module_close` is called. That function wakes up all the processes in the queue (there's no mechanism to only wake up one of them). It then returns and the process which just closed the file can continue to run. In time, the scheduler decides that that process has had enough and gives control of the CPU to another process. Eventually, one of the processes which was in the queue will be given control of the CPU by the scheduler. It starts at the point right after the call to `wait_event_interruptible`.

This means that the process is still in kernel mode - as far as the process is concerned, it issued the open system call and the system call has not returned

yet. The process does not know somebody else used the CPU for most of the time between the moment it issued the call and the moment it returned.

It can then proceed to set a global variable to tell all the other processes that the file is still open and go on with its life. When the other processes get a piece of the CPU, they'll see that global variable and go back to sleep.

So we will use `tail -f` to keep the file open in the background, and attempt to access it with another background process. This way, we don't need to switch to another terminal window or virtual terminal to run the second process. As soon as the first background process is killed with kill %1 , the second is woken up, is able to access the file and finally terminates.

To make our life more interesting, `module_close` does not have a monopoly on waking up the processes which wait to access the file. A signal, such as *Ctrl +c* (**SIGINT**) can also wake up a process. This is because we used `wait_event_interruptible`. We could have used `wait_event` instead, but that would have resulted in extremely angry users whose *Ctrl+c*'s are ignored.

In that case, we want to return with `-EINTR` immediately. This is important so users can, for example, kill the process before it receives the file.

There is one more point to remember. Some times processes don't want to sleep, they want either to get what they want immediately, or to be told it cannot be done. Such processes use the `O_NONBLOCK` flag when opening the file. The kernel is supposed to respond by returning with the error code `-EAGAIN` from operations which would otherwise block, such as opening the file in this example. The program `cat_nonblock`, available in the `examples/other` directory, can be used to open a file with `O_NONBLOCK`.

```
$ sudo insmod sleep.ko
$ cat_nonblock /proc/sleep
Last input:
$ tail -f /proc/sleep &
Last input:
Last input:
Last input:
Last input:
Last input:
Last input:
Last input:
tail: /proc/sleep: file truncated
[1] 6540
$ cat_nonblock /proc/sleep
Open would block
$ kill %1
[1]+  Terminated              tail -f /proc/sleep
$ cat_nonblock /proc/sleep
Last input:
$
```

```c
/*
 * sleep.c - create a /proc file, and if several processes try to open it
 * at the same time, put all but one to sleep.
 */

#include <linux/atomic.h>
#include <linux/fs.h>
#include <linux/kernel.h> /* for sprintf() */
#include <linux/module.h> /* Specifically, a module */
#include <linux/printk.h>
#include <linux/proc_fs.h> /* Necessary because we use proc fs */
#include <linux/types.h>
#include <linux/uaccess.h> /* for get_user and put_user */
#include <linux/version.h>
#include <linux/wait.h> /* For putting processes to sleep and
                                    waking them up */

#include <asm/current.h>
#include <asm/errno.h>

#if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 6, 0)
#define HAVE_PROC_OPS
#endif

/* Here we keep the last message received, to prove that we can process our
 * input.
 */
#define MESSAGE_LENGTH 80
static char message[MESSAGE_LENGTH];

static struct proc_dir_entry *our_proc_file;
#define PROC_ENTRY_FILENAME "sleep"

/* Since we use the file operations struct, we can't use the special proc
 * output provisions - we have to use a standard read function, which is this
 * function.
 */
static ssize_t module_output(struct file *file, /* see include/linux/fs.h   */
                             char __user *buf, /* The buffer to put data to
                                                  (in the user segment)    */
                             size_t len, /* The length of the buffer */
                             loff_t *offset)
{
    static int finished = 0;
    int i;
    char output_msg[MESSAGE_LENGTH + 30];

    /* Return 0 to signify end of file - that we have nothing more to say
     * at this point.
     */
    if (finished) {
        finished = 0;
        return 0;
    }

    sprintf(output_msg, "Last input:%s\n", message);
```

```
57      for (i = 0; i < len && output_msg[i]; i++)
58          put_user(output_msg[i], buf + i);
59
60      finished = 1;
61      return i; /* Return the number of bytes "read" */
62  }
63
64  /* This function receives input from the user when the user writes to the
65   * /proc file.
66   */
67  static ssize_t module_input(struct file *file, /* The file itself */
68                              const char __user *buf, /* The buffer with input
                                ↪  */
69                              size_t length, /* The buffer's length */
70                              loff_t *offset) /* offset to file - ignore */
71  {
72      int i;
73
74      /* Put the input into message, where module_output will later be able
75       * to use it.
76       */
77      for (i = 0; i < MESSAGE_LENGTH - 1 && i < length; i++)
78          get_user(message[i], buf + i);
79      /* we want a standard, zero terminated string */
80      message[i] = '\0';
81
82      /* We need to return the number of input characters used */
83      return i;
84  }
85
86  /* 1 if the file is currently open by somebody */
87  static atomic_t already_open = ATOMIC_INIT(0);
88
89  /* Queue of processes who want our file */
90  static DECLARE_WAIT_QUEUE_HEAD(waitq);
91
92  /* Called when the /proc file is opened */
93  static int module_open(struct inode *inode, struct file *file)
94  {
95      /* Try to get without blocking  */
96      if (!atomic_cmpxchg(&already_open, 0, 1)) {
97          /* Success without blocking, allow the access */
98          return 0;
99      }
100     /* If the file's flags include O_NONBLOCK, it means the process does not
101      * want to wait for the file. In this case, because the file is already
    ↪  open,
102      * we should fail with -EAGAIN, meaning "you will have to try again",
103      * instead of blocking a process which would rather stay awake.
104      */
105     if (file->f_flags & O_NONBLOCK)
106         return -EAGAIN;
107
108     while (atomic_cmpxchg(&already_open, 0, 1)) {
109         int i, is_sig = 0;
110
111         /* This function puts the current process, including any system
```

```
112             * calls, such as us, to sleep.  Execution will be resumed right
113             * after the function call, either because somebody called
114             * wake_up(&waitq) (only module_close does that, when the file
115             * is closed) or when a signal, such as Ctrl-C, is sent
116             * to the process
117             */
118            wait_event_interruptible(waitq, !atomic_read(&already_open));
119
120            /* If we woke up because we got a signal we're not blocking,
121             * return -EINTR (fail the system call).  This allows processes
122             * to be killed or stopped.
123             */
124            for (i = 0; i < _NSIG_WORDS && !is_sig; i++)
125                is_sig = current->pending.signal.sig[i] &
                    ↪  ~current->blocked.sig[i];
126
127            if (is_sig) {
128                /* Return -EINTR if we got a signal */
129                return -EINTR;
130            }
131        }
132
133        return 0; /* Allow the access */
134    }
135
136    /* Called when the /proc file is closed */
137    static int module_close(struct inode *inode, struct file *file)
138    {
139        /* Set already_open to zero, so one of the processes in the waitq will
140         * be able to set already_open back to one and to open the file. All
141         * the other processes will be called when already_open is back to one,
142         * so they'll go back to sleep.
143         */
144        atomic_set(&already_open, 0);
145
146        /* Wake up all the processes in waitq, so if anybody is waiting for the
147         * file, they can have it.
148         */
149        wake_up(&waitq);
150
151        return 0; /* success */
152    }
153
154    /* Structures to register as the /proc file, with pointers to all the relevant
155     * functions.
156     */
157
158    /* File operations for our /proc file. This is where we place pointers to all
159     * the functions called when somebody tries to do something to our file. NULL
160     * means we don't want to deal with something.
161     */
162    #ifdef HAVE_PROC_OPS
163    static const struct proc_ops file_ops_4_our_proc_file = {
164        .proc_read = module_output, /* "read" from the file */
165        .proc_write = module_input, /* "write" to the file */
166        .proc_open = module_open, /* called when the /proc file is opened */
167        .proc_release = module_close, /* called when it's closed */
```

```
    .proc_lseek = noop_llseek, /* return file->f_pos */
};
#else
static const struct file_operations file_ops_4_our_proc_file = {
    .read = module_output,
    .write = module_input,
    .open = module_open,
    .release = module_close,
    .llseek = noop_llseek,
};
#endif

/* Initialize the module - register the /proc file */
static int __init sleep_init(void)
{
    our_proc_file =
        proc_create(PROC_ENTRY_FILENAME, 0644, NULL,
        ↪  &file_ops_4_our_proc_file);
    if (our_proc_file == NULL) {
        pr_debug("Error: Could not initialize /proc/%s\n",
        ↪  PROC_ENTRY_FILENAME);
        return -ENOMEM;
    }
    proc_set_size(our_proc_file, 80);
    proc_set_user(our_proc_file, GLOBAL_ROOT_UID, GLOBAL_ROOT_GID);

    pr_info("/proc/%s created\n", PROC_ENTRY_FILENAME);

    return 0;
}

/* Cleanup - unregister our file from /proc.  This could get dangerous if
 * there are still processes waiting in waitq, because they are inside our
 * open function, which will get unloaded. I'll explain how to avoid removal
 * of a kernel module in such a case in chapter 10.
 */
static void __exit sleep_exit(void)
{
    remove_proc_entry(PROC_ENTRY_FILENAME, NULL);
    pr_debug("/proc/%s removed\n", PROC_ENTRY_FILENAME);
}

module_init(sleep_init);
module_exit(sleep_exit);

MODULE_LICENSE("GPL");
```

```
/*
 *  cat_nonblock.c - open a file and display its contents, but exit rather
↪  than
 *  wait for input.
 */
#include <errno.h> /* for errno */
#include <fcntl.h> /* for open */
#include <stdio.h> /* standard I/O */
#include <stdlib.h> /* for exit */
#include <unistd.h> /* for read */
```

```c

#define MAX_BYTES 1024 * 4

int main(int argc, char *argv[])
{
    int fd; /* The file descriptor for the file to read */
    size_t bytes; /* The number of bytes read */
    char buffer[MAX_BYTES]; /* The buffer for the bytes */

    /* Usage */
    if (argc != 2) {
        printf("Usage: %s <filename>\n", argv[0]);
        puts("Reads the content of a file, but doesn't wait for input");
        exit(EXIT_FAILURE);
    }

    /* Open the file for reading in non blocking mode */
    fd = open(argv[1], O_RDONLY | O_NONBLOCK);

    /* If open failed */
    if (fd == -1) {
        puts(errno == EAGAIN ? "Open would block" : "Open failed");
        exit(EXIT_FAILURE);
    }

    /* Read the file and output its contents */
    do {
        /* Read characters from the file */
        bytes = read(fd, buffer, MAX_BYTES);

        /* If there's an error, report it and die */
        if (bytes == -1) {
            if (errno == EAGAIN)
                puts("Normally I'd block, but you told me not to");
            else
                puts("Another read error");
            exit(EXIT_FAILURE);
        }

        /* Print the characters */
        if (bytes > 0) {
            for (int i = 0; i < bytes; i++)
                putchar(buffer[i]);
        }

        /* While there are no errors and the file isn't over */
    } while (bytes > 0);

    close(fd);
    return 0;
}
```

12.2 Completions

Sometimes one thing should happen before another within a module having multiple threads. Rather than using `/bin/sleep` commands, the kernel has another way to do this which allows timeouts or interrupts to also happen.

Completions as code synchronization mechanism have three main parts, initialization of struct completion synchronization object, the waiting or barrier part through `wait_for_completion()`, and the signalling side through a call to `complete()`.

In the subsequent example, two threads are initiated: crank and flywheel. It is imperative that the crank thread starts before the flywheel thread. A completion state is established for each of these threads, with a distinct completion defined for both the crank and flywheel threads. At the exit point of each thread the respective completion state is updated, and `wait_for_completion` is used by the flywheel thread to ensure that it does not begin prematurely. The crank thread uses the `complete_all()` function to update the completion, which lets the flywheel thread continue.

So even though `flywheel_thread` is started first you should notice when you load this module and run `dmesg`, that turning the crank always happens first because the flywheel thread waits for the crank thread to complete.

There are other variations of the `wait_for_completion` function, which include timeouts or being interrupted, but this basic mechanism is enough for many common situations without adding a lot of complexity.

```
1  /*
2   * completions.c
3   */
4  #include <linux/completion.h>
5  #include <linux/err.h> /* for IS_ERR() */
6  #include <linux/init.h>
7  #include <linux/kthread.h>
8  #include <linux/module.h>
9  #include <linux/printk.h>
10 #include <linux/version.h>
11
12 static struct completion crank_comp;
13 static struct completion flywheel_comp;
14
15 static int machine_crank_thread(void *arg)
16 {
17     pr_info("Turn the crank\n");
18
19     complete_all(&crank_comp);
20 #if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 17, 0)
21     kthread_complete_and_exit(&crank_comp, 0);
22 #else
23     complete_and_exit(&crank_comp, 0);
24 #endif
25 }
26
27 static int machine_flywheel_spinup_thread(void *arg)
28 {
```

```
29      wait_for_completion(&crank_comp);
30
31      pr_info("Flywheel spins up\n");
32
33      complete_all(&flywheel_comp);
34  #if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 17, 0)
35      kthread_complete_and_exit(&flywheel_comp, 0);
36  #else
37      complete_and_exit(&flywheel_comp, 0);
38  #endif
39  }
40
41  static int __init completions_init(void)
42  {
43      struct task_struct *crank_thread;
44      struct task_struct *flywheel_thread;
45
46      pr_info("completions example\n");
47
48      init_completion(&crank_comp);
49      init_completion(&flywheel_comp);
50
51      crank_thread = kthread_create(machine_crank_thread, NULL, "KThread
        ↪  Crank");
52      if (IS_ERR(crank_thread))
53          goto ERROR_THREAD_1;
54
55      flywheel_thread = kthread_create(machine_flywheel_spinup_thread, NULL,
56                                       "KThread Flywheel");
57      if (IS_ERR(flywheel_thread))
58          goto ERROR_THREAD_2;
59
60      wake_up_process(flywheel_thread);
61      wake_up_process(crank_thread);
62
63      return 0;
64
65  ERROR_THREAD_2:
66      kthread_stop(crank_thread);
67  ERROR_THREAD_1:
68
69      return -1;
70  }
71
72  static void __exit completions_exit(void)
73  {
74      wait_for_completion(&crank_comp);
75      wait_for_completion(&flywheel_comp);
76
77      pr_info("completions exit\n");
78  }
79
80  module_init(completions_init);
81  module_exit(completions_exit);
82
83  MODULE_DESCRIPTION("Completions example");
```

```
MODULE_LICENSE("GPL");
```

13 Synchronization

If processes running on different CPUs or in different threads try to access the same memory, then it is possible that strange things can happen or your system can lock up. To avoid this, various types of mutual exclusion kernel functions are available. These indicate if a section of code is "locked" or "unlocked" so that simultaneous attempts to run it can not happen.

13.1 Mutex

You can use kernel mutexes (mutual exclusions) in much the same manner that you might deploy them in userland. This may be all that is needed to avoid collisions in most cases.

Mutexes in the Linux kernel enforce strict ownership: only the task that successfully acquired the mutex can release (or unlock) it. Attempting to release a mutex held by another task or releasing an unheld mutex multiple times by the same task typically leads to errors or undefined behavior. If a task tries to lock a mutex it already holds, it may be blocked or sleep, where the task waits for itself to release the lock.

Before use, a mutex must be initialized through specific APIs (such as `mutex_init` or by using the `DEFINE_MUTEX` macro for compile-time initialization). And it is prohibited to directly modify the internal structure of a mutex using a memory manipulation function like `memset`.

```
/*
 * example_mutex.c
 */
#include <linux/module.h>
#include <linux/mutex.h>
#include <linux/printk.h>

static DEFINE_MUTEX(mymutex);

static int __init example_mutex_init(void)
{
    int ret;

    pr_info("example_mutex init\n");

    ret = mutex_trylock(&mymutex);
    if (ret != 0) {
        pr_info("mutex is locked\n");

        if (mutex_is_locked(&mymutex) == 0)
            pr_info("The mutex failed to lock!\n");

        mutex_unlock(&mymutex);
```

```
24          pr_info("mutex is unlocked\n");
25      } else
26          pr_info("Failed to lock\n");
27
28      return 0;
29  }
30
31  static void __exit example_mutex_exit(void)
32  {
33      pr_info("example_mutex exit\n");
34  }
35
36  module_init(example_mutex_init);
37  module_exit(example_mutex_exit);
38
39  MODULE_DESCRIPTION("Mutex example");
40  MODULE_LICENSE("GPL");
```

The various suffixes appended to mutex functions in the Linux kernel primarily dictate how a task waiting to acquire a lock will behave, particularly concerning its interruptibility.

When a task calls `mutex_lock()`, and if the mutex is currently unavailable, the task enters a sleep state until it can successfully obtain the lock. During this period, the task cannot be interrupted. In contrast, functions with the `_interruptible` suffix, such as `mutex_lock_interruptible()`, behave similarly to `mutex_lock()` but allow the waiting process to be interrupted by signals. If a task receives a signal (like a termination signal) while waiting for the lock, it will exit the waiting state and return an error code (`-EINTR`). This is useful for applications that need to handle external events even while waiting for a lock.

Beyond these fundamental locking behaviors, other mutex functions offer specialized capabilities. Functions like `mutex_lock_nested` and `mutex_lock_interruptible_nested()` incorporate the `__nested()` functionality, providing support for nested locking. This prior locking mechanism aids in managing lock acquisition and preventing deadlocks, often employing a subclass parameter for more precise deadlock detection. The latter variant combines nested locking with the ability for the waiting process to be interrupted by signals. Another function is `mutex_trylock()`, which attempts to acquire the mutex without blocking. It returns 1 if the lock is successfully acquired and 0 if the mutex is already held by another task.

Despite the fact that `mutex_trylock` does not sleep, it is still generally not safe for use in interrupt context because its implementation isn't atomic. If an interrupt occurs between checking the lock's availability and its acquisition, this can lead to race conditions and potential data corruption.

13.2 Spinlocks

As the name suggests, spinlocks lock up the CPU that the code is running on, taking 100% of its resources. Because of this you should only use the spinlock mechanism around code which is likely to take no more than a few milliseconds

to run and so will not noticeably slow anything down from the user's point of view.

The example here is `"irq safe"` in that if interrupts happen during the lock then they will not be forgotten and will activate when the unlock happens, using the `flags` variable to retain their state.

```c
/*
 * example_spinlock.c
 */
#include <linux/init.h>
#include <linux/module.h>
#include <linux/printk.h>
#include <linux/spinlock.h>

static DEFINE_SPINLOCK(sl_static);
static spinlock_t sl_dynamic;

static void example_spinlock_static(void)
{
    unsigned long flags;

    spin_lock_irqsave(&sl_static, flags);
    pr_info("Locked static spinlock\n");

    /* Do something or other safely. Because this uses 100% CPU time, this
     * code should take no more than a few milliseconds to run.
     */

    spin_unlock_irqrestore(&sl_static, flags);
    pr_info("Unlocked static spinlock\n");
}

static void example_spinlock_dynamic(void)
{
    unsigned long flags;

    spin_lock_init(&sl_dynamic);
    spin_lock_irqsave(&sl_dynamic, flags);
    pr_info("Locked dynamic spinlock\n");

    /* Do something or other safely. Because this uses 100% CPU time, this
     * code should take no more than a few milliseconds to run.
     */

    spin_unlock_irqrestore(&sl_dynamic, flags);
    pr_info("Unlocked dynamic spinlock\n");
}

static int __init example_spinlock_init(void)
{
    pr_info("example spinlock started\n");

    example_spinlock_static();
    example_spinlock_dynamic();

    return 0;
```

```
51  }
52
53  static void __exit example_spinlock_exit(void)
54  {
55      pr_info("example spinlock exit\n");
56  }
57
58  module_init(example_spinlock_init);
59  module_exit(example_spinlock_exit);
60
61  MODULE_DESCRIPTION("Spinlock example");
62  MODULE_LICENSE("GPL");
```

Taking 100% of a CPU's resources comes with greater responsibility. Situations where the kernel code monopolizes a CPU are called **atomic contexts**. Holding a spinlock is one of those situations. Sleeping in atomic contexts may leave the system hanging, as the occupied CPU devotes 100% of its resources doing nothing but sleeping. In some worse cases the system may crash. Thus, sleeping in atomic contexts is considered a bug in the kernel. They are sometimes called "sleep-in-atomic-context" in some materials.

Note that sleeping here is not limited to calling the sleep functions explicitly. If subsequent function calls eventually invoke a function that sleeps, it is also considered sleeping. Thus, it is important to pay attention to functions being used in atomic context. There's no documentation recording all such functions, but code comments may help. Sometimes you may find comments in kernel source code stating that a function "may sleep", "might sleep", or more explicitly "the caller should not hold a spinlock". Those comments are hints that a function may implicitly sleep and must not be called in atomic contexts.

Now, let's differentiate between a few types of spinlock functions in the Linux kernel: `spin_lock()`, `spin_lock_irq()`, `spin_lock_irqsave()`, and `spin_lock_bh()`.

`spin_lock()` does not allow the CPU to sleep while waiting for the lock, which makes it suitable for most use cases where the critical section is short. However, this is problematic for real-time Linux because spinlocks in this configuration behave as sleeping locks. This can prevent other tasks from running and cause the system to become unresponsive. To address this in real-time Linux environments, a `raw_spin_lock()` is used, which behaves similarly to a `spin_lock()` but without causing the system to sleep.

On non-`PREEMPT_RT` kernels, `spin_lock_irq()` disables local hard interrupts before acquiring the lock, and `spin_unlock_irq()` unconditionally re-enables them when the lock is released. Because this pair does not save the previous interrupt state, use it only when you know hard interrupts are enabled on entry and it is correct to re-enable them on exit. In contrast, `spin_lock_irqsave()` disables local hard interrupts and stores the previous state in an opaque `unsigned long flags` variable. The matching `spin_unlock_irqrestore()` restores that saved state, so it is safe regardless of whether hard interrupts were enabled or disabled before entering the critical section. This makes `spin_lock_irqsave()` the preferred interface when a function may be called from both process context and hardirq

context, or from any path where the caller's interrupt state is unknown. On `PREEMPT_RT` kernels, `spinlock_t` becomes an RT-mutex-based sleeping lock, so the `_irq` and `_irqsave` suffixes do not actually disable hardware interrupts. If you genuinely need hard interrupts disabled, use the `raw_spin_lock_irqsave()` family (see `raw_spin_lock()` above), but note that raw spinlocks really spin, disable preemption, and must not be held across any potentially sleeping operation.

Next, `spin_lock_bh()` disables **softirqs** (software interrupts) but allows hardware interrupts to continue. Unlike `spin_lock_irq()` and `spin_lock_irqsave()`, which disable both hardware and software interrupts, `spin_lock_bh()` is useful when hardware interrupts need to remain active.

A common deadlock scenario illustrates why the right variant matters. Suppose process-context code acquires a spinlock with plain `spin_lock()`, then a hardware interrupt fires on the same CPU. The interrupt handler tries to acquire the same lock and spins forever, because the CPU that holds the lock is the same CPU that needs to release it, but it is stuck in the interrupt handler. The solution: use `spin_lock_irqsave()` in the process-context path so that local interrupts are disabled while the lock is held. A hardirq top-half handler can normally use plain `spin_lock()` for that lock because local interrupts are already disabled on entry. Threaded IRQ handlers (registered with `request_threaded_irq()`) run in process context and follow process-context locking rules instead. The quick reference:

Data shared between ...	Recommended primitive
Process context only	`spin_lock()` / `mutex_lock()`
Process and softirq/tasklet	`spin_lock_bh()`
Process and hardirq	`spin_lock_irqsave()`
Hardirq and softirq	`spin_lock_irqsave()` (in softirq)

For more information about spinlock usage and lock types, see the following resources:

- Lesson 1: Spin locks
- Lock types and their rules

13.3 Read and write locks

Read and write locks are specialised kinds of spinlocks so that you can exclusively read from something or write to something. Like the earlier spinlocks example, the one below shows an "irq safe" situation in which if other functions were triggered from irqs which might also read and write to whatever you are concerned with then they would not disrupt the logic. As before it is a good idea to keep anything done within the lock as short as possible so that it does not hang up the system and cause users to start revolting against the tyranny of your module.

```c
/*
 * example_rwlock.c
 */
#include <linux/module.h>
#include <linux/printk.h>
#include <linux/rwlock.h>

static DEFINE_RWLOCK(myrwlock);

static void example_read_lock(void)
{
    unsigned long flags;

    read_lock_irqsave(&myrwlock, flags);
    pr_info("Read Locked\n");

    /* Read from something */

    read_unlock_irqrestore(&myrwlock, flags);
    pr_info("Read Unlocked\n");
}

static void example_write_lock(void)
{
    unsigned long flags;

    write_lock_irqsave(&myrwlock, flags);
    pr_info("Write Locked\n");

    /* Write to something */

    write_unlock_irqrestore(&myrwlock, flags);
    pr_info("Write Unlocked\n");
}

static int __init example_rwlock_init(void)
{
    pr_info("example_rwlock started\n");

    example_read_lock();
    example_write_lock();

    return 0;
}

static void __exit example_rwlock_exit(void)
{
    pr_info("example_rwlock exit\n");
}

module_init(example_rwlock_init);
module_exit(example_rwlock_exit);

MODULE_DESCRIPTION("Read/Write locks example");
MODULE_LICENSE("GPL");
```

Of course, if you know for sure that there are no functions triggered by irqs which could possibly interfere with your logic then you can use the simpler `read_lock(&myrwlock)` and `read_unlock(&myrwlock)` or the corresponding write functions.

13.4 Atomic operations

If you are doing simple arithmetic: adding, subtracting or bitwise operations, then there is another way in the multi-CPU and multi-hyperthreaded world to stop other parts of the system from messing with your mojo. By using atomic operations you can be confident that your addition, subtraction or bit flip did actually happen and was not overwritten by some other shenanigans. An example is shown below.

```
1  /*
2   * example_atomic.c
3   */
4  #include <linux/atomic.h>
5  #include <linux/bitops.h>
6  #include <linux/module.h>
7  #include <linux/printk.h>
8
9  #define BYTE_TO_BINARY_PATTERN "%c%c%c%c%c%c%c%c"
10 #define BYTE_TO_BINARY(byte)
   ↪  \
11     ((byte & 0x80) ? '1' : '0'), ((byte & 0x40) ? '1' : '0'),
   ↪  \
12         ((byte & 0x20) ? '1' : '0'), ((byte & 0x10) ? '1' : '0'),
   ↪  \
13         ((byte & 0x08) ? '1' : '0'), ((byte & 0x04) ? '1' : '0'),
   ↪  \
14         ((byte & 0x02) ? '1' : '0'), ((byte & 0x01) ? '1' : '0')
15
16 static void atomic_add_subtract(void)
17 {
18     atomic_t debbie;
19     atomic_t chris = ATOMIC_INIT(50);
20
21     atomic_set(&debbie, 45);
22
23     /* subtract one */
24     atomic_dec(&debbie);
25
26     atomic_add(7, &debbie);
27
28     /* add one */
29     atomic_inc(&debbie);
30
31     pr_info("chris: %d, debbie: %d\n", atomic_read(&chris),
32             atomic_read(&debbie));
33 }
34
35 static void atomic_bitwise(void)
36 {
```

```
37      unsigned long word = 0;
38
39      pr_info("Bits 0: " BYTE_TO_BINARY_PATTERN, BYTE_TO_BINARY(word));
40      set_bit(3, &word);
41      set_bit(5, &word);
42      pr_info("Bits 1: " BYTE_TO_BINARY_PATTERN, BYTE_TO_BINARY(word));
43      clear_bit(5, &word);
44      pr_info("Bits 2: " BYTE_TO_BINARY_PATTERN, BYTE_TO_BINARY(word));
45      change_bit(3, &word);
46
47      pr_info("Bits 3: " BYTE_TO_BINARY_PATTERN, BYTE_TO_BINARY(word));
48      if (test_and_set_bit(3, &word))
49          pr_info("wrong\n");
50      pr_info("Bits 4: " BYTE_TO_BINARY_PATTERN, BYTE_TO_BINARY(word));
51
52      word = 255;
53      pr_info("Bits 5: " BYTE_TO_BINARY_PATTERN "\n", BYTE_TO_BINARY(word));
54  }
55
56  static int __init example_atomic_init(void)
57  {
58      pr_info("example_atomic started\n");
59
60      atomic_add_subtract();
61      atomic_bitwise();
62
63      return 0;
64  }
65
66  static void __exit example_atomic_exit(void)
67  {
68      pr_info("example_atomic exit\n");
69  }
70
71  module_init(example_atomic_init);
72  module_exit(example_atomic_exit);
73
74  MODULE_DESCRIPTION("Atomic operations example");
75  MODULE_LICENSE("GPL");
```

Before the C11 standard adopted the built-in atomic types, the kernel already provided a small set of atomic types by using a bunch of tricky architecture-specific codes. Implementing the atomic types by C11 atomics may allow the kernel to throw away the architecture-specific codes and make the kernel code be more friendly to the people who understand the standard. But there are some problems, such as the memory model of the kernel doesn't match the model formed by the C11 atomics. For further details, see:

- kernel documentation of atomic types
- Time to move to C11 atomics?
- Atomic usage patterns in the kernel

13.5 Read-Copy-Update (RCU)

Read-Copy-Update (RCU) is a synchronization mechanism that allows extremely fast, lock-free reads while still permitting concurrent updates. It is highly optimized for read-mostly scenarios.

Unlike spinlocks or mutexes, RCU readers do not acquire any locks or write to shared memory. This completely avoids cache-line bouncing and results in near-zero overhead for the read path. The tradeoff is that the updaters bear the cost of synchronization. When modifying a data structure, an updater creates a copy, modifies it, and then publishes the new version by replacing the old pointer in a single atomic operation.

Because concurrent readers might still be accessing the old data, the updater cannot free it immediately. Instead, it must wait for a "grace period" to elapse—a duration long enough to ensure that all pre-existing readers have left their critical sections. Only then can the old memory be safely reclaimed.

The RCU API relies on a few fundamental primitives:

- `rcu_read_lock()` and `rcu_read_unlock()`: Define a read-side critical section. Readers must not sleep inside this section.
- `rcu_dereference()`: Safely fetches an RCU-protected pointer for readers, preventing stale or reordered pointer loads.
- `rcu_assign_pointer()`: Used by updaters to publish a new pointer.
- `synchronize_rcu()`: Blocks the updater until all pre-existing RCU readers have completed their critical sections.

Keep in mind that RCU only protects the readers. Concurrent writers must still serialize among themselves using a standard lock, such as a spinlock or a mutex. For more comprehensive details, you can refer to the official Linux kernel documentation on RCU: https://docs.kernel.org/RCU/whatisRCU.html.

The following example demonstrates a basic RCU implementation where a reader safely accesses shared data while a writer updates it.

```
1  #include <linux/module.h>
2  #include <linux/rcupdate.h>
3  #include <linux/slab.h>
4  #include <linux/spinlock.h>
5  #include <linux/kthread.h>
6  #include <linux/delay.h>
7
8  struct my_data {
9      int value;
10 };
11
12 static struct my_data __rcu *shared_data;
13 static DEFINE_SPINLOCK(data_lock);
14 static struct task_struct *reader_thread;
15
16 static int reader_thread_fn(void *data)
```

```
{
    while (!kthread_should_stop()) {
        struct my_data *p;
        int val;
        bool valid = false;

        rcu_read_lock();
        p = rcu_dereference(shared_data);
        if (p) {
            val = p->value;
            valid = true;
        }
        rcu_read_unlock();

        if (valid)
            pr_info("RCU Reader: value = %d\n", val);
        else
            pr_info("RCU Reader: data is NULL\n");

        msleep(500);
    }
    return 0;
}

static void update_data(int new_value)
{
    struct my_data *new_p, *old_p;

    new_p = kmalloc(sizeof(*new_p), GFP_KERNEL);
    if (!new_p)
        return;
    new_p->value = new_value;

    spin_lock(&data_lock);

    old_p = rcu_dereference_protected(shared_data,
    ↪  lockdep_is_held(&data_lock));
    rcu_assign_pointer(shared_data, new_p);

    spin_unlock(&data_lock);

    if (old_p) {
        synchronize_rcu();
        kfree(old_p);
        pr_info("RCU Updater: old memory freed\n");
    }
}

static int __init example_rcu_init(void)
{
    pr_info("Loading RCU example module\n");

    update_data(42);

    reader_thread = kthread_run(reader_thread_fn, NULL, "rcu_reader");
    if (IS_ERR(reader_thread))
        return PTR_ERR(reader_thread);
```

```
73
74      msleep(1000);
75      update_data(100);
76      msleep(1000);
77
78      return 0;
79  }
80
81  static void __exit example_rcu_exit(void)
82  {
83      struct my_data *old_p;
84
85      pr_info("Unloading RCU example module\n");
86
87      kthread_stop(reader_thread);
88
89      spin_lock(&data_lock);
90      old_p = rcu_dereference_protected(shared_data,
        ↪  lockdep_is_held(&data_lock));
91      RCU_INIT_POINTER(shared_data, NULL);
92      spin_unlock(&data_lock);
93
94      if (old_p) {
95          synchronize_rcu();
96          kfree(old_p);
97      }
98  }
99
100 module_init(example_rcu_init);
101 module_exit(example_rcu_exit);
102
103 MODULE_LICENSE("GPL");
104 MODULE_AUTHOR("Kuan-Wei Chiu <visitorckw@gmail.com>");
105 MODULE_DESCRIPTION("RCU Example Module");
```

14 Replacing Print Macros

14.1 Replacement

In Section 1.7, it was noted that the X Window System and kernel module programming are not conducive to integration. This remains valid during the development of kernel modules. However, in practical scenarios, the necessity emerges to relay messages to the tty (teletype) originating the module load command.

The term "tty" originates from *teletype*, which initially referred to a combined keyboard-printer for Unix system communication. Today, it signifies a text stream abstraction employed by Unix programs, encompassing physical terminals, xterms in X displays, and network connections like SSH.

To achieve this, the "current" pointer is leveraged to access the active task's tty structure. Within this structure lies a pointer to a string write function, facilitating the string's transmission to the tty.

```c
/*
 * print_string.c - Send output to the tty we're running on, regardless if
 * it is through X11, telnet, etc.  We do this by printing the string to the
 * tty associated with the current task.
 */
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/sched.h> /* For current */
#include <linux/tty.h> /* For the tty declarations */

static void print_string(char *str)
{
    /* The tty for the current task */
    struct tty_struct *my_tty = get_current_tty();

    /* If my_tty is NULL, the current task has no tty you can print to (i.e.,
     * if it is a daemon). If so, there is nothing we can do.
     */
    if (my_tty) {
        const struct tty_operations *ttyops = my_tty->driver->ops;
        /* my_tty->driver is a struct which holds the tty's functions,
         * one of which (write) is used to write strings to the tty.
         * It can be used to take a string either from the user's or
         * kernel's memory segment.
         *
         * The function's 1st parameter is the tty to write to, because the
         * same function would normally be used for all tty's of a certain
         * type.
         * The 2nd parameter is a pointer to a string.
         * The 3rd parameter is the length of the string.
         *
         * As you will see below, sometimes it's necessary to use
         * preprocessor stuff to create code that works for different
         * kernel versions. The (naive) approach we've taken here does not
         * scale well. The right way to deal with this is described in
         * section 2 of
         * linux/Documentation/SubmittingPatches
         */
        (ttyops->write)(my_tty, /* The tty itself */
                        str, /* String */
                        strlen(str)); /* Length */

        /* ttys were originally hardware devices, which (usually) strictly
         * followed the ASCII standard. In ASCII, to move to a new line you
         * need two characters, a carriage return and a line feed. On Unix,
         * the ASCII line feed is used for both purposes - so we can not
         * just use \n, because it would not have a carriage return and the
         * next line will start at the column right after the line feed.
         *
         * This is why text files are different between Unix and MS Windows.
         * In CP/M and derivatives, like MS-DOS and MS Windows, the ASCII
         * standard was strictly adhered to, and therefore a newline requires
         * both a LF and a CR.
         */
        (ttyops->write)(my_tty, "\015\012", 2);
```

```
    }
}

static int __init print_string_init(void)
{
    print_string("The module has been inserted.  Hello world!");
    return 0;
}

static void __exit print_string_exit(void)
{
    print_string("The module has been removed.  Farewell world!");
}

module_init(print_string_init);
module_exit(print_string_exit);

MODULE_LICENSE("GPL");
```

14.2 Flashing keyboard LEDs

In certain conditions, you may desire a simpler and more direct way to communicate to the external world. Flashing keyboard LEDs can be such a solution: It is an immediate way to attract attention or to display a status condition. Keyboard LEDs are present on every hardware, they are always visible, they do not need any setup, and their use is rather simple and non-intrusive, compared to writing to a tty or a file.

From v4.14 to v4.15, the timer API made a series of changes to improve memory safety. A buffer overflow in the area of a `timer_list` structure may be able to overwrite the `function` and `data` fields, providing the attacker with a way to use return-oriented programming (ROP) to call arbitrary functions within the kernel. Also, the function prototype of the callback, containing an `unsigned long` argument, will prevent the compiler from performing type checking. Furthermore, the function prototype with `unsigned long` argument may be an obstacle to the forward-edge protection of *control-flow integrity*. Thus, it is better to use a unique prototype to separate from the cluster that takes an `unsigned long` argument. The timer callback should be passed a pointer to the `timer_list` structure rather than an `unsigned long` argument. Then, it wraps all the information the callback needs, including the `timer_list` structure, into a larger structure, and it can use the `container_of` macro instead of the `unsigned long` value. For more information, see: Improving the kernel timers API.

Before Linux v4.14, `setup_timer` was used to initialize the timer and the `timer_list` structure looked like:

```
struct timer_list {
    unsigned long expires;
    void (*function)(unsigned long);
    unsigned long data;
    u32 flags;
```

```
    /* ... */
};

void setup_timer(struct timer_list *timer, void (*callback)(unsigned long),
                 unsigned long data);
```

Since Linux v4.14, `timer_setup` is adopted and the kernel step by step converting to `timer_setup` from `setup_timer`. One of the reasons why the API was changed is that it needed to coexist with the old version of the interface. Moreover, the `timer_setup` was implemented by `setup_timer` at first.

```
void timer_setup(struct timer_list *timer,
                 void (*callback)(struct timer_list *), unsigned int flags);
```

The `setup_timer` was then removed since v4.15. As a result, the `timer_list` structure had changed to the following.

```
struct timer_list {
    unsigned long expires;
    void (*function)(struct timer_list *);
    u32 flags;
    /* ... */
};
```

The following source code illustrates a minimal kernel module which, when loaded, starts blinking the keyboard LEDs until it is unloaded.

```
/*
 * kbleds.c - Blink keyboard leds until the module is unloaded.
 */

#include <linux/init.h>
#include <linux/kd.h> /* For KDSETLED */
#include <linux/module.h>
#include <linux/version.h>
#include <linux/tty.h> /* For tty_struct */
#include <linux/vt.h> /* For MAX_NR_CONSOLES */
#include <linux/vt_kern.h> /* for fg_console */
#include <linux/console_struct.h> /* For vc_cons */

MODULE_DESCRIPTION("Example module illustrating the use of Keyboard LEDs.");

static struct timer_list my_timer;
static struct tty_driver *my_driver;
static unsigned long kbledstatus = 0;

#define BLINK_DELAY HZ / 5
#define ALL_LEDS_ON 0x07
#define RESTORE_LEDS 0xFF

/* Function my_timer_func blinks the keyboard LEDs periodically by invoking
 * command KDSETLED of ioctl() on the keyboard driver. To learn more on
↪  virtual
```

```
 * terminal ioctl operations, please see file:
 *   drivers/tty/vt/vt_ioctl.c, function vt_ioctl().
 *
 * The argument to KDSETLED is alternatively set to 7 (thus causing the led
 * mode to be set to LED_SHOW_IOCTL, and all the leds are lit) and to 0xFF
 * (any value above 7 switches back the led mode to LED_SHOW_FLAGS, thus
 * the LEDs reflect the actual keyboard status).  To learn more on this,
 * please see file: drivers/tty/vt/keyboard.c, function setledstate().
 */
static void my_timer_func(struct timer_list *unused)
{
    struct tty_struct *t = vc_cons[fg_console].d->port.tty;

    if (kbledstatus == ALL_LEDS_ON)
        kbledstatus = RESTORE_LEDS;
    else
        kbledstatus = ALL_LEDS_ON;

    (my_driver->ops->ioctl)(t, KDSETLED, kbledstatus);

    my_timer.expires = jiffies + BLINK_DELAY;
    add_timer(&my_timer);
}

static int __init kbleds_init(void)
{
    int i;

    pr_info("kbleds: loading\n");
    pr_info("kbleds: fgconsole is %x\n", fg_console);
    for (i = 0; i < MAX_NR_CONSOLES; i++) {
        if (!vc_cons[i].d)
            break;
        pr_info("poet_atkm: console[%i/%i] #%i, tty %p\n", i, MAX_NR_CONSOLES,
                vc_cons[i].d->vc_num, (void *)vc_cons[i].d->port.tty);
    }
    pr_info("kbleds: finished scanning consoles\n");

    my_driver = vc_cons[fg_console].d->port.tty->driver;
    pr_info("kbleds: tty driver name %s\n", my_driver->driver_name);

    /* Set up the LED blink timer the first time. */
    timer_setup(&my_timer, my_timer_func, 0);
    my_timer.expires = jiffies + BLINK_DELAY;
    add_timer(&my_timer);

    return 0;
}

static void __exit kbleds_cleanup(void)
{
    pr_info("kbleds: unloading...\n");
#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 2, 0)
    timer_delete_sync(&my_timer);
#else
    del_timer_sync(&my_timer);
#endif
```

```
83      (my_driver->ops->ioctl)(vc_cons[fg_console].d->port.tty, KDSETLED,
84                              RESTORE_LEDS);
85  }
86
87  module_init(kbleds_init);
88  module_exit(kbleds_cleanup);
89
90  MODULE_LICENSE("GPL");
```

If none of the examples in this chapter fit your debugging needs, there might yet be some other tricks to try. Ever wondered what `CONFIG_DEBUG_LL` in `make menuconfig` is good for? If you activate that you get low level access to the serial port. While this might not sound very powerful by itself, you can patch kernel/printk/printk.c or any other essential kernel function to print ASCII characters, thus making it possible to trace virtually everything that your code does over a serial line. If you find yourself porting the kernel to some new and former unsupported architecture, this is usually amongst the first things that should be implemented. Logging over a netconsole might also be worth a try.

While you have seen lots of stuff that can be used to aid debugging here, there are some things to be aware of. Debugging is almost always intrusive. Adding debug code can change the situation enough to make the bug seem to disappear. Thus, you should keep debug code to a minimum and make sure it does not show up in production code.

4 Memory, Time, and Data Movement

LDD3 devoted separate chapters to time handling, memory allocation, and moving data between the CPU, userspace, and devices. Those topics are still central in Linux 5.10 and later, but the modern kernel expects more care around sleeping rules, lifetime management, and API selection. This chapter collects the concepts that most often determine whether a driver is merely functional or actually robust.

15 Timekeeping and Deferred Execution

Time in the kernel comes in several flavors, and choosing the wrong one is a common source of subtle bugs. For measuring elapsed intervals inside the kernel, prefer the monotonic clocks exposed through the ktime interfaces. For user-visible timestamps, use the real-time interfaces only if you truly want wall-clock behavior that can jump when the system clock is adjusted. Older code often leans heavily on jiffies; that is still valid for coarse timeouts, but newer code should treat jiffies as the low-resolution option rather than the universal time type.

Delays also need to match context. Busy-wait helpers such as `udelay()`, `ndelay()`, and `mdelay()` keep the CPU occupied and should be reserved for very short hardware sequences or atomic context where sleeping is forbidden. In process context, sleeping helpers such as `msleep()`, `usleep_range()`, and the wait-event family are usually preferable because they let the scheduler run other work. On a modern kernel, `usleep_range()` is often a better fit than `udelay()` once the delay is long enough that spinning would just waste CPU time and energy.

Kernel timers remain useful for deferred work that only needs a callback at some future point. As discussed in Section 14.2, recent kernels use `timer_setup()` and a typed `struct timer_list *` callback. For higher precision or explicit clock selection, `hrtimer` exists, but it comes with tighter constraints and is best

introduced only when ordinary timers are not precise enough.

Deferred work now has a clearer hierarchy than many older texts suggest. If the work may sleep or needs to call APIs that can sleep, queue it to a workqueue. If it must run in hardirq or softirq context, keep it short and think carefully about whether a threaded interrupt handler or workqueue would be simpler. Tasklets still exist in current kernels, but they are being phased out and are best treated as legacy infrastructure for existing code rather than the default tool for new modules. The examples in Section 22.1, Section 22.2, and Section 23.4 show these tradeoffs in increasing order of flexibility.

16 Memory Allocation and Ownership

Dynamic allocation is where driver correctness starts to depend on execution context. The first question is not "how much memory do I need?" but "am I allowed to sleep right now?". Allocations with `GFP_KERNEL` may sleep and are therefore appropriate for process context. Allocations from hard interrupt context, softirq context, or any region holding a raw spinlock must use non-sleeping flags such as `GFP_ATOMIC` and should be kept small. Using the wrong flag is one of the fastest ways to trigger "sleeping function called from invalid context" warnings.

For most small and medium-sized objects, `kmalloc()` and its relatives are the starting point. `kzalloc()` is preferred when the object contains fields that should begin life zeroed, and `kcalloc()` avoids integer-overflow mistakes when allocating arrays. If the size can become large or highly variable, `kvmalloc()` is often the modern choice because it tries `kmalloc()` first and falls back to `vmalloc()` when needed. That keeps the common case fast without forcing the caller to maintain two code paths.

`vmalloc()` has a different tradeoff profile. It provides virtually contiguous memory, not physically contiguous memory, and therefore has higher overhead and different cache/TLB behavior. It is useful for large software buffers, but it is not a substitute for memory that must be physically contiguous for DMA. Similarly, driver authors should resist the temptation to guess at allocator internals or to depend on low-level layout details that are not part of the API.

When a module needs to frequently allocate and free objects of the same size—such as custom data structures, network packet descriptors, or state containers—using `kmalloc()` repeatedly can lead to memory fragmentation and unnecessary overhead. For these highly repetitive allocations, the slab allocator interface is the correct tool. Driver authors should use `kmem_cache_create()` during module initialization to set up a dedicated cache for their specific structure. Once the cache is created, `kmem_cache_alloc()` and `kmem_cache_free()` provide extremely fast, fragmentation-resistant memory management for those objects. It is crucial to symmetrically destroy the cache with `kmem_cache_destroy()` during module cleanup to prevent memory leaks.

For a practical demonstration of creating a custom slab cache, allocating objects from it, and ensuring proper cleanup, see the following example.

```c
/*
 * kmem_cache.c - Demonstrates the use of the slab allocator.
 */
#include <linux/init.h>
#include <linux/module.h>
#include <linux/slab.h>

struct my_custom_struct {
    int id;
    char name[32];
};

static struct kmem_cache *my_cachep;
static struct my_custom_struct *obj1, *obj2;

static int __init kmem_cache_example_init(void)
{
    pr_info("kmem_cache_example: Initializing module\n");

    my_cachep =
        kmem_cache_create("my_custom_cache", sizeof(struct my_custom_struct),
        ↪  0,
                          SLAB_HWCACHE_ALIGN, NULL);

    if (!my_cachep) {
        pr_err("kmem_cache_example: Failed to create cache\n");
        return -ENOMEM;
    }

    obj1 = kmem_cache_alloc(my_cachep, GFP_KERNEL);
    obj2 = kmem_cache_alloc(my_cachep, GFP_KERNEL);

    if (!obj1 || !obj2) {
        pr_err("kmem_cache_example: Failed to allocate objects\n");
        if (obj1)
            kmem_cache_free(my_cachep, obj1);
        if (obj2)
            kmem_cache_free(my_cachep, obj2);
        kmem_cache_destroy(my_cachep);
        return -ENOMEM;
    }

    obj1->id = 1;
    obj2->id = 2;
    pr_info("kmem_cache_example: Allocated obj1 at %p (id: %d)\n", obj1,
            obj1->id);
    pr_info("kmem_cache_example: Allocated obj2 at %p (id: %d)\n", obj2,
            obj2->id);

    return 0;
}

static void __exit kmem_cache_example_exit(void)
{
    if (obj1)
        kmem_cache_free(my_cachep, obj1);
```

```
    if (obj2)
        kmem_cache_free(my_cachep, obj2);

    if (my_cachep)
        kmem_cache_destroy(my_cachep);

    pr_info("kmem_cache_example: Cleaned up and exited\n");
}

module_init(kmem_cache_example_init);
module_exit(kmem_cache_example_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Kuan-Wei Chiu <visitorckw@gmail.com>");
MODULE_DESCRIPTION("A simple example of kmem_cache");
```

Ownership is just as important as allocation. In Linux 5.10 and later, many drivers benefit from managed-resource helpers such as `devm_kzalloc()` and friends when they are written as proper device-model drivers with a `struct device`. Managed allocation ties resource cleanup to device detach and simplifies error paths considerably. For standalone modules or global objects, explicit unwind paths are still required, and they should be structured so that every successful acquisition has exactly one matching release.

Reference counting deserves dedicated attention. Do not use `atomic_t` as an ad hoc reference counter in new code. Use `refcount_t` or `kref` so overflow and misuse are easier to detect. When an object can outlive the operation that found it, the lifetime rules need to be stated clearly in code, not left as an implicit convention.

17 Copying Data Across the User-Kernel Boundary

Every character driver eventually has to move data between userspace and kernel space. The important rule is that a userspace pointer is not a normal pointer. It may fault, it may refer to memory that disappears after rescheduling, and it must only be touched through the `copy_to_user()`, `copy_from_user()`, `get_user()`, and `put_user()` families or other APIs that explicitly document `__user` handling.

Dereferencing a user pointer directly (e.g. `*ubuf`) is never correct, even if the address looks valid. The kernel cannot simply trap a page fault the way a user process does: an unhandled fault in kernel mode triggers an oops or a panic. The `copy_from_user()` family works because each potentially faulting instruction is recorded in a compiler-generated *exception table*. When a page fault occurs at one of those recorded addresses, the fault handler redirects execution to a fixup path that returns `-EFAULT` instead of crashing. Modern CPUs enforce this boundary in hardware. On x86, SMAP (Supervisor Mode Access Prevention) faults on any kernel-mode access to user pages unless the instruction is bracketed

by STAC/CLAC. On arm64, PAN (Privileged Access Never) provides the same protection. The uaccess helpers issue the required instructions on the kernel's behalf; raw pointer dereference does not, so it triggers an immediate hardware exception rather than a subtle data-corruption bug.

These helpers may sleep because servicing a page fault can require blocking. As a result, they must not be called while holding spinlocks, from hardirq context, or from any other atomic context. This is a frequent source of bugs when a driver evolves from a simple example into a version that adds locking later. The safe pattern is to gather the data you need while protected, drop the non-sleeping lock, and then copy to or from userspace.

Short copies are not automatically errors. The helpers return the number of bytes that could not be transferred, and drivers usually translate that into a successful partial read or write when some progress was made, or `-EFAULT` when nothing could be copied. Getting these corner cases right is part of presenting a sane userspace ABI. It is also good practice to validate lengths early, prefer fixed layouts for `ioctl` payloads, and reserve space in uapi structures for future extensions instead of breaking binary compatibility later.

18 Memory-Mapped I/O and Register Access

Once a driver starts talking to hardware registers, ordinary C pointer access is no longer enough. Device registers are memory-mapped I/O, not normal RAM, and they must be requested, mapped, and accessed through the proper kernel interfaces. On modern kernels, platform and PCI drivers usually rely on helpers such as `platform_get_resource()`, `devm_ioremap_resource()`, or the subsystem-specific wrappers that combine resource lookup and mapping.

After mapping, use the accessor APIs such as `readb()`, `readl()`, `writeb()`, and `writel()` rather than dereferencing pointers directly. Those accessors exist because different architectures impose different ordering and width requirements. Relaxed accessors are also available, but they should only be used when the driver author can explain the ordering guarantees at the boundary with the device. If you cannot state why relaxed ordering is correct, it probably is not the right default.

Port I/O still exists on some architectures and for some buses, but MMIO-backed devices are the dominant case for Linux 5.10-and-later driver work. Even when the underlying example hardware is simple, it is worth adopting the resource-management pattern used by real drivers because that is what scales to platform buses, PCI, and Device Tree described hardware.

19 DMA and mmap

Direct memory access is one of the areas where old folklore causes real damage. A driver must not assume that a CPU virtual address can be turned into a DMA address with `virt_to_phys()` and handed to hardware. The DMA

API exists because IOMMUs, cache coherency rules, and device addressing limits vary by architecture and platform. If a device performs DMA, use the DMA mapping interfaces and establish the correct mask with helpers such as `dma_set_mask_and_coherent()`.

The API splits broadly into coherent mappings and streaming mappings. Coherent allocations, typically created with `dma_alloc_coherent()`, are easy to reason about and are a good fit for descriptors or control rings. Streaming mappings, created with `dma_map_single()` or related helpers, are better for transient data buffers but require explicit unmapping and, on some systems, synchronization between CPU and device ownership. The core rule is that the buffer ownership protocol must be explicit: know when the CPU may touch the memory and when the device may touch it.

19.1 DMA Addressing and Ownership Rules

The first step in a DMA-capable driver is telling the kernel what address width the device can actually drive. That is the purpose of `dma_set_mask_and_coherent()` or its split variants. If a device can only issue 32-bit DMA addresses on a system with memory above 4 GiB, the kernel needs to know that before any buffers are allocated or mapped. Getting the mask wrong can lead to failures that only show up on certain machines, which is one reason DMA bugs are so frustrating to debug.

The second step is choosing the ownership model. For coherent allocations, the CPU and device see the same memory without explicit cache maintenance by the driver, so they are a natural fit for descriptor rings, producer-consumer indices, and other control structures. That does not mean no synchronization is needed. The driver still has to order accesses correctly with respect to doorbells, status bits, and descriptor visibility. Coherent memory solves cache-coherency problems, not higher-level protocol mistakes.

Streaming mappings are more performance-oriented but stricter. The driver maps a CPU buffer for a specific transfer direction with interfaces such as `dma_map_single()`, `dma_map_page()`, or scatter-gather helpers, hands the DMA address to the device, and later unmaps or synchronizes it. The important invariant is that the CPU must not treat the buffer as ordinary memory while the device owns it. On non-coherent systems, ignoring that rule can lead to stale or silently corrupted data; on coherent systems, it can still violate the driver's own protocol and create races.

19.2 Scatter-Gather and DMA API Patterns

Once transfers grow beyond a single small buffer, drivers usually stop thinking in terms of one DMA address and start thinking in terms of scatter-gather lists. The Linux DMA API supports this through `struct scatterlist` and helpers such as `sg_init_table()`, `dma_map_sg()`, and `dma_unmap_sg()`. That allows a driver to hand the device a list of segments without first copying everything into one physically contiguous buffer.

This matters for both throughput and allocation pressure. Large physically contiguous buffers are expensive and sometimes impossible to obtain on long-running systems. Scatter-gather lets the IOMMU or the device itself stitch the transfer together from ordinary pages, which is much closer to how high-performance storage and network devices actually operate.

The streaming DMA direction also deserves care. Use the correct direction flag, such as `DMA_TO_DEVICE`, `DMA_FROM_DEVICE`, or `DMA_BIDIRECTIONAL`, because the architecture and IOMMU code use that information to apply the right synchronization rules. `DMA_BIDIRECTIONAL` is not a harmless default; it is a weaker statement of intent and can impose avoidable costs.

19.3 DMA Debugging and Common Mistakes

When bringing up a DMA driver, enable the kernel's DMA-API debugging facilities if the configuration supports them. They can catch double unmaps, direction mismatches, mapping leaks, and other lifetime errors early enough to be understandable. The most common beginner mistakes are all variations of the same theme: forgetting that the DMA address space is an abstraction.

Do not mix CPU physical addresses with DMA addresses. Do not keep using a streaming-mapped buffer after unmapping it. Do not map memory that the device will never actually access. Do not assume that a successful mapping on one machine means the code is portable to systems with non-coherent caches, SWIOTLB bounce buffering, or an active IOMMU. If a driver stays disciplined about those rules, the DMA API becomes manageable rather than mysterious.

The following sample creates a synthetic platform device so the module can walk through a realistic DMA setup path without depending on external hardware. It sets a DMA mask, allocates coherent memory, performs a streaming mapping, and logs the resulting DMA addresses during probe. The sample stays within interfaces available on Linux v5.10, so it also builds cleanly on newer 6.x kernels without needing 6.x-only driver callbacks:

```
// SPDX-License-Identifier: GPL-2.0
/*
 * dma.c - Minimal DMA API demonstration using a synthetic platform device.
 *
 * This module does not drive real hardware. It exists to show the shape of a
 * modern DMA-capable probe path: set the DMA mask, allocate coherent memory,
 * create a streaming mapping, and tear it all down cleanly.
 */

#include <linux/dma-mapping.h>
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/platform_device.h>
#include <linux/slab.h>
#include <linux/version.h>

static unsigned int coherent_size = PAGE_SIZE;
module_param(coherent_size, uint, 0444);
```

```
MODULE_PARM_DESC(coherent_size, "Size of the coherent DMA buffer");

static unsigned int streaming_size = PAGE_SIZE;
module_param(streaming_size, uint, 0444);
MODULE_PARM_DESC(streaming_size, "Size of the streaming DMA buffer");

static u64 dma_demo_mask = DMA_BIT_MASK(32);

struct dma_demo_dev {
    void *coherent_buf;
    dma_addr_t coherent_handle;
    void *streaming_buf;
    dma_addr_t streaming_handle;
    size_t coherent_len;
    size_t streaming_len;
};

static int dma_demo_probe(struct platform_device *pdev)
{
    struct device *dev = &pdev->dev;
    struct dma_demo_dev *demo;
    int ret;

    ret = dma_set_mask_and_coherent(dev, DMA_BIT_MASK(32));
    if (ret)
        return dev_err_probe(dev, ret, "failed to set DMA mask\n");

    demo = devm_kzalloc(dev, sizeof(*demo), GFP_KERNEL);
    if (!demo)
        return -ENOMEM;

    demo->coherent_len = coherent_size;
    demo->streaming_len = streaming_size;

    demo->coherent_buf = dmam_alloc_coherent(
        dev, demo->coherent_len, &demo->coherent_handle, GFP_KERNEL);
    if (!demo->coherent_buf)
        return dev_err_probe(dev, -ENOMEM,
                             "failed to allocate coherent buffer\n");

    demo->streaming_buf = devm_kzalloc(dev, demo->streaming_len, GFP_KERNEL);
    if (!demo->streaming_buf)
        return -ENOMEM;

    memset(demo->streaming_buf, 0x5a, demo->streaming_len);

    demo->streaming_handle = dma_map_single(dev, demo->streaming_buf,
                                            demo->streaming_len,
                                            ↪  DMA_TO_DEVICE);
    if (dma_mapping_error(dev, demo->streaming_handle))
        return dev_err_probe(dev, -EIO, "failed to map streaming buffer\n");

    dma_unmap_single(dev, demo->streaming_handle, demo->streaming_len,
                     DMA_TO_DEVICE);

    platform_set_drvdata(pdev, demo);

```

```
76      dev_info(dev, "coherent=%pad/%zu streaming=%zu DMA mask=0x%llx\n",
77               &demo->coherent_handle, demo->coherent_len, demo->streaming_len,
78               (unsigned long long)*dev->dma_mask);
79
80      return 0;
81  }
82
83  /*
84   * The .remove callback changed from int to void return in Linux 6.11.
85   * Between 5.18 and 6.10 the void variant was spelled .remove_new while
86   * .remove still returned int.  On 5.10-5.17 only .remove (int) exists.
87   */
88  #if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 18, 0)
89  static void dma_demo_remove(struct platform_device *pdev)
90  {
91      dev_info(&pdev->dev, "DMA demo removed\n");
92  }
93  #else
94  static int dma_demo_remove(struct platform_device *pdev)
95  {
96      dev_info(&pdev->dev, "DMA demo removed\n");
97
98      return 0;
99  }
100 #endif
101
102 static struct platform_driver dma_demo_driver = {
103         .probe = dma_demo_probe,
104 #if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 11, 0)
105         .remove = dma_demo_remove,
106 #elif LINUX_VERSION_CODE >= KERNEL_VERSION(5, 18, 0)
107         .remove_new = dma_demo_remove,
108 #else
109         .remove = dma_demo_remove,
110 #endif
111         .driver = {
112                 .name = "lkmpg_dma_demo",
113         },
114 };
115
116 static struct platform_device *dma_demo_pdev;
117
118 static int __init dma_demo_init(void)
119 {
120     int ret;
121
122     dma_demo_pdev =
123         platform_device_alloc("lkmpg_dma_demo", PLATFORM_DEVID_NONE);
124     if (!dma_demo_pdev)
125         return -ENOMEM;
126
127     dma_demo_pdev->dev.dma_mask = &dma_demo_mask;
128     dma_demo_pdev->dev.coherent_dma_mask = DMA_BIT_MASK(32);
129
130     ret = platform_device_add(dma_demo_pdev);
131     if (ret) {
132         platform_device_put(dma_demo_pdev);
```

```
        return ret;
    }

    ret = platform_driver_register(&dma_demo_driver);
    if (ret) {
        platform_device_unregister(dma_demo_pdev);
        return ret;
    }

    return 0;
}

static void __exit dma_demo_exit(void)
{
    platform_driver_unregister(&dma_demo_driver);
    platform_device_unregister(dma_demo_pdev);
}

module_init(dma_demo_init);
module_exit(dma_demo_exit);

MODULE_DESCRIPTION("LKMPG DMA API demo module");
MODULE_LICENSE("GPL");
```

Some drivers also map buffers into userspace with `mmap`. That can provide zero-copy data paths, but it introduces its own lifetime and coherency problems. The `mmap` file operation is therefore not just a performance feature; it is an ABI commitment. If a driver exposes memory this way, it must define what happens across fork, device removal, short-lived mappings, and cache visibility boundaries. That is why many drivers begin with `read()` and `write()` interfaces and only add `mmap` when measurement shows the extra complexity is worth it.

20 Modern Interface Choices for Linux 5.10 and Later

One of the harder parts of learning from LDD3 is separating durable concepts from interfaces that are now discouraged. The conceptual material remains excellent; the concrete APIs often need translation. Several examples in this guide intentionally preserve older techniques for teaching purposes, but new production drivers should make different default choices.

For procfs, prefer `struct proc_ops`, as shown in Section 7.1, rather than older `file_operations`-based registration patterns. For GPIO, the descriptor-based `gpiod_*` API is the preferred direction for new drivers, while the legacy integer GPIO API is best viewed as compatibility infrastructure and example material. For deferred work, choose workqueues or threaded interrupts before tasklets. For reference counts, choose `refcount_t` or `kref` before `atomic_t`. For device-managed drivers, prefer `devm_*` helpers so probe error paths and remove paths stay understandable.

The strongest warning applies to syscall-table patching. As described in Section 11, direct system-call hooking is increasingly blocked by modern kernel hardening and is not a viable design pattern on current Linux 5.10-and-later kernels. Treat it as a historical and diagnostic example, not as a foundation for new module work.

5

Hardware Integration and Advanced Topics

The remaining material brings together hardware-facing examples and the kernel subsystems that support them. It also closes the guide with a compact set of optimization notes, common mistakes, and references for readers who want to keep exploring beyond the examples in this book.

21 GPIO

21.1 GPIO

General Purpose Input/Output (GPIO) appears on the development board as pins. It acts as a bridge for communication between the development board and external devices. You can think of it like a switch: users can turn it on or off (Input), and the development board can also turn it on or off (Output).

To implement a GPIO device driver, you use the `gpio_request()` function to enable a specific GPIO pin. After successfully enabling it, you can check that the pin is being used by looking at /sys/kernel/debug/gpio.

```
1 cat /sys/kernel/debug/gpio
```

There are other ways to register GPIOs. For example, you can use `gpio_request_one()` to register a GPIO while setting its direction (input or output) and initial state at the same time. You can also use `gpio_request_array()` to register multiple GPIOs at once. However, note that `gpio_request_array()` has been removed since Linux v6.10.

When using GPIO, you must set it as either output with `gpio_direction_output()` or input with `gpio_direction_input()`.

- when the GPIO is set as output, you can use `gpio_set_value()` to choose to set it to high voltage or low voltage.

- when the GPIO is set as input, you can use `gpio_get_value()` to read whether the voltage is high or low.

21.2 Control the LED's on/off state

In Section 10, we learned how to communicate with device files. Therefore, we will further use device files to control the LED on and off.

In the implementation, a pull-down resistor is used. The anode of the LED is connected to GPIO4, and the cathode is connected to GND. For more details about the Raspberry Pi pin assignments, refer to Raspberry Pi Pinout. The materials used include a Raspberry Pi 5, an LED, jumper wires, and a 220Ω resistor.

```c
/*
 * led.c - Using GPIO to control the LED on/off
 */

#include <linux/cdev.h>
#include <linux/delay.h>
#include <linux/device.h>
#include <linux/fs.h>
#include <linux/gpio.h>
#include <linux/init.h>
#include <linux/module.h>
#include <linux/printk.h>
#include <linux/types.h>
#include <linux/uaccess.h>
#include <linux/version.h>

#include <asm/errno.h>

#define DEVICE_NAME "gpio_led"
#define DEVICE_CNT 1
#define BUF_LEN 2

static char control_signal[BUF_LEN];
static unsigned long device_buffer_size = 0;

struct LED_dev {
    dev_t dev_num;
    int major_num, minor_num;
    struct cdev cdev;
    struct class *cls;
    struct device *dev;
};

static struct LED_dev led_device;

/* Define GPIOs for LEDs.
 * TODO: According to the requirements, search /sys/kernel/debug/gpio to
 * find the corresponding GPIO location.
 */
#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 10, 0)
static unsigned int led_gpio = 4;
static unsigned int led_flags = GPIOF_OUT_INIT_LOW;
```

```
43  static const char *led_label = "LED 1";
44  #else
45  static struct gpio leds[] = { { 4, GPIOF_OUT_INIT_LOW, "LED 1" } };
46  #endif
47
48  /* This is called whenever a process attempts to open the device file */
49  static int device_open(struct inode *inode, struct file *file)
50  {
51      return 0;
52  }
53
54  static int device_release(struct inode *inode, struct file *file)
55  {
56      return 0;
57  }
58
59  static ssize_t device_write(struct file *file, const char __user *buffer,
60                              size_t length, loff_t *offset)
61  {
62      device_buffer_size = min(BUF_LEN, length);
63
64      if (copy_from_user(control_signal, buffer, device_buffer_size)) {
65          return -EFAULT;
66      }
67
68      /* Determine the received signal to decide the LED on/off state. */
69      switch (control_signal[0]) {
70      case '0':
71  #if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 10, 0)
72          gpio_set_value(led_gpio, 0);
73  #else
74          gpio_set_value(leds[0].gpio, 0);
75  #endif
76          pr_info("LED OFF");
77          break;
78      case '1':
79  #if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 10, 0)
80          gpio_set_value(led_gpio, 1);
81  #else
82          gpio_set_value(leds[0].gpio, 1);
83  #endif
84          pr_info("LED ON");
85          break;
86      default:
87          pr_warn("Invalid value!\n");
88          break;
89      }
90
91      *offset += device_buffer_size;
92
93      /* Again, return the number of input characters used. */
94      return device_buffer_size;
95  }
96
97  static struct file_operations fops = {
98  #if LINUX_VERSION_CODE < KERNEL_VERSION(6, 4, 0)
99      .owner = THIS_MODULE,
```

```
#endif
    .write = device_write,
    .open = device_open,
    .release = device_release,
};

/* Initialize the module - Register the character device */
static int __init led_init(void)
{
    int ret = 0;

    /* Determine whether dynamic allocation of the device number is needed. */
    if (led_device.major_num) {
        led_device.dev_num = MKDEV(led_device.major_num,
        ↪  led_device.minor_num);
        ret =
            register_chrdev_region(led_device.dev_num, DEVICE_CNT,
            ↪  DEVICE_NAME);
    } else {
        ret = alloc_chrdev_region(&led_device.dev_num, 0, DEVICE_CNT,
                                  DEVICE_NAME);
    }

    /* Negative values signify an error */
    if (ret < 0) {
        pr_alert("Failed to register character device, error: %d\n", ret);
        return ret;
    }

    pr_info("Major = %d, Minor = %d\n", MAJOR(led_device.dev_num),
            MINOR(led_device.dev_num));

    /* Prevents module unloading while operations are in use */
#if LINUX_VERSION_CODE < KERNEL_VERSION(6, 4, 0)
    led_device.cdev.owner = THIS_MODULE;
#endif

    cdev_init(&led_device.cdev, &fops);
    ret = cdev_add(&led_device.cdev, led_device.dev_num, 1);
    if (ret) {
        pr_err("Failed to add the device to the system\n");
        goto fail1;
    }

#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 4, 0)
    led_device.cls = class_create(DEVICE_NAME);
#else
    led_device.cls = class_create(THIS_MODULE, DEVICE_NAME);
#endif
    if (IS_ERR(led_device.cls)) {
        pr_err("Failed to create class for device\n");
        ret = PTR_ERR(led_device.cls);
        goto fail2;
    }

    led_device.dev = device_create(led_device.cls, NULL, led_device.dev_num,
                                   NULL, DEVICE_NAME);
```

```
155     if (IS_ERR(led_device.dev)) {
156         pr_err("Failed to create the device file\n");
157         ret = PTR_ERR(led_device.dev);
158         goto fail3;
159     }
160 
161     pr_info("Device created on /dev/%s\n", DEVICE_NAME);
162 
163 #if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 10, 0)
164     ret = gpio_request(led_gpio, led_label);
165 #else
166     ret = gpio_request(leds[0].gpio, leds[0].label);
167 #endif
168 
169     if (ret) {
170         pr_err("Unable to request GPIOs for LEDs: %d\n", ret);
171         goto fail4;
172     }
173 
174 #if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 10, 0)
175     ret = gpio_direction_output(led_gpio, led_flags);
176 #else
177     ret = gpio_direction_output(leds[0].gpio, leds[0].flags);
178 #endif
179 
180     if (ret) {
181 #if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 10, 0)
182         pr_err("Failed to set GPIO %d direction\n", led_gpio);
183 #else
184         pr_err("Failed to set GPIO %d direction\n", leds[0].gpio);
185 #endif
186         goto fail5;
187     }
188 
189     return 0;
190 
191 fail5:
192 #if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 10, 0)
193     gpio_free(led_gpio);
194 #else
195     gpio_free(leds[0].gpio);
196 #endif
197 
198 fail4:
199     device_destroy(led_device.cls, led_device.dev_num);
200 
201 fail3:
202     class_destroy(led_device.cls);
203 
204 fail2:
205     cdev_del(&led_device.cdev);
206 
207 fail1:
208     unregister_chrdev_region(led_device.dev_num, DEVICE_CNT);
209 
210     return ret;
211 }
```

```
212
213 static void __exit led_exit(void)
214 {
215 #if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 10, 0)
216     gpio_set_value(led_gpio, 0);
217     gpio_free(led_gpio);
218 #else
219     gpio_set_value(leds[0].gpio, 0);
220     gpio_free(leds[0].gpio);
221 #endif
222
223     device_destroy(led_device.cls, led_device.dev_num);
224     class_destroy(led_device.cls);
225     cdev_del(&led_device.cdev);
226     unregister_chrdev_region(led_device.dev_num, DEVICE_CNT);
227 }
228
229 module_init(led_init);
230 module_exit(led_exit);
231
232 MODULE_LICENSE("GPL");
```

Make and install the module:

```
1 make
2 sudo insmod led.ko
```

Switch on the LED:

```
1 echo "1" | sudo tee /dev/gpio_led
```

Switch off the LED:

```
1 echo "0" | sudo tee /dev/gpio_led
```

Finally, remove the module:

```
1 sudo rmmod led
```

21.3 DHT11 sensor

The DHT11 sensor is a well-known entry-level sensor commonly used to measure humidity and temperature. In this subsection, we will use GPIO to communicate through a single data line. The DHT11 communication protocol can be referred to in the datasheet.

In the implementation, the data pin of the DHT11 sensor is connected to GPIO4 on the Raspberry Pi. The sensor's VCC and GND pins are connected

to 3.3V and GND, respectively. For more details about the Raspberry Pi pin assignments, refer to Raspberry Pi Pinout. The materials used include a Raspberry Pi 5, a DHT11 sensor, and jumper wires.

```c
/*
 *  dht11.c - Using GPIO to read temperature and humidity from DHT11 sensor.
 */

#include <linux/cdev.h>
#include <linux/delay.h>
#include <linux/device.h>
#include <linux/fs.h>
#include <linux/gpio.h>
#include <linux/init.h>
#include <linux/module.h>
#include <linux/printk.h>
#include <linux/types.h>
#include <linux/uaccess.h>
#include <linux/version.h>

#include <asm/errno.h>

#define GPIO_PIN_4 575
#define DEVICE_NAME "dht11"
#define DEVICE_CNT 1

static char msg[64];

struct dht11_dev {
    dev_t dev_num;
    int major_num, minor_num;
    struct cdev cdev;
    struct class *cls;
    struct device *dev;
};

static struct dht11_dev dht11_device;

/* Define GPIOs for LEDs.
 * TODO: According to the requirements, search /sys/kernel/debug/gpio to
 * find the corresponding GPIO location.
 */
#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 10, 0)
static unsigned int dht11_gpio_pin = GPIO_PIN_4;
static const char *dht11_gpio_label = "Signal";
#else
static struct gpio dht11[] = { { GPIO_PIN_4, GPIOF_OUT_INIT_HIGH, "Signal" }
↪  };
#define dht11_gpio_pin (dht11[0].gpio)
#define dht11_gpio_label (dht11[0].label)
#endif

static int dht11_read_data(void)
{
    int timeout;
    uint8_t sensor_data[5] = { 0 };
    uint8_t i, j;
```

```
53
54      gpio_set_value(dht11_gpio_pin, 0);
55      mdelay(20);
56      gpio_set_value(dht11_gpio_pin, 1);
57      udelay(30);
58      gpio_direction_input(dht11_gpio_pin);
59      udelay(2);
60
61      timeout = 300;
62      while (gpio_get_value(dht11_gpio_pin) && timeout--)
63          udelay(1);
64
65      if (timeout == -1)
66          return -ETIMEDOUT;
67
68      timeout = 300;
69      while (!gpio_get_value(dht11_gpio_pin) && timeout--)
70          udelay(1);
71
72      if (timeout == -1)
73          return -ETIMEDOUT;
74
75      timeout = 300;
76      while (gpio_get_value(dht11_gpio_pin) && timeout--)
77          udelay(1);
78
79      if (timeout == -1)
80          return -ETIMEDOUT;
81
82      for (j = 0; j < 5; j++) {
83          uint8_t byte = 0;
84          for (i = 0; i < 8; i++) {
85              timeout = 300;
86              while (gpio_get_value(dht11_gpio_pin) && timeout--)
87                  udelay(1);
88
89              if (timeout == -1)
90                  return -ETIMEDOUT;
91
92              timeout = 300;
93              while (!gpio_get_value(dht11_gpio_pin) && timeout--)
94                  udelay(1);
95
96              if (timeout == -1)
97                  return -ETIMEDOUT;
98
99              udelay(50);
100             byte <<= 1;
101             if (gpio_get_value(dht11_gpio_pin))
102                 byte |= 0x01;
103         }
104         sensor_data[j] = byte;
105     }
106
107     if (sensor_data[4] != (uint8_t)(sensor_data[0] + sensor_data[1] +
108                                     sensor_data[2] + sensor_data[3]))
109         return -EIO;
```

```
110
111     gpio_direction_output(dht11_gpio_pin, 1);
112     sprintf(msg, "Humidity: %d%%\nTemperature: %d deg C\n", sensor_data[0],
113             sensor_data[2]);
114
115     return 0;
116 }
117
118 static int device_open(struct inode *inode, struct file *file)
119 {
120     int ret, retry;
121
122     for (retry = 0; retry < 5; ++retry) {
123         ret = dht11_read_data();
124         if (ret == 0)
125             return 0;
126         msleep(10);
127     }
128     gpio_direction_output(dht11_gpio_pin, 1);
129
130     return ret;
131 }
132
133 static int device_release(struct inode *inode, struct file *file)
134 {
135     return 0;
136 }
137
138 static ssize_t device_read(struct file *filp, char __user *buffer,
139                            size_t length, loff_t *offset)
140 {
141     int msg_len = strlen(msg);
142
143     if (*offset >= msg_len)
144         return 0;
145
146     size_t remain = msg_len - *offset;
147     size_t bytes_read = min(length, remain);
148
149     if (copy_to_user(buffer, msg + *offset, bytes_read))
150         return -EFAULT;
151
152     *offset += bytes_read;
153
154     return bytes_read;
155 }
156
157 static struct file_operations fops = {
158     .owner = THIS_MODULE,
159     .open = device_open,
160     .release = device_release,
161     .read = device_read,
162 };
163
164 /* Initialize the module - Register the character device */
165 static int __init dht11_init(void)
166 {
```

```
167     int ret = 0;
168
169     /* Determine whether dynamic allocation of the device number is needed. */
170     if (dht11_device.major_num) {
171         dht11_device.dev_num =
172             MKDEV(dht11_device.major_num, dht11_device.minor_num);
173         ret = register_chrdev_region(dht11_device.dev_num, DEVICE_CNT,
174                                      DEVICE_NAME);
175     } else {
176         ret = alloc_chrdev_region(&dht11_device.dev_num, 0, DEVICE_CNT,
177                                   DEVICE_NAME);
178     }
179
180     /* Negative values signify an error */
181     if (ret < 0) {
182         pr_alert("Failed to register character device, error: %d\n", ret);
183         return ret;
184     }
185
186     pr_info("Major = %d, Minor = %d\n", MAJOR(dht11_device.dev_num),
187             MINOR(dht11_device.dev_num));
188
189     /* Prevents module unloading while operations are in use */
190     dht11_device.cdev.owner = THIS_MODULE;
191
192     cdev_init(&dht11_device.cdev, &fops);
193     ret = cdev_add(&dht11_device.cdev, dht11_device.dev_num, 1);
194     if (ret) {
195         pr_err("Failed to add the device to the system\n");
196         goto fail1;
197     }
198
199 #if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 4, 0)
200     dht11_device.cls = class_create(DEVICE_NAME);
201 #else
202     dht11_device.cls = class_create(THIS_MODULE, DEVICE_NAME);
203 #endif
204     if (IS_ERR(dht11_device.cls)) {
205         pr_err("Failed to create class for device\n");
206         ret = PTR_ERR(dht11_device.cls);
207         goto fail2;
208     }
209
210     dht11_device.dev = device_create(dht11_device.cls, NULL,
211                                      dht11_device.dev_num, NULL, DEVICE_NAME);
212     if (IS_ERR(dht11_device.dev)) {
213         pr_err("Failed to create the device file\n");
214         ret = PTR_ERR(dht11_device.dev);
215         goto fail3;
216     }
217
218     pr_info("Device created on /dev/%s\n", DEVICE_NAME);
219
220     ret = gpio_request(dht11_gpio_pin, dht11_gpio_label);
221
222     if (ret) {
223         pr_err("Unable to request GPIOs for dht11: %d\n", ret);
```

```
        goto fail4;
    }

    ret = gpio_direction_output(dht11_gpio_pin, 1);

    if (ret) {
        pr_err("Failed to set GPIO %d direction\n", dht11_gpio_pin);
        goto fail5;
    }

    return 0;

fail5:
    gpio_free(dht11_gpio_pin);

fail4:
    device_destroy(dht11_device.cls, dht11_device.dev_num);

fail3:
    class_destroy(dht11_device.cls);

fail2:
    cdev_del(&dht11_device.cdev);

fail1:
    unregister_chrdev_region(dht11_device.dev_num, DEVICE_CNT);

    return ret;
}

static void __exit dht11_exit(void)
{
    gpio_set_value(dht11_gpio_pin, 0);
    gpio_free(dht11_gpio_pin);

    device_destroy(dht11_device.cls, dht11_device.dev_num);
    class_destroy(dht11_device.cls);
    cdev_del(&dht11_device.cdev);
    unregister_chrdev_region(dht11_device.dev_num, DEVICE_CNT);
}

module_init(dht11_init);
module_exit(dht11_exit);

MODULE_LICENSE("GPL");
```

Make and install the module:

```
make
sudo insmod dht11.ko
```

Check the Output of the DHT11 Sensor:

```
sudo cat /dev/dht11
```

Expected Output:

```
$ sudo cat /dev/dht11
Humidity: 61%
Temperature: 30°C
```

Finally, remove the module:

```
sudo rmmod dht11
```

22 Scheduling Tasks

There are two main ways of running tasks: tasklets and work queues. Tasklets are a quick and easy way of scheduling a single function to be run. For example, when triggered from an interrupt, whereas work queues are more complicated but also better suited to running multiple things in a sequence.

It is possible that in future tasklets may be replaced by *threaded IRQs*. However, discussion about that has been ongoing since 2007 (Eliminating tasklets and The end of tasklets), so expecting immediate changes would be unwise. See the Section 23.1 for alternatives that avoid the tasklet debate.

22.1 Tasklets

Here is an example tasklet module. The `tasklet_fn` function runs for a few seconds. In the meantime, execution of the `example_tasklet_init` function may continue to the exit point, depending on whether it is interrupted by **softirq**.

```c
/*
 * example_tasklet.c
 */
#include <linux/delay.h>
#include <linux/interrupt.h>
#include <linux/module.h>
#include <linux/printk.h>

/* Macro DECLARE_TASKLET_OLD exists for compatibility.
 * See https://lwn.net/Articles/830964/
 */
#ifndef DECLARE_TASKLET_OLD
#define DECLARE_TASKLET_OLD(arg1, arg2) DECLARE_TASKLET(arg1, arg2, 0L)
#endif

static void tasklet_fn(unsigned long data)
{
    pr_info("Example tasklet starts\n");
    mdelay(5000);
    pr_info("Example tasklet ends\n");
}

static DECLARE_TASKLET_OLD(mytask, tasklet_fn);
```

```
24
25  static int __init example_tasklet_init(void)
26  {
27      pr_info("tasklet example init\n");
28      tasklet_schedule(&mytask);
29      mdelay(200);
30      pr_info("Example tasklet init continues...\n");
31      return 0;
32  }
33
34  static void __exit example_tasklet_exit(void)
35  {
36      pr_info("tasklet example exit\n");
37      tasklet_kill(&mytask);
38  }
39
40  module_init(example_tasklet_init);
41  module_exit(example_tasklet_exit);
42
43  MODULE_DESCRIPTION("Tasklet example");
44  MODULE_LICENSE("GPL");
```

So with this example loaded `dmesg` should show:

```
tasklet example init
Example tasklet starts
Example tasklet init continues...
Example tasklet ends
```

Although tasklet is easy to use, it comes with several drawbacks, and developers have been discussing their removal from the Linux kernel. The tasklet callback runs in atomic context, inside a software interrupt, meaning that it cannot sleep or access user-space data, so not all work can be done in a tasklet handler. Also, the kernel only allows one instance of any given tasklet to be running at any given time; multiple different tasklet callbacks can run in parallel.

In recent kernels, tasklets can be replaced by workqueues, timers, or threaded interrupts. [1] While the removal of tasklets remains a longer-term goal, the current kernel contains more than a hundred uses of tasklets. Now developers are proceeding with the API changes and the macro `DECLARE_TASKLET_OLD` exists for compatibility. For further information, see https://lwn.net/Articles/830964/.

22.2 Work queues

To add a task to the scheduler we can use a workqueue. The kernel then uses the Completely Fair Scheduler (CFS) to execute work within the queue.

[1] The goal of threaded interrupts is to push more of the work to separate threads, so that the minimum needed for acknowledging an interrupt is reduced, and therefore the time spent handling the interrupt (where it can't handle any other interrupts at the same time) is reduced. See https://lwn.net/Articles/302043/.

```c
/*
 * sched.c
 */
#include <linux/init.h>
#include <linux/module.h>
#include <linux/workqueue.h>

static struct workqueue_struct *queue = NULL;
static struct work_struct work;

static void work_handler(struct work_struct *data)
{
    pr_info("work handler function.\n");
}

static int __init sched_init(void)
{
    queue = alloc_workqueue("HELLOWORLD", WQ_UNBOUND, 1);
    if (!queue) {
        pr_err("Failed to allocate workqueue\n");
        return -ENOMEM;
    }
    INIT_WORK(&work, work_handler);
    queue_work(queue, &work);
    return 0;
}

static void __exit sched_exit(void)
{
    flush_workqueue(queue);
    destroy_workqueue(queue);
}

module_init(sched_init);
module_exit(sched_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Workqueue example");
```

23 Interrupt Handlers

23.1 Interrupt Handlers

Except for the last chapter, everything we did in the kernel so far we have done as a response to a process asking for it, either by dealing with a special file, sending an `ioctl()`, or issuing a system call. But the job of the kernel is not just to respond to process requests. Another job, which is every bit as important, is to speak to the hardware connected to the machine.

There are two types of interaction between the CPU and the rest of the computer's hardware. The first type is when the CPU gives orders to the hardware, the other is when the hardware needs to tell the CPU something. The second,

called interrupts, is much harder to implement because it has to be dealt with when convenient for the hardware, not the CPU. Hardware devices typically have a very small amount of RAM, and if you do not read their information when available, it is lost.

Under Linux, hardware interrupts are called IRQs (Interrupt ReQuests). There are two types of IRQs, short and long. A short IRQ is one which is expected to take a very short period of time, during which the rest of the machine will be blocked and no other interrupts will be handled. A long IRQ is one which can take longer, and during which other interrupts may occur (but not interrupts from the same device). If at all possible, it is better to declare an interrupt handler to be long.

When the CPU receives an interrupt, it stops whatever it is doing (unless it is processing a more important interrupt, in which case it will deal with this one only when the more important one is done), saves certain parameters on the stack and calls the interrupt handler. This means that certain things are not allowed in the interrupt handler itself, because the system is in an unknown state. Linux kernel solves the problem by splitting interrupt handling into two parts. The first part executes right away and masks the interrupt line. Hardware interrupts must be handled quickly, and that is why we need the second part to handle the heavy work deferred from an interrupt handler. Historically, BH (Linux naming for *Bottom Halves*) statistically book-keeps the deferred functions. **Softirq** and its higher level abstraction, **Tasklet**, replace BH since Linux 2.3.

The way to implement this is to call `request_irq()` to get your interrupt handler called when the relevant IRQ is received.

In practice IRQ handling can be a bit more complex. Hardware is often designed in a way that chains two interrupt controllers, so that all the IRQs from interrupt controller B are cascaded to a certain IRQ from interrupt controller A. Of course, that requires that the kernel finds out which IRQ it really was afterwards and that adds overhead. Other architectures offer some special, very low overhead, so called "fast IRQ" or FIQs. To take advantage of them requires handlers to be written in assembly language, so they do not really fit into the kernel. They can be made to work similar to the others, but after that procedure, they are no longer any faster than "common" IRQs. SMP enabled kernels running on systems with more than one processor need to solve another truckload of problems. It is not enough to know if a certain IRQ has happened, it's also important to know what CPU(s) it was for. People still interested in more details, might want to refer to "APIC" now.

This function receives the IRQ number, the name of the function, flags, a name for `/proc/interrupts` and a parameter to be passed to the interrupt handler. Usually there is a certain number of IRQs available. How many IRQs there are is hardware-dependent.

The flags can be used to specify behaviors of the IRQ. For example, use `IRQF_SHARED` to indicate you are willing to share the IRQ with other interrupt handlers (usually because a number of hardware devices sit on the same IRQ); use the `IRQF_ONESHOT` to indicate that the IRQ is not reenabled after the handler

finished. It should be noted that in some materials, you may encounter another set of IRQ flags named with the `SA` prefix. For example, the `SA_SHIRQ` and the `SA_INTERRUPT`. Those are the IRQ flags in the older kernels. They have been removed completely. Today only the `IRQF` flags are in use. This function will only succeed if there is not already a handler on this IRQ, or if you are both willing to share.

23.2 Detecting button presses

Many popular single board computers, such as Raspberry Pi or Beagleboards, have a bunch of GPIO pins. Attaching buttons to those and then having a button press do something is a classic case in which you might need to use interrupts, so that instead of having the CPU waste time and battery power polling for a change in input state, it is better for the input to trigger the CPU to then run a particular handling function.

Here is an example where buttons are connected to GPIO numbers 17 and 18 and an LED is connected to GPIO 4. You can change those numbers to whatever is appropriate for your board.

```
1  /*
2   * intrpt.c - Handling GPIO with interrupts
3   *
4   * Based upon the RPi example by Stefan Wendler (devnull@kaltpost.de)
5   * from:
6   *   https://github.com/wendlers/rpi-kmod-samples
7   *
8   * Press one button to turn on a LED and another to turn it off.
9   */
10
11 #include <linux/gpio.h>
12 #include <linux/interrupt.h>
13 #include <linux/kernel.h> /* for ARRAY_SIZE() */
14 #include <linux/module.h>
15 #include <linux/printk.h>
16 #include <linux/version.h>
17
18 #if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 10, 0)
19 #define NO_GPIO_REQUEST_ARRAY
20 #endif
21
22 static int button_irqs[] = { -1, -1 };
23
24 /* Define GPIOs for LEDs.
25  * TODO: Change the numbers for the GPIO on your board.
26  */
27 #ifdef NO_GPIO_REQUEST_ARRAY
28 static unsigned int led_gpio = 4;
29 static unsigned int button_gpios[] = { 17, 18 };
30 #else
31 static struct gpio leds[] = { { 4, GPIOF_OUT_INIT_LOW, "LED 1" } };
32
33 /* Define GPIOs for BUTTONS
34  * TODO: Change the numbers for the GPIO on your board.
```

```
 */
static struct gpio buttons[] = { { 17, GPIOF_IN, "LED 1 ON BUTTON" },
                                 { 18, GPIOF_IN, "LED 1 OFF BUTTON" } };
#endif

/* interrupt function triggered when a button is pressed. */
static irqreturn_t button_isr(int irq, void *data)
{
#ifdef NO_GPIO_REQUEST_ARRAY
    if (irq == button_irqs[0] && !gpio_get_value(led_gpio))
        gpio_set_value(led_gpio, 1);
    else if (irq == button_irqs[1] && gpio_get_value(led_gpio))
        gpio_set_value(led_gpio, 0);
#else
    /* first button */
    if (irq == button_irqs[0] && !gpio_get_value(leds[0].gpio))
        gpio_set_value(leds[0].gpio, 1);
    /* second button */
    else if (irq == button_irqs[1] && gpio_get_value(leds[0].gpio))
        gpio_set_value(leds[0].gpio, 0);
#endif

    return IRQ_HANDLED;
}

static int __init intrpt_init(void)
{
    int ret = 0;

    pr_info("%s\n", __func__);

    /* register LED gpios */
#ifdef NO_GPIO_REQUEST_ARRAY
    ret = gpio_request(led_gpio, "LED 1");
#else
    ret = gpio_request_array(leds, ARRAY_SIZE(leds));
#endif

    if (ret) {
        pr_err("Unable to request GPIOs for LEDs: %d\n", ret);
        return ret;
    }

    /* register BUTTON gpios */
#ifdef NO_GPIO_REQUEST_ARRAY
    ret = gpio_request(button_gpios[0], "LED 1 ON BUTTON");

    if (ret) {
        pr_err("Unable to request GPIOs for BUTTONs: %d\n", ret);
        goto fail1;
    }

    ret = gpio_request(button_gpios[1], "LED 1 OFF BUTTON");

    if (ret) {
        pr_err("Unable to request GPIOs for BUTTONs: %d\n", ret);
        goto fail2;
```

```
92      }
93  #else
94      ret = gpio_request_array(buttons, ARRAY_SIZE(buttons));
95
96      if (ret) {
97          pr_err("Unable to request GPIOs for BUTTONs: %d\n", ret);
98          goto fail1;
99      }
100 #endif
101
102 #ifdef NO_GPIO_REQUEST_ARRAY
103     pr_info("Current button1 value: %d\n", gpio_get_value(button_gpios[0]));
104
105     ret = gpio_to_irq(button_gpios[0]);
106 #else
107     pr_info("Current button1 value: %d\n", gpio_get_value(buttons[0].gpio));
108
109     ret = gpio_to_irq(buttons[0].gpio);
110 #endif
111
112     if (ret < 0) {
113         pr_err("Unable to request IRQ: %d\n", ret);
114 #ifdef NO_GPIO_REQUEST_ARRAY
115         goto fail3;
116 #else
117         goto fail2;
118 #endif
119     }
120
121     button_irqs[0] = ret;
122
123     pr_info("Successfully requested BUTTON1 IRQ # %d\n", button_irqs[0]);
124
125     ret = request_irq(button_irqs[0], button_isr,
126                       IRQF_TRIGGER_RISING | IRQF_TRIGGER_FALLING,
127                       "gpiomod#button1", NULL);
128
129     if (ret) {
130         pr_err("Unable to request IRQ: %d\n", ret);
131 #ifdef NO_GPIO_REQUEST_ARRAY
132         goto fail3;
133 #else
134         goto fail2;
135 #endif
136     }
137
138 #ifdef NO_GPIO_REQUEST_ARRAY
139     ret = gpio_to_irq(button_gpios[1]);
140 #else
141     ret = gpio_to_irq(buttons[1].gpio);
142 #endif
143
144     if (ret < 0) {
145         pr_err("Unable to request IRQ: %d\n", ret);
146 #ifdef NO_GPIO_REQUEST_ARRAY
147         goto fail4;
148 #else
```

```
149         goto fail2;
150 #endif
151     }
152
153     button_irqs[1] = ret;
154
155     pr_info("Successfully requested BUTTON2 IRQ # %d\n", button_irqs[1]);
156
157     ret = request_irq(button_irqs[1], button_isr,
158                       IRQF_TRIGGER_RISING | IRQF_TRIGGER_FALLING,
159                       "gpiomod#button2", NULL);
160
161     if (ret) {
162         pr_err("Unable to request IRQ: %d\n", ret);
163 #ifdef NO_GPIO_REQUEST_ARRAY
164         goto fail4;
165 #else
166         goto fail3;
167 #endif
168     }
169
170     return 0;
171
172 /* cleanup what has been setup so far */
173 #ifdef NO_GPIO_REQUEST_ARRAY
174 fail4:
175     free_irq(button_irqs[0], NULL);
176
177 fail3:
178     gpio_free(button_gpios[1]);
179
180 fail2:
181     gpio_free(button_gpios[0]);
182
183 fail1:
184     gpio_free(led_gpio);
185 #else
186 fail3:
187     free_irq(button_irqs[0], NULL);
188
189 fail2:
190     gpio_free_array(buttons, ARRAY_SIZE(buttons));
191
192 fail1:
193     gpio_free_array(leds, ARRAY_SIZE(leds));
194 #endif
195
196     return ret;
197 }
198
199 static void __exit intrpt_exit(void)
200 {
201     pr_info("%s\n", __func__);
202
203     /* free irqs */
204     free_irq(button_irqs[0], NULL);
205     free_irq(button_irqs[1], NULL);
```

```
206
207     /* turn all LEDs off */
208 #ifdef NO_GPIO_REQUEST_ARRAY
209     gpio_set_value(led_gpio, 0);
210 #else
211     int i;
212     for (i = 0; i < ARRAY_SIZE(leds); i++)
213         gpio_set_value(leds[i].gpio, 0);
214 #endif
215
216     /* unregister */
217 #ifdef NO_GPIO_REQUEST_ARRAY
218     gpio_free(led_gpio);
219     gpio_free(button_gpios[0]);
220     gpio_free(button_gpios[1]);
221 #else
222     gpio_free_array(leds, ARRAY_SIZE(leds));
223     gpio_free_array(buttons, ARRAY_SIZE(buttons));
224 #endif
225 }
226
227 module_init(intrpt_init);
228 module_exit(intrpt_exit);
229
230 MODULE_LICENSE("GPL");
231 MODULE_DESCRIPTION("Handle some GPIO interrupts");
```

23.3 Bottom Half

Suppose you want to do a bunch of stuff inside of an interrupt routine. A common way to avoid blocking the interrupt for a significant duration is to defer the time-consuming part to a workqueue. This pushes the bulk of the work off into the scheduler. This approach helps speed up the interrupt handling process itself, allowing the system to respond to the next hardware interrupt more quickly.

Kernel developers generally discourage using tasklets due to their design limitations, such as memory management issues and unpredictable latencies. Instead, they recommend more robust mechanisms like workqueues or **softirqs**. To address tasklet shortcomings, Linux contributors introduced the BH workqueue, activated with the `WQ_BH` flag. This workqueue retains critical features, such as execution in atomic (**softirq**) context on the same CPU and the inability to sleep.

The example below extends the previous code to include an additional task executed in process context when an interrupt is triggered.

```
1 /*
2  * bottomhalf.c - Top and bottom half interrupt handling
3  *
4  * Based upon the RPi example by Stefan Wendler (devnull@kaltpost.de)
5  * from:
6  *    https://github.com/wendlers/rpi-kmod-samples
7  *
```

```
 * Press one button to turn on an LED and another to turn it off
 */

#include <linux/delay.h>
#include <linux/gpio.h>
#include <linux/interrupt.h>
#include <linux/module.h>
#include <linux/printk.h>
#include <linux/init.h>
#include <linux/version.h>
#include <linux/workqueue.h>

#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 10, 0)
#define NO_GPIO_REQUEST_ARRAY
#endif

static int button_irqs[] = { -1, -1 };

/* Define GPIOs for LEDs.
 * TODO: Change the numbers for the GPIO on your board.
 */
#ifdef NO_GPIO_REQUEST_ARRAY
static unsigned int led_gpio = 4;
static unsigned int button_gpios[] = { 17, 18 };
#else
static struct gpio leds[] = { { 4, GPIOF_OUT_INIT_LOW, "LED 1" } };

/* Define GPIOs for BUTTONS
 * TODO: Change the numbers for the GPIO on your board.
 */
static struct gpio buttons[] = {
    { 17, GPIOF_IN, "LED 1 ON BUTTON" },
    { 18, GPIOF_IN, "LED 1 OFF BUTTON" },
};
#endif

/* Workqueue function containing some non-trivial amount of processing */
static void bottomhalf_work_fn(struct work_struct *work)
{
    pr_info("Bottom half workqueue starts\n");
    /* do something which takes a while */
    msleep(500);

    pr_info("Bottom half workqueue ends\n");
}

static DECLARE_WORK(bottomhalf_work, bottomhalf_work_fn);

/* interrupt function triggered when a button is pressed */
static irqreturn_t button_isr(int irq, void *data)
{
    /* Do something quickly right now */
#ifdef NO_GPIO_REQUEST_ARRAY
    if (irq == button_irqs[0] && !gpio_get_value(led_gpio))
        gpio_set_value(led_gpio, 1);
    else if (irq == button_irqs[1] && gpio_get_value(led_gpio))
        gpio_set_value(led_gpio, 0);
```

```
65  #else
66      if (irq == button_irqs[0] && !gpio_get_value(leds[0].gpio))
67          gpio_set_value(leds[0].gpio, 1);
68      else if (irq == button_irqs[1] && gpio_get_value(leds[0].gpio))
69          gpio_set_value(leds[0].gpio, 0);
70  #endif
71
72      /* Do the rest at leisure via the scheduler */
73      schedule_work(&bottomhalf_work);
74      return IRQ_HANDLED;
75  }
76
77  static int __init bottomhalf_init(void)
78  {
79      int ret = 0;
80
81      pr_info("%s\n", __func__);
82
83      /* register LED gpios */
84  #ifdef NO_GPIO_REQUEST_ARRAY
85      ret = gpio_request(led_gpio, "LED 1");
86  #else
87      ret = gpio_request_array(leds, ARRAY_SIZE(leds));
88  #endif
89
90      if (ret) {
91          pr_err("Unable to request GPIOs for LEDs: %d\n", ret);
92          return ret;
93      }
94
95  #ifdef NO_GPIO_REQUEST_ARRAY
96      ret = gpio_direction_output(led_gpio, 0);
97
98      if (ret) {
99          pr_err("Unable to configure LED GPIO direction: %d\n", ret);
100         goto fail2;
101     }
102 #endif
103
104     /* register BUTTON gpios */
105 #ifdef NO_GPIO_REQUEST_ARRAY
106     ret = gpio_request(button_gpios[0], "LED 1 ON BUTTON");
107
108     if (ret) {
109         pr_err("Unable to request GPIOs for BUTTONs: %d\n", ret);
110         goto fail2;
111     }
112
113     ret = gpio_request(button_gpios[1], "LED 1 OFF BUTTON");
114
115     if (ret) {
116         pr_err("Unable to request GPIOs for BUTTONs: %d\n", ret);
117         goto fail3;
118     }
119
120     ret = gpio_direction_input(button_gpios[0]);
121
```

```
122     if (ret) {
123         pr_err("Unable to configure BUTTON1 GPIO direction: %d\n", ret);
124         goto fail4;
125     }
126
127     ret = gpio_direction_input(button_gpios[1]);
128
129     if (ret) {
130         pr_err("Unable to configure BUTTON2 GPIO direction: %d\n", ret);
131         goto fail4;
132     }
133 #else
134     ret = gpio_request_array(buttons, ARRAY_SIZE(buttons));
135
136     if (ret) {
137         pr_err("Unable to request GPIOs for BUTTONs: %d\n", ret);
138         goto fail1;
139     }
140 #endif
141
142 #ifdef NO_GPIO_REQUEST_ARRAY
143     pr_info("Current button1 value: %d\n", gpio_get_value(button_gpios[0]));
144
145     ret = gpio_to_irq(button_gpios[0]);
146 #else
147     pr_info("Current button1 value: %d\n", gpio_get_value(buttons[0].gpio));
148
149     ret = gpio_to_irq(buttons[0].gpio);
150 #endif
151
152     if (ret < 0) {
153         pr_err("Unable to request IRQ: %d\n", ret);
154 #ifdef NO_GPIO_REQUEST_ARRAY
155         goto fail4;
156 #else
157         goto fail2;
158 #endif
159     }
160
161     button_irqs[0] = ret;
162
163     pr_info("Successfully requested BUTTON1 IRQ # %d\n", button_irqs[0]);
164
165     ret = request_irq(button_irqs[0], button_isr,
166                       IRQF_TRIGGER_RISING | IRQF_TRIGGER_FALLING,
167                       "gpiomod#button1", NULL);
168
169     if (ret) {
170         pr_err("Unable to request IRQ: %d\n", ret);
171 #ifdef NO_GPIO_REQUEST_ARRAY
172         goto fail4;
173 #else
174         goto fail2;
175 #endif
176     }
177
178 #ifdef NO_GPIO_REQUEST_ARRAY
```

```
179     ret = gpio_to_irq(button_gpios[1]);
180 #else
181     ret = gpio_to_irq(buttons[1].gpio);
182 #endif
183 
184     if (ret < 0) {
185         pr_err("Unable to request IRQ: %d\n", ret);
186 #ifdef NO_GPIO_REQUEST_ARRAY
187         goto fail5;
188 #else
189         goto fail2;
190 #endif
191     }
192 
193     button_irqs[1] = ret;
194 
195     pr_info("Successfully requested BUTTON2 IRQ # %d\n", button_irqs[1]);
196 
197     ret = request_irq(button_irqs[1], button_isr,
198                       IRQF_TRIGGER_RISING | IRQF_TRIGGER_FALLING,
199                       "gpiomod#button2", NULL);
200 
201     if (ret) {
202         pr_err("Unable to request IRQ: %d\n", ret);
203 #ifdef NO_GPIO_REQUEST_ARRAY
204         goto fail5;
205 #else
206         goto fail3;
207 #endif
208     }
209 
210     return 0;
211 
212 /* cleanup what has been setup so far */
213 #ifdef NO_GPIO_REQUEST_ARRAY
214 fail5:
215     free_irq(button_irqs[0], NULL);
216 
217 fail4:
218     gpio_free(button_gpios[1]);
219 
220 fail3:
221     gpio_free(button_gpios[0]);
222 
223 fail2:
224     gpio_free(led_gpio);
225 #else
226 fail3:
227     free_irq(button_irqs[0], NULL);
228 
229 fail2:
230     gpio_free_array(buttons, ARRAY_SIZE(buttons));
231 
232 fail1:
233     gpio_free_array(leds, ARRAY_SIZE(leds));
234 #endif
235 
```

```
236     return ret;
237 }
238
239 static void __exit bottomhalf_exit(void)
240 {
241     pr_info("%s\n", __func__);
242
243     /* free irqs */
244     free_irq(button_irqs[0], NULL);
245     free_irq(button_irqs[1], NULL);
246
247     /* turn all LEDs off */
248 #ifdef NO_GPIO_REQUEST_ARRAY
249     gpio_set_value(led_gpio, 0);
250 #else
251     int i;
252     for (i = 0; i < ARRAY_SIZE(leds); i++)
253         gpio_set_value(leds[i].gpio, 0);
254 #endif
255
256     /* unregister */
257 #ifdef NO_GPIO_REQUEST_ARRAY
258     gpio_free(led_gpio);
259     gpio_free(button_gpios[0]);
260     gpio_free(button_gpios[1]);
261 #else
262     gpio_free_array(leds, ARRAY_SIZE(leds));
263     gpio_free_array(buttons, ARRAY_SIZE(buttons));
264 #endif
265 }
266
267 module_init(bottomhalf_init);
268 module_exit(bottomhalf_exit);
269
270 MODULE_LICENSE("GPL");
271 MODULE_DESCRIPTION("Interrupt with top and bottom half");
```

23.4 Threaded IRQ

Threaded IRQ is a mechanism to organize both top-half and bottom-half of an IRQ at once. A threaded IRQ splits the one handler in `request_irq()` into two: one for the top-half, the other for the bottom-half. The `request_threaded_irq()` is the function for using threaded IRQs. Two handlers are registered at once in the `request_threaded_irq()`.

Those two handlers run in different context. The top-half handler runs in interrupt context. It's the equivalence of the handler passed to the `request_irq()`. The bottom-half handler on the other hand runs in its own thread. This thread is created on registration of a threaded IRQ. Its sole purpose is to run this bottom-half handler. This is where a threaded IRQ is "threaded". If `IRQ_WAKE_THREAD` is returned by the top-half handler, that bottom-half serving thread will wake up. The thread then runs the bottom-half handler.

Here is an example of how to do the same thing as before, with top and

bottom halves, but using threads.

```c
/*
 * bh_threaded.c - Top and bottom half interrupt handling
 *
 * Based upon the RPi example by Stefan Wendler (devnull@kaltpost.de)
 * from:
 *    https://github.com/wendlers/rpi-kmod-samples
 *
 * Press one button to turn on a LED and another to turn it off
 */

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/gpio.h>
#include <linux/delay.h>
#include <linux/interrupt.h>
#include <linux/version.h>

#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 10, 0)
#define NO_GPIO_REQUEST_ARRAY
#endif

static int button_irqs[] = { -1, -1 };

/* Define GPIOs for LEDs.
 * FIXME: Change the numbers for the GPIO on your board.
 */
#ifdef NO_GPIO_REQUEST_ARRAY
static unsigned int led_gpio = 4;
static unsigned int button_gpios[] = { 17, 18 };
#else
static struct gpio leds[] = { { 4, GPIOF_OUT_INIT_LOW, "LED 1" } };

/* Define GPIOs for BUTTONS
 * FIXME: Change the numbers for the GPIO on your board.
 */
static struct gpio buttons[] = {
    { 17, GPIOF_IN, "LED 1 ON BUTTON" },
    { 18, GPIOF_IN, "LED 1 OFF BUTTON" },
};
#endif

/* This happens immediately, when the IRQ is triggered */
static irqreturn_t button_top_half(int irq, void *ident)
{
    return IRQ_WAKE_THREAD;
}

/* This can happen at leisure, freeing up IRQs for other high priority task */
static irqreturn_t button_bottom_half(int irq, void *ident)
{
    pr_info("Bottom half task starts\n");
    mdelay(500); /* do something which takes a while */
    pr_info("Bottom half task ends\n");
    return IRQ_HANDLED;
}
```

```
56
57  static int __init bottomhalf_init(void)
58  {
59      int ret = 0;
60
61      pr_info("%s\n", __func__);
62
63  /* register LED gpios */
64  #ifdef NO_GPIO_REQUEST_ARRAY
65      ret = gpio_request(led_gpio, "LED 1");
66  #else
67      ret = gpio_request_array(leds, ARRAY_SIZE(leds));
68  #endif
69
70      if (ret) {
71          pr_err("Unable to request GPIOs for LEDs: %d\n", ret);
72          return ret;
73      }
74
75  #ifdef NO_GPIO_REQUEST_ARRAY
76      ret = gpio_direction_output(led_gpio, 0);
77
78      if (ret) {
79          pr_err("Unable to configure LED GPIO direction: %d\n", ret);
80          goto fail2;
81      }
82  #endif
83
84  /* register BUTTON gpios */
85  #ifdef NO_GPIO_REQUEST_ARRAY
86      ret = gpio_request(button_gpios[0], "LED 1 ON BUTTON");
87
88      if (ret) {
89          pr_err("Unable to request GPIOs for BUTTONs: %d\n", ret);
90          goto fail2;
91      }
92
93      ret = gpio_request(button_gpios[1], "LED 1 OFF BUTTON");
94
95      if (ret) {
96          pr_err("Unable to request GPIOs for BUTTONs: %d\n", ret);
97          goto fail3;
98      }
99
100     ret = gpio_direction_input(button_gpios[0]);
101
102     if (ret) {
103         pr_err("Unable to configure BUTTON1 GPIO direction: %d\n", ret);
104         goto fail4;
105     }
106
107     ret = gpio_direction_input(button_gpios[1]);
108
109     if (ret) {
110         pr_err("Unable to configure BUTTON2 GPIO direction: %d\n", ret);
111         goto fail4;
112     }
```

```
#else
    ret = gpio_request_array(buttons, ARRAY_SIZE(buttons));

    if (ret) {
        pr_err("Unable to request GPIOs for BUTTONs: %d\n", ret);
        goto fail1;
    }
#endif

#ifdef NO_GPIO_REQUEST_ARRAY
    pr_info("Current button1 value: %d\n", gpio_get_value(button_gpios[0]));

    ret = gpio_to_irq(button_gpios[0]);
#else
    pr_info("Current button1 value: %d\n", gpio_get_value(buttons[0].gpio));

    ret = gpio_to_irq(buttons[0].gpio);
#endif

    if (ret < 0) {
        pr_err("Unable to request IRQ: %d\n", ret);
#ifdef NO_GPIO_REQUEST_ARRAY
        goto fail4;
#else
        goto fail2;
#endif
    }

    button_irqs[0] = ret;

    pr_info("Successfully requested BUTTON1 IRQ # %d\n", button_irqs[0]);

    ret = request_threaded_irq(button_irqs[0], button_top_half,
                               button_bottom_half,
                               IRQF_TRIGGER_RISING | IRQF_TRIGGER_FALLING,
#ifdef NO_GPIO_REQUEST_ARRAY
                               "gpiomod#button1", NULL);
#else
                               "gpiomod#button1", &buttons[0]);
#endif

    if (ret) {
        pr_err("Unable to request IRQ: %d\n", ret);
#ifdef NO_GPIO_REQUEST_ARRAY
        goto fail4;
#else
        goto fail2;
#endif
    }

#ifdef NO_GPIO_REQUEST_ARRAY
    ret = gpio_to_irq(button_gpios[1]);
#else
    ret = gpio_to_irq(buttons[1].gpio);
#endif

    if (ret < 0) {
```

```
170         pr_err("Unable to request IRQ: %d\n", ret);
171 #ifdef NO_GPIO_REQUEST_ARRAY
172         goto fail5;
173 #else
174         goto fail2;
175 #endif
176     }
177 
178     button_irqs[1] = ret;
179 
180     pr_info("Successfully requested BUTTON2 IRQ # %d\n", button_irqs[1]);
181 
182     ret = request_threaded_irq(button_irqs[1], button_top_half,
183                                button_bottom_half,
184                                IRQF_TRIGGER_RISING | IRQF_TRIGGER_FALLING,
185 #ifdef NO_GPIO_REQUEST_ARRAY
186                                "gpiomod#button2", NULL);
187 #else
188                                "gpiomod#button2", &buttons[1]);
189 #endif
190 
191     if (ret) {
192         pr_err("Unable to request IRQ: %d\n", ret);
193 #ifdef NO_GPIO_REQUEST_ARRAY
194         goto fail5;
195 #else
196         goto fail3;
197 #endif
198     }
199 
200     return 0;
201 
202 /* cleanup what has been setup so far */
203 #ifdef NO_GPIO_REQUEST_ARRAY
204 fail5:
205     free_irq(button_irqs[0], NULL);
206 
207 fail4:
208     gpio_free(button_gpios[1]);
209 
210 fail3:
211     gpio_free(button_gpios[0]);
212 
213 fail2:
214     gpio_free(led_gpio);
215 #else
216 fail3:
217     free_irq(button_irqs[0], &buttons[0]);
218 
219 fail2:
220     gpio_free_array(buttons, ARRAY_SIZE(buttons));
221 
222 fail1:
223     gpio_free_array(leds, ARRAY_SIZE(leds));
224 #endif
225 
226     return ret;
```

```
227 }
228
229 static void __exit bottomhalf_exit(void)
230 {
231     pr_info("%s\n", __func__);
232
233     /* free irqs */
234 #ifdef NO_GPIO_REQUEST_ARRAY
235     free_irq(button_irqs[0], NULL);
236     free_irq(button_irqs[1], NULL);
237 #else
238     free_irq(button_irqs[0], &buttons[0]);
239     free_irq(button_irqs[1], &buttons[1]);
240 #endif
241
242 /* turn all LEDs off */
243 #ifdef NO_GPIO_REQUEST_ARRAY
244     gpio_set_value(led_gpio, 0);
245 #else
246     int i;
247     for (i = 0; i < ARRAY_SIZE(leds); i++)
248         gpio_set_value(leds[i].gpio, 0);
249 #endif
250
251 /* unregister */
252 #ifdef NO_GPIO_REQUEST_ARRAY
253     gpio_free(led_gpio);
254     gpio_free(button_gpios[0]);
255     gpio_free(button_gpios[1]);
256 #else
257     gpio_free_array(leds, ARRAY_SIZE(leds));
258     gpio_free_array(buttons, ARRAY_SIZE(buttons));
259 #endif
260 }
261
262 module_init(bottomhalf_init);
263 module_exit(bottomhalf_exit);
264
265 MODULE_LICENSE("GPL");
266 MODULE_DESCRIPTION("Interrupt with top and bottom half");
```

A threaded IRQ is registered using `request_threaded_irq()`. This function only takes one additional parameter than the `request_irq()` – the bottom-half handling function that runs in its own thread. In this example it is the `button_bottom_half()`. Usage of other parameters are the same as `request_irq()`.

Presence of both handlers is not mandatory. If either of them is not needed, pass the `NULL` instead. A `NULL` top-half handler implies that no action is taken except to wake up the bottom-half serving thread, which runs the bottom-half handler. Similarly, a `NULL` bottom-half handler effectively acts as if `request_irq()` were used. In fact, this is how `request_irq()` is implemented.

Note that passing `NULL` to both handlers is considered an error and will make registration fail.

24 Virtual Input Device Driver

The input device driver is a module that provides a way to communicate with the interaction device via the event. For example, the keyboard can send the press or release event to tell the kernel what we want to do. The input device driver will allocate a new input structure with `input_allocate_device()` and sets up input bitfields, device id, version, etc. After that, registers it by calling `input_register_device()`.

Here is an example, vinput, It is an API to allow easy development of virtual input drivers. The driver needs to export a `vinput_device()` that contains the virtual device name and `vinput_ops` structure that describes:

- the init function: `init()`
- the input event injection function: `send()`
- the readback function: `read()`

Then using `vinput_register_device()` and `vinput_unregister_device()` will add a new device to the list of support virtual input devices.

```
1 int init(struct vinput *);
```

This function is passed a `struct vinput` already initialized with an allocated `struct input_dev`. The `init()` function is responsible for initializing the capabilities of the input device and register it.

```
1 int send(struct vinput *, char *, int);
```

This function will receive a user string to interpret and inject the event using the `input_report_XXXX` or `input_event` call. The string is already copied from user.

```
1 int read(struct vinput *, char *, int);
```

This function is used for debugging and should fill the buffer parameter with the last event sent in the virtual input device format. The buffer will then be copied to user.

vinput devices are created and destroyed using sysfs. And, event injection is done through a `/dev` node. The device name will be used by the userland to export a new virtual input device.

The `class_attribute` structure is similar to other attribute types we talked about in Section 8:

```c
struct class_attribute {
    struct attribute attr;
    ssize_t (*show)(struct class *class, struct class_attribute *attr,
                    char *buf);
    ssize_t (*store)(struct class *class, struct class_attribute *attr,
                    const char *buf, size_t count);
};
```

In `vinput.c`, the macro `CLASS_ATTR_WO(export/unexport)` defined in include/linux/device.h (in this case, `device.h` is included in include/linux/input.h) will generate the `class_attribute` structures which are named `class_attr_export/unexport`. Then, put them into `vinput_class_attrs` array and the macro `ATTRIBUTE_GROUPS(vinput_class)` will generate the `struct attribute_group vinput_class_group` that should be assigned in `vinput_class`. Finally, call `class_register(&vinput_class)` to create attributes in sysfs.

To create a `vinputX` sysfs entry and `/dev` node.

```
echo "vkbd" | sudo tee /sys/class/vinput/export
```

To unexport the device, just echo its id in unexport:

```
echo "0" | sudo tee /sys/class/vinput/unexport
```

```c
/*
 * vinput.h
 */

#ifndef VINPUT_H
#define VINPUT_H

#include <linux/input.h>
#include <linux/spinlock.h>

#define VINPUT_MAX_LEN 128
#define MAX_VINPUT 32
#define VINPUT_MINORS MAX_VINPUT

#define dev_to_vinput(dev) container_of(dev, struct vinput, dev)

struct vinput_device;

struct vinput {
    long id;
    long devno;
    long last_entry;
    spinlock_t lock;

    void *priv_data;

```

```
    struct device dev;
    struct list_head list;
    struct input_dev *input;
    struct vinput_device *type;
};

struct vinput_ops {
    int (*init)(struct vinput *);
    int (*kill)(struct vinput *);
    int (*send)(struct vinput *, char *, int);
    int (*read)(struct vinput *, char *, int);
};

struct vinput_device {
    char name[16];
    struct list_head list;
    struct vinput_ops *ops;
};

int vinput_register(struct vinput_device *dev);
void vinput_unregister(struct vinput_device *dev);

#endif
```

```
/*
 * vinput.c
 */

#include <linux/cdev.h>
#include <linux/input.h>
#include <linux/module.h>
#include <linux/slab.h>
#include <linux/spinlock.h>
#include <linux/version.h>

#include <asm/uaccess.h>

#include "vinput.h"

#define DRIVER_NAME "vinput"

#define dev_to_vinput(dev) container_of(dev, struct vinput, dev)

static DECLARE_BITMAP(vinput_ids, VINPUT_MINORS);

static LIST_HEAD(vinput_devices);
static LIST_HEAD(vinput_vdevices);

static int vinput_dev;
static struct spinlock vinput_lock;
static struct class vinput_class;

/* Search the name of vinput device in the vinput_devices linked list,
 * which added at vinput_register().
 */
static struct vinput_device *vinput_get_device_by_type(const char *type)
{
```

```
    int found = 0;
    struct vinput_device *vinput;
    struct list_head *curr;

    spin_lock(&vinput_lock);
    list_for_each (curr, &vinput_devices) {
        vinput = list_entry(curr, struct vinput_device, list);
        if (vinput && strncmp(type, vinput->name, strlen(vinput->name)) == 0)
        ↪  {
            found = 1;
            break;
        }
    }
    spin_unlock(&vinput_lock);

    if (found)
        return vinput;
    return ERR_PTR(-ENODEV);
}

/* Search the id of virtual device in the vinput_vdevices linked list,
 * which added at vinput_alloc_vdevice().
 */
static struct vinput *vinput_get_vdevice_by_id(long id)
{
    struct vinput *vinput = NULL;
    struct list_head *curr;

    spin_lock(&vinput_lock);
    list_for_each (curr, &vinput_vdevices) {
        vinput = list_entry(curr, struct vinput, list);
        if (vinput && vinput->id == id)
            break;
    }
    spin_unlock(&vinput_lock);

    if (vinput && vinput->id == id)
        return vinput;
    return ERR_PTR(-ENODEV);
}

static int vinput_open(struct inode *inode, struct file *file)
{
    int err = 0;
    struct vinput *vinput = NULL;

    vinput = vinput_get_vdevice_by_id(iminor(inode));

    if (IS_ERR(vinput))
        err = PTR_ERR(vinput);
    else
        file->private_data = vinput;

    return err;
}

static int vinput_release(struct inode *inode, struct file *file)
```

```
{
    return 0;
}

static ssize_t vinput_read(struct file *file, char __user *buffer, size_t
↪  count,
                           loff_t *offset)
{
    int len;
    char buff[VINPUT_MAX_LEN + 1];
    struct vinput *vinput = file->private_data;

    len = vinput->type->ops->read(vinput, buff, count);

    if (*offset > len)
        count = 0;
    else if (count + *offset > VINPUT_MAX_LEN)
        count = len - *offset;

    if (raw_copy_to_user(buffer, buff + *offset, count))
        return -EFAULT;

    *offset += count;

    return count;
}

static ssize_t vinput_write(struct file *file, const char __user *buffer,
                            size_t count, loff_t *offset)
{
    char buff[VINPUT_MAX_LEN + 1];
    struct vinput *vinput = file->private_data;

    memset(buff, 0, sizeof(char) * (VINPUT_MAX_LEN + 1));

    if (count > VINPUT_MAX_LEN) {
        dev_warn(&vinput->dev, "Too long. %d bytes allowed\n",
        ↪  VINPUT_MAX_LEN);
        return -EINVAL;
    }

    if (raw_copy_from_user(buff, buffer, count))
        return -EFAULT;

    return vinput->type->ops->send(vinput, buff, count);
}

static const struct file_operations vinput_fops = {
    .owner = THIS_MODULE,
    .open = vinput_open,
    .release = vinput_release,
    .read = vinput_read,
    .write = vinput_write,
};

static void vinput_unregister_vdevice(struct vinput *vinput)
{
```

```
145     input_unregister_device(vinput->input);
146     if (vinput->type->ops->kill)
147         vinput->type->ops->kill(vinput);
148 }
149 
150 static void vinput_destroy_vdevice(struct vinput *vinput)
151 {
152     /* Remove from the list first */
153     spin_lock(&vinput_lock);
154     list_del(&vinput->list);
155     clear_bit(vinput->id, vinput_ids);
156     spin_unlock(&vinput_lock);
157 
158     kfree(vinput);
159 }
160 
161 static void vinput_release_dev(struct device *dev)
162 {
163     struct vinput *vinput = dev_to_vinput(dev);
164     int id = vinput->id;
165 
166     vinput_destroy_vdevice(vinput);
167 
168     pr_debug("released vinput%d.\n", id);
169 }
170 
171 static struct vinput *vinput_alloc_vdevice(void)
172 {
173     int err;
174     struct vinput *vinput = kzalloc(sizeof(struct vinput), GFP_KERNEL);
175 
176     if (!vinput) {
177         pr_err("vinput: Cannot allocate vinput input device\n");
178         return ERR_PTR(-ENOMEM);
179     }
180 
181     spin_lock_init(&vinput->lock);
182 
183     spin_lock(&vinput_lock);
184     vinput->id = find_first_zero_bit(vinput_ids, VINPUT_MINORS);
185     if (vinput->id >= VINPUT_MINORS) {
186         err = -ENOBUFS;
187         goto fail_id;
188     }
189     set_bit(vinput->id, vinput_ids);
190     list_add(&vinput->list, &vinput_vdevices);
191     spin_unlock(&vinput_lock);
192 
193     /* allocate the input device */
194     vinput->input = input_allocate_device();
195     if (vinput->input == NULL) {
196         pr_err("vinput: Cannot allocate vinput input device\n");
197         err = -ENOMEM;
198         goto fail_input_dev;
199     }
200 
201     /* initialize device */
```

```
202     vinput->dev.class = &vinput_class;
203     vinput->dev.release = vinput_release_dev;
204     vinput->dev.devt = MKDEV(vinput_dev, vinput->id);
205     dev_set_name(&vinput->dev, DRIVER_NAME "%lu", vinput->id);
206
207     return vinput;
208
209 fail_input_dev:
210     spin_lock(&vinput_lock);
211     list_del(&vinput->list);
212 fail_id:
213     spin_unlock(&vinput_lock);
214     kfree(vinput);
215
216     return ERR_PTR(err);
217 }
218
219 static int vinput_register_vdevice(struct vinput *vinput)
220 {
221     int err = 0;
222
223     /* register the input device */
224     vinput->input->name = vinput->type->name;
225     vinput->input->phys = "vinput";
226     vinput->input->dev.parent = &vinput->dev;
227
228     vinput->input->id.bustype = BUS_VIRTUAL;
229     vinput->input->id.product = 0x0000;
230     vinput->input->id.vendor = 0x0000;
231     vinput->input->id.version = 0x0000;
232
233     err = vinput->type->ops->init(vinput);
234
235     if (err == 0)
236         dev_info(&vinput->dev, "Registered virtual input %s %ld\n",
237                  vinput->type->name, vinput->id);
238
239     return err;
240 }
241
242 #if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 4, 0)
243 static ssize_t export_store(const struct class *class,
244                             const struct class_attribute *attr,
245 #else
246 static ssize_t export_store(struct class *class, struct class_attribute *attr,
247 #endif
248                             const char *buf, size_t len)
249 {
250     int err;
251     struct vinput *vinput;
252     struct vinput_device *device;
253
254     device = vinput_get_device_by_type(buf);
255     if (IS_ERR(device)) {
256         pr_info("vinput: This virtual device isn't registered\n");
257         err = PTR_ERR(device);
258         goto fail;
```

```
    }

    vinput = vinput_alloc_vdevice();
    if (IS_ERR(vinput)) {
        err = PTR_ERR(vinput);
        goto fail;
    }

    vinput->type = device;
    err = device_register(&vinput->dev);
    if (err < 0)
        goto fail_register;

    err = vinput_register_vdevice(vinput);
    if (err < 0)
        goto fail_register_vinput;

    return len;

fail_register_vinput:
    input_free_device(vinput->input);
    device_unregister(&vinput->dev);
    /* avoid calling vinput_destroy_vdevice() twice */
    return err;
fail_register:
    input_free_device(vinput->input);
    vinput_destroy_vdevice(vinput);
fail:
    return err;
}
/* This macro generates class_attr_export structure and export_store() */
static CLASS_ATTR_WO(export);

#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 4, 0)
static ssize_t unexport_store(const struct class *class,
                              const struct class_attribute *attr,
#else
static ssize_t unexport_store(struct class *class, struct class_attribute
↪  *attr,
#endif
                              const char *buf, size_t len)
{
    int err;
    unsigned long id;
    struct vinput *vinput;

    err = kstrtol(buf, 10, &id);
    if (err) {
        err = -EINVAL;
        goto failed;
    }

    vinput = vinput_get_vdevice_by_id(id);
    if (IS_ERR(vinput)) {
        pr_err("vinput: No such vinput device %ld\n", id);
        err = PTR_ERR(vinput);
        goto failed;
```

```
315     }
316
317     vinput_unregister_vdevice(vinput);
318     device_unregister(&vinput->dev);
319
320     return len;
321 failed:
322     return err;
323 }
324 /* This macro generates class_attr_unexport structure and unexport_store() */
325 static CLASS_ATTR_WO(unexport);
326
327 static struct attribute *vinput_class_attrs[] = {
328     &class_attr_export.attr,
329     &class_attr_unexport.attr,
330     NULL,
331 };
332
333 /* This macro generates vinput_class_groups structure */
334 ATTRIBUTE_GROUPS(vinput_class);
335
336 static struct class vinput_class = {
337     .name = "vinput",
338 /* .owner was removed in Linux v6.4 via upstream commit 6e30a66433af ("driver
    ↪  core: class: remove
339  * struct module owner out of struct class")
340  */
341 #if LINUX_VERSION_CODE < KERNEL_VERSION(6, 4, 0)
342     .owner = THIS_MODULE,
343 #endif
344     .class_groups = vinput_class_groups,
345 };
346
347 int vinput_register(struct vinput_device *dev)
348 {
349     spin_lock(&vinput_lock);
350     list_add(&dev->list, &vinput_devices);
351     spin_unlock(&vinput_lock);
352
353     pr_info("vinput: registered new virtual input device '%s'\n", dev->name);
354
355     return 0;
356 }
357 EXPORT_SYMBOL(vinput_register);
358
359 void vinput_unregister(struct vinput_device *dev)
360 {
361     struct list_head *curr, *next;
362
363     /* Remove from the list first */
364     spin_lock(&vinput_lock);
365     list_del(&dev->list);
366     spin_unlock(&vinput_lock);
367
368     /* unregister all devices of this type */
369     list_for_each_safe (curr, next, &vinput_vdevices) {
370         struct vinput *vinput = list_entry(curr, struct vinput, list);
```

```
371         if (vinput && vinput->type == dev) {
372             vinput_unregister_vdevice(vinput);
373             device_unregister(&vinput->dev);
374         }
375     }
376 
377     pr_info("vinput: unregistered virtual input device '%s'\n", dev->name);
378 }
379 EXPORT_SYMBOL(vinput_unregister);
380 
381 static int __init vinput_init(void)
382 {
383     int err = 0;
384 
385     pr_info("vinput: Loading virtual input driver\n");
386 
387     vinput_dev = register_chrdev(0, DRIVER_NAME, &vinput_fops);
388     if (vinput_dev < 0) {
389         pr_err("vinput: Unable to allocate char dev region\n");
390         err = vinput_dev;
391         goto failed_alloc;
392     }
393 
394     spin_lock_init(&vinput_lock);
395 
396     err = class_register(&vinput_class);
397     if (err < 0) {
398         pr_err("vinput: Unable to register vinput class\n");
399         goto failed_class;
400     }
401 
402     return 0;
403 failed_class:
404     unregister_chrdev(vinput_dev, DRIVER_NAME);
405 failed_alloc:
406     return err;
407 }
408 
409 static void __exit vinput_end(void)
410 {
411     pr_info("vinput: Unloading virtual input driver\n");
412 
413     unregister_chrdev(vinput_dev, DRIVER_NAME);
414     class_unregister(&vinput_class);
415 }
416 
417 module_init(vinput_init);
418 module_exit(vinput_end);
419 
420 MODULE_LICENSE("GPL");
421 MODULE_DESCRIPTION("Emulate input events");
```

Here the virtual keyboard is one of example to use vinput. It supports all `KEY_MAX` keycodes. The injection format is the `KEY_CODE` such as defined in include/linux/input.h. A positive value means `KEY_PRESS` while a negative value is a `KEY_RELEASE`. The keyboard supports repetition when the key stays

pressed for too long. The following demonstrates how simulation work.

Simulate a key press on "g" (`KEY_G` = 34):

```
echo "+34" | sudo tee /dev/vinput0
```

Simulate a key release on "g" (`KEY_G` = 34):

```
echo "-34" | sudo tee /dev/vinput0
```

```
/*
 * vkbd.c
 */

#include <linux/init.h>
#include <linux/input.h>
#include <linux/module.h>
#include <linux/spinlock.h>

#include "vinput.h"

#define VINPUT_KBD "vkbd"
#define VINPUT_RELEASE 0
#define VINPUT_PRESS 1

static unsigned short vkeymap[KEY_MAX];

static int vinput_vkbd_init(struct vinput *vinput)
{
    int i;

    /* Set up the input bitfield */
    vinput->input->evbit[0] = BIT_MASK(EV_KEY) | BIT_MASK(EV_REP);
    vinput->input->keycodesize = sizeof(unsigned short);
    vinput->input->keycodemax = KEY_MAX;
    vinput->input->keycode = vkeymap;

    for (i = 0; i < KEY_MAX; i++)
        set_bit(vkeymap[i], vinput->input->keybit);

    /* vinput will help us allocate new input device structure via
     * input_allocate_device(). So, we can register it straightforwardly.
     */
    return input_register_device(vinput->input);
}

static int vinput_vkbd_read(struct vinput *vinput, char *buff, int len)
{
    spin_lock(&vinput->lock);
    len = snprintf(buff, len, "%+ld\n", vinput->last_entry);
    spin_unlock(&vinput->lock);

    return len;
}
```

```
45
46  static int vinput_vkbd_send(struct vinput *vinput, char *buff, int len)
47  {
48      int ret;
49      long key = 0;
50      short type = VINPUT_PRESS;
51
52      /* Determine which event was received (press or release)
53       * and store the state.
54       */
55      if (buff[0] == '+')
56          ret = kstrtol(buff + 1, 10, &key);
57      else
58          ret = kstrtol(buff, 10, &key);
59      if (ret)
60          dev_err(&vinput->dev, "error during kstrtol: -%d\n", ret);
61      spin_lock(&vinput->lock);
62      vinput->last_entry = key;
63      spin_unlock(&vinput->lock);
64
65      if (key < 0) {
66          type = VINPUT_RELEASE;
67          key = -key;
68      }
69
70      dev_info(&vinput->dev, "Event %s code %ld\n",
71               (type == VINPUT_RELEASE) ? "VINPUT_RELEASE" : "VINPUT_PRESS",
                 ↪  key);
72
73      /* Report the state received to input subsystem. */
74      input_report_key(vinput->input, key, type);
75      /* Tell input subsystem that it finished the report. */
76      input_sync(vinput->input);
77
78      return len;
79  }
80
81  static struct vinput_ops vkbd_ops = {
82      .init = vinput_vkbd_init,
83      .send = vinput_vkbd_send,
84      .read = vinput_vkbd_read,
85  };
86
87  static struct vinput_device vkbd_dev = {
88      .name = VINPUT_KBD,
89      .ops = &vkbd_ops,
90  };
91
92  static int __init vkbd_init(void)
93  {
94      int i;
95
96      for (i = 0; i < KEY_MAX; i++)
97          vkeymap[i] = i;
98      return vinput_register(&vkbd_dev);
99  }
100
```

```
101 static void __exit vkbd_end(void)
102 {
103     vinput_unregister(&vkbd_dev);
104 }
105
106 module_init(vkbd_init);
107 module_exit(vkbd_end);
108
109 MODULE_LICENSE("GPL");
110 MODULE_DESCRIPTION("Emulate keyboard input events through /dev/vinput");
```

25 PCI Drivers

PCI and PCI Express are the most common discovery-based buses on desktop and server systems. Unlike the earlier GPIO or simple platform examples, a PCI driver usually does not instantiate devices itself. Instead, the kernel enumerates devices during boot or hotplug, identifies them by vendor and device identifiers, and calls into a driver whose match table declares support for that hardware.

The kernel-side entry point is `struct pci_driver`. Much like `platform_driver`, it provides `probe` and `remove` callbacks plus an ID table. The driver is then registered with `pci_register_driver()` or, more often in modern code, with the helper macros that wrap it. When the bus finds a matching device, the probe function receives a `struct pci_dev *` and can start claiming resources.

The common probe sequence has a recognizable shape:

1. Enable the device with `pcim_enable_device()` or `pci_enable_device()`.
2. Set an appropriate DMA mask before preparing DMA-capable queues or buffers.
3. Request and map BAR resources, commonly through managed helpers layered on top of the resource API.
4. Configure interrupts, often preferring MSI or MSI-X when the device and platform support them.
5. Register the higher-level interface the driver exposes, such as a network device, block device, or character device.

BARs (Base Address Registers) are how PCI devices expose MMIO or I/O port regions. The driver should never assume fixed physical addresses; it must query the device through the PCI core and map the returned resource. That is one of the clearest examples of why real drivers are built around bus enumeration rather than hard-coded addresses.

Interrupt handling is also more structured than in ad hoc examples. Legacy INTx interrupts still exist, but MSI and MSI-X are usually preferred for modern PCIe hardware because they integrate better with multicore systems and avoid many sharing issues of pin-based interrupts. Still, the same context rules apply:

the top half should do the minimum required work, and any operation that may sleep belongs in a threaded handler, NAPI poll loop, workqueue, or subsystem-specific deferred path.

PCI drivers also make heavy use of the device model. Each `pci_dev` embeds a `struct device`, which means managed resources, sysfs integration, power management hooks, DMA configuration, and hot-unplug notification all flow through the same core infrastructure described later in Section 29. Once a driver moves beyond toy examples, that integration is usually more important than the raw bus API itself.

Linux 5.10-and-later code should also prefer the managed PCI interfaces where they fit. Helpers such as `pcim_enable_device()` and the device-managed resource APIs reduce teardown bugs dramatically, especially in drivers with multiple BARs or several error paths in probe. The goal is not just brevity; it is making probe failure and device removal boringly correct.

26 USB Drivers

USB is one of the biggest conceptual gaps between classic module tutorials and production driver work. The bus is hotpluggable by design, devices may appear and disappear at any time, configuration happens through descriptors rather than fixed wiring, and many drivers bind to interfaces rather than whole devices. That means USB code spends more effort on discovery, lifetime management, and I/O submission than on direct register access.

26.1 USB Device Model Basics

The USB core enumerates each newly attached device, reads its descriptors, and creates kernel objects that represent the device and its interfaces. Most function drivers bind to a `struct usb_interface` rather than directly to `struct usb_device`. This matters because one physical USB device can expose multiple interfaces, such as a combined webcam and microphone or a composite gadget with separate control and data functions.

A USB driver typically declares a `struct usb_driver` and an ID table of `struct usb_device_id` entries. Matching can be done on vendor/product identifiers, device class, interface class, or combinations of those fields. Once registered with `usb_register_driver()`, the driver is called via `probe` when a matching interface appears and `disconnect` when it goes away. The disconnect path is not optional bookkeeping; it is a normal and frequent event on USB systems.

The minimal skeleton looks conceptually like this:

```
static const struct usb_device_id my_table[] = {
    { USB_DEVICE(0x1234, 0x5678) },
    { }
};
MODULE_DEVICE_TABLE(usb, my_table);
```

```
6
7   static struct usb_driver my_driver = {
8       .name = "my_usb_driver",
9       .probe = my_probe,
10      .disconnect = my_disconnect,
11      .id_table = my_table,
12  };
```

The `probe` callback usually stores per-interface state with `usb_set_intfdata()`, retrieves endpoint information from the current alternate setting, allocates transfer buffers, and registers whatever higher-level interface will be visible to userspace. The matching `disconnect` callback must stop new I/O, tear down submitted transfers, and release those resources without assuming that userspace already closed every file descriptor.

26.2 Endpoints, Pipes, and URBs

USB communication is organized around endpoints. Descriptors tell the driver which endpoints exist, in which direction data flows, and whether each endpoint carries control, bulk, interrupt, or isochronous traffic. Drivers do not poke MMIO registers directly; instead they submit requests to the USB core, which hands them to a host-controller driver appropriate for the system's hardware.

The fundamental I/O object is the URB, or USB Request Block. A URB describes one transfer: the target pipe, a buffer, a length, a completion callback, and assorted flags. The driver allocates or initializes a URB, fills it with helper routines such as `usb_fill_bulk_urb()` or `usb_fill_int_urb()`, and submits it with `usb_submit_urb()`. Completion is asynchronous. That means the callback may run long after the initiating operation returned, and it may race with disconnect, timeout, suspend, or userspace shutdown.

URB lifetime rules are therefore central. The completion handler must know whether the enclosing device object is still alive, and the disconnect path must cancel outstanding traffic with interfaces such as `usb_kill_urb()` or `usb_poison_urb()` before tearing down the state they reference. This is a place where reference counting and explicit ownership discipline are not optional style points; they are the difference between a robust driver and a use-after-free bug.

Control transfers are often used for small device-specific commands and enumeration-time setup. Bulk transfers are the usual choice for throughput-oriented data such as storage or vendor-defined streams. Interrupt transfers are polled by the host at bounded intervals and are common for HID-style status traffic. Isochronous transfers exist for time-sensitive media streams, but they have more complex buffering and error behavior than a beginner guide usually needs to cover in detail.

26.3 USB and Userspace Interfaces

One of the strengths of USB is that many common device classes already have generic kernel drivers. Keyboards and mice usually bind to HID. Mass-storage

devices bind to the USB storage stack and appear as block devices. Serial adapters often bind to `usb-serial`. Before writing a custom driver, check whether the hardware really needs one or whether it should fit an existing class driver plus a tiny device-specific extension.

When a custom driver is needed, it still has choices about its userspace ABI. Some USB drivers register a character device node. Others integrate with an existing subsystem such as input, tty, networking, video4linux, IIO, or sound. That subsystem-first approach is usually the right one because it gives the device a standard userspace contract instead of inventing a private control scheme from scratch.

Sysfs also plays an important role in USB. Enumerated devices appear under `/sys/bus/usb/devices/`, where you can inspect descriptors, interface numbers, modalias strings, and power-management attributes. That is often the fastest way to confirm what the kernel thinks the hardware is before you write a line of driver code. For example:

```
ls /sys/bus/usb/devices/
cat /sys/bus/usb/devices/1-1/idVendor
cat /sys/bus/usb/devices/1-1/idProduct
cat /sys/bus/usb/devices/1-1/modalias
```

The modalias matters because it is what allows udev and module autoloading to connect a newly attached device to the right driver module. This is one of the cleanest examples of the kernel device model, bus matching, and userspace policy working together.

26.4 Hotplug, Suspend, and Failure Modes

USB drivers must assume surprise removal. Any blocking I/O may be interrupted because the cable was unplugged, a hub reset occurred, the device was power-cycled, or the bus suspended and resumed. This is why disconnect handling should flip an internal "device gone" state early, wake sleepers, and make future file operations fail cleanly instead of touching freed transport objects.

Power management also matters more on USB than many beginners expect. Autosuspend is common on laptops and embedded systems, and a driver that keeps a device permanently active without need may waste power or interfere with system suspend. At the same time, a driver that ignores runtime PM interactions may find that its transfers fail unexpectedly after resume. The exact hooks depend on the class and kernel subsystem involved, but the general rule is stable: device presence and device usability are not the same thing on a hotpluggable bus.

For Linux 5.10 and later, the safest mental model is to treat every submitted URB and every open userspace handle as potentially concurrent with disconnect. Once that assumption becomes part of the design, the rest of the driver tends to fall into a healthier shape.

27 Block Drivers

Block drivers are the foundation for disks, SSDs, loop devices, RAM disks, and many storage abstractions that ultimately present sector-oriented I/O to the rest of the kernel. Conceptually they sit between the generic block layer and the actual transport or media implementation. Unlike a character driver, a block driver is deeply integrated with caching, readahead, filesystem writeback, request merging, partition scanning, and storage-specific ordering constraints.

That integration is why block drivers are rarely tiny. The driver is not just answering `read()` and `write()` calls directly. It is participating in a larger pipeline that turns filesystem activity into BIOs and requests, schedules those requests, and completes them asynchronously. For Linux 5.10 and later, the key abstraction to understand is **blk-mq**, the multi-queue block layer used by modern block drivers.

27.1 The Modern Block Stack

Older texts often focus on request queues and callback paths that predate `blk-mq`. The durable concept is still useful: the kernel aggregates block I/O and hands it to the driver in a form that can be scheduled efficiently. But the concrete API for new drivers is the multi-queue interface built around `struct blk_mq_tag_set`, `struct blk_mq_ops`, a request queue created from that tag set, and a `gendisk` object that represents the exported disk. The helper names vary by kernel vintage: older 5.10-era code often shows `blk_mq_init_sq_queue()` or a separate `alloc_disk()` path, while newer kernels also provide helpers such as `blk_mq_alloc_disk()`.

The usual lifecycle looks like this:

1. Allocate and configure the driver's private device state.
2. Initialize a blk-mq tag set describing queue depth, hardware queues, and request callbacks.
3. Create the request queue from that tag set.
4. Allocate a disk object, set its capacity and block-device operations, and attach the queue.
5. Publish the disk with `add_disk()` once the device is actually ready to accept requests.

On teardown, the order matters just as much. The driver must stop new I/O from entering, drain or fail outstanding requests, remove the exported disk, and only then free queue-related resources. This is another subsystem where lifetime bugs often appear at removal time rather than during the happy-path data transfer.

27.2 BIOs, Requests, and Completion

At the lower end of the block layer, filesystem and page-cache activity becomes BIOs, which describe one or more contiguous ranges of block I/O backed by one or more pages. The block layer can merge or split those BIOs and eventually hands the driver a request object containing the transfer to execute. The driver then programs the hardware or software backend and completes the request asynchronously when the data has really been transferred.

That means a block driver should think in terms of request completion, not "returning bytes". The driver may process multiple requests in parallel, issue them out of order if the device semantics permit it, or fail them individually with an error status. The correctness boundary is whether the completed request accurately reflects what happened on the medium or backend, not whether some userspace syscall has already returned.

For software block devices such as RAM disks or loop-like backends, the underlying "hardware" may simply be memory or another file. Even then, it helps to follow the same layered model because that is what keeps the code compatible with the generic block layer's expectations around flushing, discard, writeback, and queue limits.

27.3 Queue Limits, Flushes, and Reality

Real storage devices have alignment, segment-count, maximum-transfer, cache, and flush semantics that must be communicated to the block layer. That is what queue limits are for. A driver that lies about those constraints may appear to work under light load and then fail badly once filesystems start issuing larger or more varied I/O patterns.

Flush and FUA semantics are especially important. Filesystems rely on them for persistence guarantees. If a device has a volatile write cache, the driver must either implement the necessary flush operations correctly or report the capability honestly so higher layers can compensate. This is one place where a toy block driver can teach the mechanics, but only a careful reading of storage semantics makes it production-safe.

Linux 5.10-and-later block drivers also need to fit into the device model and often the DMA model discussed earlier. NVMe, SCSI HBAs, virtio-blk, and MMC drivers all live at that intersection: requests arrive from blk-mq, are translated into transport-specific commands, often move data through DMA, and complete back into the generic block layer.

The sample below is intentionally modest: it registers a tiny RAM-backed block device with blk-mq and services requests by copying sectors to and from an in-memory buffer. That keeps the example safe to load in a VM while still showing the modern queue setup and request-completion flow. Because the block layer helper set changed after Linux v5.10, the sample uses a small version guard to select either the older queue-plus-`alloc_disk()` path or the newer `blk_mq_alloc_disk()` helper:

```c
// SPDX-License-Identifier: GPL-2.0
/*
 * blkram.c - Tiny RAM-backed block device using blk-mq.
 *
 * Covers Linux 5.10 through 6.17+ by using version guards at the three
 * major block-layer API transitions:
 *
 *   5.10-5.14  alloc_disk() + blk_mq_init_queue() + blk_cleanup_queue()
 *   5.15-6.8   blk_mq_alloc_disk(set, queuedata) [2-arg macro]
 *   6.9+       blk_mq_alloc_disk(set, lim, queuedata) [3-arg macro]
 *
 * Teardown likewise varies: blk_cleanup_queue() was removed in 5.15,
 * blk_cleanup_disk() was removed in 5.18; modern kernels use
 * del_gendisk() + put_disk().
 */

#include <linux/blk-mq.h>
#include <linux/blkdev.h>
#include <linux/errno.h>
#include <linux/highmem.h>
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/slab.h>
#include <linux/version.h>
#include <linux/vmalloc.h>

/* genhd.h was removed in 5.18; its content lives in blkdev.h since 5.17. */
#if LINUX_VERSION_CODE < KERNEL_VERSION(5, 18, 0)
#include <linux/genhd.h>
#endif

#define BLKRAM_SECTOR_SIZE 512

static unsigned long blkram_mb = 8;
module_param(blkram_mb, ulong, 0444);
MODULE_PARM_DESC(blkram_mb, "Size of the RAM disk in MiB");

struct blkram_dev {
    struct blk_mq_tag_set tag_set;
    struct gendisk *disk;
    struct request_queue *queue;
    u8 *data;
    size_t size;
};

static struct blkram_dev *blkram;
static int blkram_major;

static blk_status_t blkram_transfer(struct blkram_dev *dev, struct request
↪  *rq)
{
    struct req_iterator iter;
    struct bio_vec bvec;
    sector_t sector = blk_rq_pos(rq);
    unsigned long offset = (unsigned long)sector * BLKRAM_SECTOR_SIZE;
```

```
56
57      rq_for_each_segment(bvec, rq, iter)
58      {
59          unsigned int len = bvec.bv_len;
60          void *iobuf;
61
62          if (offset + len > dev->size)
63              return BLK_STS_IOERR;
64
65  /* kmap_local_page() appeared in 5.11; fall back to kmap_atomic() on 5.10. */
66  #if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 11, 0)
67          iobuf = kmap_local_page(bvec.bv_page) + bvec.bv_offset;
68  #else
69          iobuf = kmap_atomic(bvec.bv_page) + bvec.bv_offset;
70  #endif
71
72          if (rq_data_dir(rq) == WRITE)
73              memcpy(dev->data + offset, iobuf, len);
74          else
75              memcpy(iobuf, dev->data + offset, len);
76
77  #if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 11, 0)
78          kunmap_local(iobuf);
79  #else
80          kunmap_atomic(iobuf);
81  #endif
82          offset += len;
83      }
84
85      return BLK_STS_OK;
86  }
87
88  static blk_status_t blkram_queue_rq(struct blk_mq_hw_ctx *hctx,
89                                      const struct blk_mq_queue_data *bd)
90  {
91      struct request *rq = bd->rq;
92      struct blkram_dev *dev = rq->q->queuedata;
93      blk_status_t status;
94
95      blk_mq_start_request(rq);
96
97      if (blk_rq_is_passthrough(rq))
98          status = BLK_STS_IOERR;
99      else
100         status = blkram_transfer(dev, rq);
101
102     blk_mq_end_request(rq, status);
103
104     return BLK_STS_OK;
105 }
106
107 static const struct blk_mq_ops blkram_mq_ops = {
108     .queue_rq = blkram_queue_rq,
109 };
110
111 static const struct block_device_operations blkram_fops = {
112     .owner = THIS_MODULE,
```

```
};

static int __init blkram_init(void)
{
    unsigned long sectors;
    int ret;

    blkram_major = register_blkdev(0, "blkram");
    if (blkram_major < 0)
        return blkram_major;

    blkram = kzalloc(sizeof(*blkram), GFP_KERNEL);
    if (!blkram) {
        ret = -ENOMEM;
        goto err_unreg;
    }

    blkram->size = blkram_mb * 1024 * 1024;
    sectors = blkram->size / BLKRAM_SECTOR_SIZE;

    blkram->data = vzalloc(blkram->size);
    if (!blkram->data) {
        ret = -ENOMEM;
        goto err_free_dev;
    }

    blkram->tag_set.ops = &blkram_mq_ops;
    blkram->tag_set.nr_hw_queues = 1;
    blkram->tag_set.queue_depth = 64;
    blkram->tag_set.numa_node = NUMA_NO_NODE;
    blkram->tag_set.cmd_size = 0;
    /* BLK_MQ_F_SHOULD_MERGE was removed in 6.6+; merging is always on. */
#if LINUX_VERSION_CODE < KERNEL_VERSION(6, 6, 0)
    blkram->tag_set.flags = BLK_MQ_F_SHOULD_MERGE;
#else
    blkram->tag_set.flags = 0;
#endif
    blkram->tag_set.driver_data = blkram;

    ret = blk_mq_alloc_tag_set(&blkram->tag_set);
    if (ret)
        goto err_free_data;

/* Three eras of block-device creation:
 *   6.9+      blk_mq_alloc_disk(set, lim, queuedata)  -- 3-arg form.
 *   5.15-6.8  blk_mq_alloc_disk(set, queuedata)       -- 2-arg form.
 *   5.10-5.14 alloc_disk() + blk_mq_init_queue()      -- separate objects.
 */
#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 9, 0)
    {
        struct queue_limits lim = {
            .logical_block_size = BLKRAM_SECTOR_SIZE,
        };
        blkram->disk = blk_mq_alloc_disk(&blkram->tag_set, &lim, blkram);
    }
    if (IS_ERR(blkram->disk)) {
        ret = PTR_ERR(blkram->disk);
```

```
        goto err_tag_set;
    }
    blkram->queue = blkram->disk->queue;
#elif LINUX_VERSION_CODE >= KERNEL_VERSION(5, 15, 0)
    blkram->disk = blk_mq_alloc_disk(&blkram->tag_set, blkram);
    if (IS_ERR(blkram->disk)) {
        ret = PTR_ERR(blkram->disk);
        goto err_tag_set;
    }
    blkram->queue = blkram->disk->queue;
#else
    blkram->queue = blk_mq_init_queue(&blkram->tag_set);
    if (IS_ERR(blkram->queue)) {
        ret = PTR_ERR(blkram->queue);
        goto err_tag_set;
    }

    blkram->disk = alloc_disk(1);
    if (!blkram->disk) {
        ret = -ENOMEM;
        goto err_cleanup_queue;
    }

    blkram->disk->queue = blkram->queue;
#endif

    blkram->queue->queuedata = blkram;
    blkram->disk->major = blkram_major;
    blkram->disk->first_minor = 0;
    blkram->disk->minors = 1;
    blkram->disk->fops = &blkram_fops;
    blkram->disk->private_data = blkram;

    snprintf(blkram->disk->disk_name, DISK_NAME_LEN, "blkram0");
#if LINUX_VERSION_CODE < KERNEL_VERSION(6, 9, 0)
    blk_queue_logical_block_size(blkram->queue, BLKRAM_SECTOR_SIZE);
#endif
    set_capacity(blkram->disk, sectors);

    /* add_disk() returns int since 5.16; earlier kernels return void. */
#if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 16, 0)
    ret = add_disk(blkram->disk);
    if (ret)
        goto err_put_disk;
#else
    add_disk(blkram->disk);
#endif

    pr_info("blkram: registered /dev/%s (%lu MiB)\n", blkram->disk->disk_name,
            blkram_mb);

    return 0;

#if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 16, 0)
err_put_disk:
    put_disk(blkram->disk);
#endif
```

```
#if LINUX_VERSION_CODE < KERNEL_VERSION(5, 15, 0)
err_cleanup_queue:
    blk_cleanup_queue(blkram->queue);
#endif
err_tag_set:
    blk_mq_free_tag_set(&blkram->tag_set);
err_free_data:
    vfree(blkram->data);
err_free_dev:
    kfree(blkram);
err_unreg:
    unregister_blkdev(blkram_major, "blkram");
    return ret;
}

static void __exit blkram_exit(void)
{
    del_gendisk(blkram->disk);
    put_disk(blkram->disk);
#if LINUX_VERSION_CODE < KERNEL_VERSION(5, 15, 0)
    blk_cleanup_queue(blkram->queue);
#endif
    blk_mq_free_tag_set(&blkram->tag_set);
    vfree(blkram->data);
    kfree(blkram);
    unregister_blkdev(blkram_major, "blkram");
}

module_init(blkram_init);
module_exit(blkram_exit);

MODULE_DESCRIPTION("LKMPG blk-mq RAM disk example");
MODULE_LICENSE("GPL");
```

28 Network Drivers

Network drivers are different from both character and block drivers because they operate on packets, not byte streams or sectors, and because they are tightly coupled to the networking stack's queueing, routing, offload, and traffic control machinery. The central exported object is `struct net_device`, which represents a network interface visible to the rest of the kernel and to userspace tools such as `ip` and `ethtool`.

Most Ethernet-style drivers allocate a network device with helpers such as `alloc_etherdev()` or `alloc_etherdev_mqs()`, store private state behind it with `netdev_priv()`, initialize the operation tables, and register the device with `register_netdev()`. Once registered, the interface can be opened, configured, brought up and down, and asked to transmit packets by the networking core. For compatibility with Linux v5.10 through v6.17 and later, the example below uses `alloc_etherdev()` rather than relying on newer naming or queue-count allocation helpers.

28.1 net_device and netdev_ops

Modern drivers provide their main callbacks through `struct net_device_ops`. The precise set depends on the device class, but the core operations usually include open, stop, start-xmit, MTU changes, and timeout handling. On Linux 5.10 and later, this structure is the stable way to hook a driver into the networking stack; older callback field names embedded directly in `struct net_device` are mostly historical context.

Bringing an interface up in the open callback usually means enabling interrupts, allocating receive buffers, arming DMA rings, and telling the stack the transmit queue may start by calling `netif_start_queue()` or the multi-queue equivalent. Stopping the interface reverses that process and must ensure that the device can no longer DMA into freed memory or complete packets against torn-down state.

Transmit is not a blocking write path. The stack gives the driver an `sk_buff` and expects the driver to queue it to hardware quickly or stop the queue temporarily if resources are exhausted. Completion typically happens later in interrupt or NAPI context, where the driver reclaims descriptors, frees transmitted buffers, updates statistics, and wakes the queue if space is available again.

28.2 Socket Buffers and Receive Paths

The packet object used throughout the networking stack is `struct sk_buff`, usually called an `skb`. It carries packet data, protocol metadata, checksum information, queue state, and references to the underlying memory. Driver authors do not need to know every field, but they do need to understand the ownership rules: once a transmitted skb is handed to hardware, the driver must keep it alive until completion; once a received skb is submitted upward into the stack, the stack owns it.

Receive handling is where performance and correctness meet. High-rate devices usually keep rings of DMA-backed receive buffers, turn completed descriptors into skbs, annotate checksum/VLAN/hash metadata if the hardware provided it, and then feed the packets into the stack. The stack-facing handoff may use helpers such as `napi_gro_receive()` or related interfaces depending on the driver's receive model and offload support.

The important conceptual point is that packet reception is not just "copy bytes and notify userspace". The driver is cooperating with protocol layers, GRO, checksum offload handling, traffic classification, and often XDP or page-pool style buffer recycling on newer high-performance paths. That is why network drivers tend to be more subsystem-driven than ad hoc.

28.3 NAPI, Interrupt Moderation, and Throughput

Classic interrupt-per-packet designs do not scale well. Under heavy traffic they spend too much time bouncing in and out of interrupt context. Linux addresses

this with NAPI, which lets a driver disable or defer receive interrupts temporarily and poll for batches of packets in softirq context. Modern high-performance network drivers almost always rely on NAPI.

In practice, a receive interrupt usually does only enough work to schedule the NAPI poll function. That poll function then processes packets up to a budget, replenishes receive buffers, and re-enables interrupts when the queue has been drained. This reduces interrupt overhead while still preserving low latency when the system is mostly idle.

NAPI also interacts with queue topology. Many devices expose multiple transmit and receive queues so traffic can scale across CPUs. That is why helpers like `alloc_etherdev_mqs()` exist and why RSS, XPS, and interrupt affinity matter for real drivers. Once a driver enters this territory, performance tuning becomes as much about queue mapping and cache locality as about the hardware itself.

28.4 Offloads, Link State, and Userspace

Network devices often support checksum offload, TSO, GSO, GRO assistance, VLAN acceleration, timestamping, and other features. The driver advertises these through feature flags on the netdev and must report them honestly. An incorrect offload declaration is worse than no offload at all because it can silently corrupt traffic or confuse higher layers.

Link-state changes also matter. The driver should report carrier transitions with helpers such as `netif_carrier_on()` and `netif_carrier_off()` so the rest of the stack and userspace can react appropriately. Statistics, coalescing controls, ring sizing, and feature toggles are commonly surfaced through ethtool operations rather than a custom ioctl interface. That subsystem integration is the network-stack equivalent of using the input or block frameworks instead of inventing a private ABI.

The following example registers a virtual Ethernet device and loops transmitted packets back into its own receive path. It is not a performance driver, but it demonstrates the structure of a modern `net_device` example: `net_device_ops`, interface bring-up and shutdown, skb ownership, and userspace-visible statistics:

```
// SPDX-License-Identifier: GPL-2.0
/*
 * vnetloop.c - Minimal virtual Ethernet device that loops transmitted packets
 * back into the receive path.
 */

#include <linux/etherdevice.h>
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/netdevice.h>
#include <linux/skbuff.h>
#include <linux/u64_stats_sync.h>

struct vnetloop_priv {
```

```
    u64 tx_packets;
    u64 tx_bytes;
    u64 rx_packets;
    u64 rx_bytes;
    struct u64_stats_sync syncp;
};

static netdev_tx_t vnetloop_xmit(struct sk_buff *skb, struct net_device *dev)
{
    struct vnetloop_priv *priv = netdev_priv(dev);
    struct sk_buff *rx_skb;
    unsigned int len = skb->len;

    rx_skb = skb_copy(skb, GFP_ATOMIC);
    if (rx_skb) {
        rx_skb->dev = dev;
        skb_reset_mac_header(rx_skb);
        rx_skb->ip_summed = CHECKSUM_UNNECESSARY;
        rx_skb->protocol = eth_type_trans(rx_skb, dev);

        u64_stats_update_begin(&priv->syncp);
        priv->tx_packets++;
        priv->tx_bytes += len;
        priv->rx_packets++;
        priv->rx_bytes += rx_skb->len;
        u64_stats_update_end(&priv->syncp);

        netif_rx(rx_skb);
    } else {
        u64_stats_update_begin(&priv->syncp);
        priv->tx_packets++;
        priv->tx_bytes += len;
        u64_stats_update_end(&priv->syncp);
    }

    dev_kfree_skb(skb);

    return NETDEV_TX_OK;
}

static int vnetloop_open(struct net_device *dev)
{
    netif_carrier_on(dev);
    netif_start_queue(dev);

    return 0;
}

static int vnetloop_stop(struct net_device *dev)
{
    netif_stop_queue(dev);
    netif_carrier_off(dev);

    return 0;
}

static void vnetloop_get_stats64(struct net_device *dev,
```

```
                                  struct rtnl_link_stats64 *stats)
{
    struct vnetloop_priv *priv = netdev_priv(dev);
    unsigned int start;

    do {
        start = u64_stats_fetch_begin(&priv->syncp);
        stats->tx_packets = priv->tx_packets;
        stats->tx_bytes = priv->tx_bytes;
        stats->rx_packets = priv->rx_packets;
        stats->rx_bytes = priv->rx_bytes;
    } while (u64_stats_fetch_retry(&priv->syncp, start));
}

static const struct net_device_ops vnetloop_netdev_ops = {
    .ndo_open = vnetloop_open,
    .ndo_stop = vnetloop_stop,
    .ndo_start_xmit = vnetloop_xmit,
    .ndo_get_stats64 = vnetloop_get_stats64,
};

static void vnetloop_setup(struct net_device *dev)
{
    dev->netdev_ops = &vnetloop_netdev_ops;
    dev->flags |= IFF_NOARP;
    dev->features |= NETIF_F_HW_CSUM;
    eth_hw_addr_random(dev);
}

static struct net_device *vnetloop_dev;

static int __init vnetloop_init(void)
{
    int ret;

    vnetloop_dev = alloc_etherdev(sizeof(struct vnetloop_priv));
    if (!vnetloop_dev)
        return -ENOMEM;

    strscpy(vnetloop_dev->name, "vnetloop%d", IFNAMSIZ);
    vnetloop_setup(vnetloop_dev);

    ret = register_netdev(vnetloop_dev);
    if (ret) {
        free_netdev(vnetloop_dev);
        return ret;
    }

    pr_info("vnetloop: registered %s\n", vnetloop_dev->name);

    return 0;
}

static void __exit vnetloop_exit(void)
{
    unregister_netdev(vnetloop_dev);
    free_netdev(vnetloop_dev);
```

```
}

module_init(vnetloop_init);
module_exit(vnetloop_exit);

MODULE_DESCRIPTION("LKMPG virtual net_device example");
MODULE_LICENSE("GPL");
```

29 Standardizing the interfaces: The Device Model

Up to this point we have seen all kinds of modules doing all kinds of things, but there was no consistency in their interfaces with the rest of the kernel. To impose some consistency such that there is at minimum a standardized way to start, suspend and resume a device model was added. An example is shown below, and you can use this as a template to add your own suspend, resume or other interface functions.

```
/*
 * devicemodel.c
 */
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/platform_device.h>
#include <linux/version.h>

struct devicemodel_data {
    char *greeting;
    int number;
};

static int devicemodel_probe(struct platform_device *dev)
{
    struct devicemodel_data *pd =
        (struct devicemodel_data *)(dev->dev.platform_data);

    pr_info("devicemodel probe\n");
    pr_info("devicemodel greeting: %s; %d\n", pd->greeting, pd->number);

    /* Your device initialization code */

    return 0;
}

#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 11, 0)
static void devicemodel_remove(struct platform_device *dev)
{
    pr_info("devicemodel example removed\n");
    /* Your device removal code */
}
#else
static int devicemodel_remove(struct platform_device *dev)
{
    pr_info("devicemodel example removed\n");
```

```
37      /* Your device removal code */
38      return 0;
39  }
40  #endif
41
42  static int devicemodel_suspend(struct device *dev)
43  {
44      pr_info("devicemodel example suspend\n");
45
46      /* Your device suspend code */
47
48      return 0;
49  }
50
51  static int devicemodel_resume(struct device *dev)
52  {
53      pr_info("devicemodel example resume\n");
54
55      /* Your device resume code */
56
57      return 0;
58  }
59
60  static const struct dev_pm_ops devicemodel_pm_ops = {
61      .suspend = devicemodel_suspend,
62      .resume = devicemodel_resume,
63      .poweroff = devicemodel_suspend,
64      .freeze = devicemodel_suspend,
65      .thaw = devicemodel_resume,
66      .restore = devicemodel_resume,
67  };
68
69  static struct platform_driver devicemodel_driver = {
70      .driver =
71          {
72              .name = "devicemodel_example",
73              .pm = &devicemodel_pm_ops,
74          },
75      .probe = devicemodel_probe,
76      .remove = devicemodel_remove,
77  };
78
79  static int __init devicemodel_init(void)
80  {
81      int ret;
82
83      pr_info("devicemodel init\n");
84
85      ret = platform_driver_register(&devicemodel_driver);
86
87      if (ret) {
88          pr_err("Unable to register driver\n");
89          return ret;
90      }
91
92      return 0;
93  }
```

```
94
95  static void __exit devicemodel_exit(void)
96  {
97      pr_info("devicemodel exit\n");
98      platform_driver_unregister(&devicemodel_driver);
99  }
100
101 module_init(devicemodel_init);
102 module_exit(devicemodel_exit);
103
104 MODULE_LICENSE("GPL");
105 MODULE_DESCRIPTION("Linux Device Model example");
```

30 Device Tree

30.1 Introduction to Device Tree

Device Tree is a data structure that describes hardware components in a system, particularly in embedded systems and ARM-based platforms. Instead of hard-coding hardware details in the kernel source, Device Tree provides a separate, human-readable description that the kernel can parse at boot time. This separation allows the same kernel binary to support multiple hardware platforms, making development and maintenance significantly easier.

Device Tree files (with `.dts` extension for source files and `.dtb` for compiled binary files) use a hierarchical structure similar to a filesystem to represent the hardware topology. Each hardware component is represented as a node with properties that describe its characteristics, such as memory addresses, interrupt numbers, and device-specific parameters.

30.2 Device Tree and Kernel Modules

While Device Tree is primarily used during kernel initialization, kernel modules can also interact with Device Tree nodes through the platform device framework. When the kernel parses the Device Tree at boot, it creates platform devices for nodes that have compatible strings. Kernel modules can then register platform drivers that match these compatible strings, allowing them to be automatically probed when the corresponding hardware is detected.

The key concepts for Device Tree interaction in kernel modules include:

- **Compatible strings**: Unique identifiers that match Device Tree nodes to their drivers
- **Property reading**: Functions to extract configuration data from Device Tree nodes
- **Platform driver framework**: Infrastructure for binding drivers to devices described in Device Tree

- **Device-specific data**: Custom properties that can be defined for specific hardware

30.3 Example: Device Tree Module

The following example demonstrates how a kernel module can interact with Device Tree nodes. This module registers a platform driver that matches specific compatible strings and extracts properties from the matched Device Tree nodes.

```
/* devicetree.c - Demonstrates device tree interaction with kernel modules */

#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/of.h>
#include <linux/of_device.h>
#include <linux/platform_device.h>
#include <linux/version.h>

#define DRIVER_NAME "lkmpg_devicetree"

/* Structure to hold device-specific data */
struct dt_device_data {
    const char *label;
    u32 reg_value;
    u32 custom_value;
    bool has_clock;
};

/* Probe function - called when device tree node matches */
static int dt_probe(struct platform_device *pdev)
{
    struct device *dev = &pdev->dev;
    struct device_node *np = dev->of_node;
    struct dt_device_data *data;
    const char *string_prop;
    u32 value;
    int ret;

    pr_info("%s: Device tree probe called for %s\n", DRIVER_NAME,
            np->full_name);

    /* Allocate memory for device data */
    data = devm_kzalloc(dev, sizeof(*data), GFP_KERNEL);
    if (!data)
        return -ENOMEM;

    /* Read a string property */
    ret = of_property_read_string(np, "label", &string_prop);
    if (ret == 0) {
        data->label = string_prop;
        pr_info("%s: Found label property: %s\n", DRIVER_NAME, data->label);
    } else {
        data->label = "unnamed";
        pr_info("%s: No label property found, using default\n", DRIVER_NAME);
```

```
47      }
48
49      /* Read a u32 property */
50      ret = of_property_read_u32(np, "reg", &value);
51      if (ret == 0) {
52          data->reg_value = value;
53          pr_info("%s: Found reg property: 0x%x\n", DRIVER_NAME,
            ↪  data->reg_value);
54      }
55
56      /* Read a custom u32 property */
57      ret = of_property_read_u32(np, "lkmpg,custom-value", &value);
58      if (ret == 0) {
59          data->custom_value = value;
60          pr_info("%s: Found custom-value property: %u\n", DRIVER_NAME,
61                  data->custom_value);
62      } else {
63          data->custom_value = 42; /* Default value */
64          pr_info("%s: No custom-value found, using default: %u\n", DRIVER_NAME,
65                  data->custom_value);
66      }
67
68      /* Check for presence of a property */
69      data->has_clock = of_property_read_bool(np, "lkmpg,has-clock");
70      pr_info("%s: has-clock property: %s\n", DRIVER_NAME,
71              data->has_clock ? "present" : "absent");
72
73      /* Store device data for later use */
74      platform_set_drvdata(pdev, data);
75
76      pr_info("%s: Device probe successful\n", DRIVER_NAME);
77      return 0;
78  }
79
80  /* Remove function - called when device is removed */
81  #if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 11, 0)
82  static void dt_remove(struct platform_device *pdev)
83  {
84      struct dt_device_data *data = platform_get_drvdata(pdev);
85
86      pr_info("%s: Removing device %s\n", DRIVER_NAME, data->label);
87      /* Cleanup is handled automatically by devm_* functions */
88  }
89  #else
90  static int dt_remove(struct platform_device *pdev)
91  {
92      struct dt_device_data *data = platform_get_drvdata(pdev);
93
94      pr_info("%s: Removing device %s\n", DRIVER_NAME, data->label);
95      /* Cleanup is handled automatically by devm_* functions */
96      return 0;
97  }
98  #endif
99
100 /* Device tree match table - defines compatible strings this driver supports
    ↪  */
101 static const struct of_device_id dt_match_table[] = {
```

```
    {
        .compatible = "lkmpg,example-device",
    },
    {
        .compatible = "lkmpg,another-device",
    },
    {} /* Sentinel */
};
MODULE_DEVICE_TABLE(of, dt_match_table);

/* Platform driver structure */
static struct platform_driver dt_driver = {
    .probe = dt_probe,
    .remove = dt_remove,
    .driver = {
        .name = DRIVER_NAME,
        .of_match_table = dt_match_table,
    },
};

/* Module initialization */
static int __init dt_init(void)
{
    int ret;

    pr_info("%s: Initializing device tree example module\n", DRIVER_NAME);

    /* Register the platform driver */
    ret = platform_driver_register(&dt_driver);
    if (ret) {
        pr_err("%s: Failed to register platform driver\n", DRIVER_NAME);
        return ret;
    }

    pr_info("%s: Module loaded successfully\n", DRIVER_NAME);
    return 0;
}

/* Module cleanup */
static void __exit dt_exit(void)
{
    pr_info("%s: Cleaning up device tree example module\n", DRIVER_NAME);
    platform_driver_unregister(&dt_driver);
}

module_init(dt_init);
module_exit(dt_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Device tree interaction example for LKMPG");
```

30.4 Device Tree Source Example

To use the above module, you would need a Device Tree entry like this:

```
/* Example device tree fragment */
lkmpg_device@0 {
    compatible = "lkmpg,example-device";
    reg = <0x40000000 0x1000>;
    label = "LKMPG Test Device";
    lkmpg,custom-value = <100>;
    lkmpg,has-clock;
};
```

The properties in this Device Tree node would be read by the module's probe function when the device is matched. The `compatible` property is used to match the device with the driver, while other properties provide device-specific configuration.

30.5 Testing Device Tree Modules

Testing Device Tree modules can be done in several ways:

1. **Using Device Tree overlays**: On systems that support it (like Raspberry Pi), you can load Device Tree overlays at runtime to add new devices without rebooting.
2. **Modifying the main Device Tree**: Add your device nodes to the system's main Device Tree source file and recompile it.
3. **Using QEMU**: For development and testing, QEMU can emulate systems with custom Device Trees, allowing you to test your modules without physical hardware. The `devtools/boot.sh` script (see Section 1.8) provides a ready-made QEMU environment where you can load and test modules interactively.

To check if your device was properly detected, you can examine the sysfs filesystem:

```
# List all platform devices
ls /sys/bus/platform/devices/

# Check device tree nodes
ls /proc/device-tree/
```

30.6 Common Device Tree Functions

Here are some commonly used Device Tree functions in kernel modules:

- `of_property_read_string()` - Read a string property

- `of_property_read_u32()` - Read a 32-bit integer property
- `of_property_read_bool()` - Check if a boolean property exists
- `of_find_property()` - Find a property by name
- `of_get_property()` - Get a property's raw value
- `of_match_device()` - Match a device against a match table
- `of_parse_phandle()` - Parse a phandle reference to another node

These functions provide a robust interface for extracting configuration data from Device Tree nodes, allowing modules to be highly configurable without code changes.

31 Optimizations

31.1 Likely and Unlikely conditions

Sometimes you might want your code to run as quickly as possible, especially if it is handling an interrupt or doing something which might cause noticeable latency. If your code contains boolean conditions and if you know that the conditions are almost always likely to evaluate as either `true` or `false`, then you can allow the compiler to optimize for this using the `likely` and `unlikely` macros. For example, when allocating memory you are almost always expecting this to succeed.

```
bvl = bvec_alloc(gfp_mask, nr_iovecs, &idx);
if (unlikely(!bvl)) {
    mempool_free(bio, bio_pool);
    bio = NULL;
    goto out;
}
```

When the `unlikely` macro is used, the compiler alters its machine instruction output, so that it continues along the false branch and only jumps if the condition is true. That avoids flushing the processor pipeline. The opposite happens if you use the `likely` macro.

31.2 Static keys

Static keys allow us to enable or disable kernel code paths based on the runtime state of a key. Their APIs have been available since 2010 (most architectures are already supported) and use self-modifying code to eliminate the overhead of cache and branch prediction. The most typical use case of static keys is for performance-sensitive kernel code, such as tracepoints, context switching, networking, etc. These hot paths of the kernel often contain branches and can

be optimized easily using this technique. Before we can use static keys in the kernel, we need to make sure that gcc supports `asm goto` inline assembly, and the following kernel configurations are set:

```
CONFIG_JUMP_LABEL=y
CONFIG_HAVE_ARCH_JUMP_LABEL=y
CONFIG_HAVE_ARCH_JUMP_LABEL_RELATIVE=y
```

To declare a static key, we need to define a global variable using the `DEFINE_STATIC_KEY_FALSE` or `DEFINE_STATIC_KEY_TRUE` macro defined in include/linux/jump_label.h. This macro initializes the key with the given initial value, which is either false or true, respectively. For example, to declare a static key with an initial value of false, we can use the following code:

```
DEFINE_STATIC_KEY_FALSE(fkey);
```

Once the static key has been declared, we need to add branching code to the module that uses the static key. For example, the code includes a fastpath, where a no-op instruction will be generated at compile time as the key is initialized to false and the branch is unlikely to be taken.

```
pr_info("fastpath 1\n");
if (static_branch_unlikely(&fkey))
    pr_alert("do unlikely thing\n");
pr_info("fastpath 2\n");
```

If the key is enabled at runtime by calling `static_branch_enable(&fkey)`, the fastpath will be patched with an unconditional jump instruction to the slowpath code `pr_alert`, so the branch will always be taken until the key is disabled again.

The following kernel module derived from `chardev.c`, demonstrates how the static key works.

```
/*
 * static_key.c
 */

#include <linux/atomic.h>
#include <linux/device.h>
#include <linux/fs.h>
#include <linux/kernel.h> /* for sprintf() */
#include <linux/module.h>
#include <linux/printk.h>
#include <linux/types.h>
#include <linux/uaccess.h> /* for get_user and put_user */
#include <linux/jump_label.h> /* for static key macros */
#include <linux/version.h>

```

```c
#include <asm/errno.h>

static int device_open(struct inode *inode, struct file *file);
static int device_release(struct inode *inode, struct file *file);
static ssize_t device_read(struct file *file, char __user *buf, size_t count,
                           loff_t *ppos);
static ssize_t device_write(struct file *file, const char __user *buf,
                            size_t count, loff_t *ppos);

#define DEVICE_NAME "key_state"
#define BUF_LEN 10

static int major;

enum {
    CDEV_NOT_USED,
    CDEV_EXCLUSIVE_OPEN,
};

static atomic_t already_open = ATOMIC_INIT(CDEV_NOT_USED);

static char msg[BUF_LEN + 1];

static struct class *cls;

static DEFINE_STATIC_KEY_FALSE(fkey);

static struct file_operations chardev_fops = {
    .owner = THIS_MODULE,
    .open = device_open,
    .release = device_release,
    .read = device_read,
    .write = device_write,
};

static int __init chardev_init(void)
{
    major = register_chrdev(0, DEVICE_NAME, &chardev_fops);
    if (major < 0) {
        pr_alert("Registering char device failed with %d\n", major);
        return major;
    }

    pr_info("I was assigned major number %d\n", major);

#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 4, 0)
    cls = class_create(DEVICE_NAME);
#else
    cls = class_create(THIS_MODULE, DEVICE_NAME);
#endif
    if (IS_ERR(cls)) {
        pr_err("Failed to create class for device\n");
        unregister_chrdev(major, DEVICE_NAME);
        return PTR_ERR(cls);
    }
    device_create(cls, NULL, MKDEV(major, 0), NULL, DEVICE_NAME);

```

```
73      pr_info("Device created on /dev/%s\n", DEVICE_NAME);
74
75      return 0;
76  }
77
78  static void __exit chardev_exit(void)
79  {
80      device_destroy(cls, MKDEV(major, 0));
81      class_destroy(cls);
82
83      /* Unregister the device */
84      unregister_chrdev(major, DEVICE_NAME);
85  }
86
87  /* Methods */
88
89  /**
90   * Called when a process tried to open the device file, like
91   * cat /dev/key_state
92   */
93  static int device_open(struct inode *inode, struct file *file)
94  {
95      if (atomic_cmpxchg(&already_open, CDEV_NOT_USED, CDEV_EXCLUSIVE_OPEN))
96          return -EBUSY;
97
98      sprintf(msg, static_key_enabled(&fkey) ? "enabled\n" : "disabled\n");
99
100     pr_info("fastpath 1\n");
101     if (static_branch_unlikely(&fkey))
102         pr_alert("do unlikely thing\n");
103     pr_info("fastpath 2\n");
104
105     return 0;
106 }
107
108 /**
109  * Called when a process closes the device file
110  */
111 static int device_release(struct inode *inode, struct file *file)
112 {
113     /* We are now ready for our next caller. */
114     atomic_set(&already_open, CDEV_NOT_USED);
115
116     return 0;
117 }
118
119 /**
120  * Called when a process, which already opened the dev file, attempts to
121  * read from it.
122  */
123 static ssize_t device_read(struct file *filp, /* see include/linux/fs.h */
124                            char __user *buffer, /* buffer to fill with data */
125                            size_t length, /* length of the buffer */
126                            loff_t *offset)
127 {
128     /* Number of the bytes actually written to the buffer */
129     int bytes_read = 0;
```

```
130     const char *msg_ptr = msg;
131
132     if (!*(msg_ptr + *offset)) { /* We are at the end of the message */
133         *offset = 0; /* reset the offset */
134         return 0; /* signify end of file */
135     }
136
137     msg_ptr += *offset;
138
139     /* Actually put the data into the buffer */
140     while (length && *msg_ptr) {
141         /**
142          * The buffer is in the user data segment, not the kernel
143          * segment so "*" assignment won't work. We have to use
144          * put_user which copies data from the kernel data segment to
145          * the user data segment.
146          */
147         put_user(*(msg_ptr++), buffer++);
148         length--;
149         bytes_read++;
150     }
151
152     *offset += bytes_read;
153
154     /* Most read functions return the number of bytes put into the buffer. */
155     return bytes_read;
156 }
157
158 /* Called when a process writes to dev file; echo "enable" > /dev/key_state */
159 static ssize_t device_write(struct file *filp, const char __user *buffer,
160                             size_t length, loff_t *offset)
161 {
162     char command[10];
163
164     if (length > 10) {
165         pr_err("command exceeded 10 char\n");
166         return -EINVAL;
167     }
168
169     if (copy_from_user(command, buffer, length))
170         return -EFAULT;
171
172     if (strncmp(command, "enable", strlen("enable")) == 0)
173         static_branch_enable(&fkey);
174     else if (strncmp(command, "disable", strlen("disable")) == 0)
175         static_branch_disable(&fkey);
176     else {
177         pr_err("Invalid command: %s\n", command);
178         return -EINVAL;
179     }
180
181     /* Again, return the number of input characters used. */
182     return length;
183 }
184
185 module_init(chardev_init);
186 module_exit(chardev_exit);
```

```
187
188 MODULE_LICENSE("GPL");
```

To check the state of the static key, we can use the `/dev/key_state` interface.

```
1 cat /dev/key_state
```

This will display the current state of the key, which is disabled by default.

To change the state of the static key, we can perform a write operation on the file:

```
1 echo enable > /dev/key_state
```

This will enable the static key, causing the code path to switch from the fastpath to the slowpath.

In some cases, the key is enabled or disabled at initialization and never changed, we can declare a static key as read-only, which means that it can only be toggled in the module init function. To declare a read-only static key, we can use the `DEFINE_STATIC_KEY_FALSE_RO` or `DEFINE_STATIC_KEY_TRUE_RO` macro instead. Attempts to change the key at runtime will result in a page fault. For more information, see Static keys

32 Common Pitfalls

32.1 Using standard libraries

You can not do that. In a kernel module, you can only use kernel functions which are the functions you can see in `/proc/kallsyms`.

32.2 Disabling interrupts

You might need to do this for a short time and that is OK, but if you do not enable them afterwards, your system will be stuck and you will have to power it off.

32.3 Kernel stack is small

Kernel stack size is architecture- and configuration-dependent, and is much smaller than the megabytes available to user-space processes. On many systems it is on the order of a few pages (for example, often 8 KiB or 16 KiB), and some architectures use separate per-CPU IRQ stacks so interrupt frames do not always share the task stack. Either way, every function call and local variable consumes scarce kernel stack space. If you declare a large array on the stack or recurse too deeply, you will silently corrupt adjacent memory or trigger

a kernel panic. Use `kmalloc()` or `kzalloc()` for any buffer larger than a few hundred bytes, and keep function nesting shallow. Static analysis tools such as `gcc -fstack-usage` or `scripts/checkstack.pl` in the kernel tree can help spot stack-heavy functions.

32.4 No floating point in the kernel

Kernel code must not use floating-point arithmetic. Ordinary kernel code cannot assume that FPU/SIMD state is available or may be used freely, so using `float` or `double` variables without the proper kernel helpers can corrupt user-space state or break kernel execution. If you need fixed-point math or division, use integer arithmetic with appropriate scaling. In the rare case where FPU use is genuinely required (e.g. certain crypto or media code), bracket it with `kernel_fpu_begin()` and `kernel_fpu_end()`, but only from process context, never from interrupt context.

32.5 Initialize buffers before copying to user space

When a driver copies data to user space via `copy_to_user()` or similar, every byte of the source buffer must be initialized. If the buffer contains uninitialized padding bytes (common with structs due to alignment) or leftover data from a previous allocation, that kernel memory leaks to user space. This is an information-disclosure vulnerability. Use `kzalloc()` instead of `kmalloc()`, or zero out padding with `memset()`, before copying data out.

32.6 Double-underscore functions are internal

Functions and macros prefixed with double underscores (e.g. `__kmalloc()`, `__list_add()`) are lower-level interfaces intended for specific internal use cases. They may assume preconditions that the public wrapper documents or enforces, such as caller-held locks or pre-validated arguments. The exact difference is API-specific, so always prefer the non-underscored wrapper (e.g. `kmalloc()`, `list_add()`) unless the kernel documentation explicitly calls for the underscore variant in your situation.

33 Secure Boot Signing Guide

33.1 Install Dependencies

You need to install `mokutil`, `kmod`, and `openssl`.

```
# Debian/Ubuntu
sudo apt install mokutil kmod openssl
```

33.2 Generate a MOK Key Pair

Create a private and a public certificate (DER format) to sign your modules:

```
cat << 'EOF' > mokconfig.cnf
[ req ]
default_bits       = 2048
distinguished_name = req_distinguished_name
prompt             = no
x509_extensions    = myexts

[ req_distinguished_name ]
CN = your_kernel_module MOK Signing Key

[ myexts ]
basicConstraints=CA:FALSE
keyUsage=digitalSignature
subjectKeyIdentifier=hash
authorityKeyIdentifier=keyid
EOF

openssl req -new -x509 -newkey rsa:2048 -nodes -days 3650
↪  -outform DER -keyout MOK.priv -out MOK.der -config
↪  mokconfig.cnf
```

33.3 Enroll the Key with MOKutil

Register your new public key with the shim bootloader. You will be prompted to create a temporary password for the next step.

```
sudo mokutil --import MOK.der
```

33.4 Complete Enrollment in UEFI

1. **Reboot** your computer.
2. Upon booting, the blue **MOK Manager** screen will automatically appear.
3. Select **Enroll MOK** -> **View key** to verify your key details, then confirm the enrollment.

4. Enter the temporary **password** you created in Step 3.
5. Continue to boot into Linux.

33.5 Sign Your Kernel Module

```
# Sign your kernel module.
sudo /usr/src/linux-headers-`uname -r`/scripts/sign-file sha256
↪  MOK.priv MOK.der /path/to/your_kernel_module.ko

# Optional: verify the signature is present.
modinfo /path/to/your_kernel_module.ko | grep -i "signer"
```

34 Where To Go From Here?

For those deeply interested in kernel programming, kernelnewbies.org and the Documentation subdirectory within the kernel source code are highly recommended. Although the latter may not always be straightforward, it serves as a valuable initial step for further exploration. Echoing Linus Torvalds' perspective, the most effective method to understand the kernel is through personal examination of the source code.

Contributions to this guide are welcome, especially if there are any significant inaccuracies identified. To contribute or report an issue, please initiate an issue at `https://github.com/sysprog21/lkmpg`. Pull requests are greatly appreciated.

Happy hacking!